GOOGLE | ALL SOURCES | TOPIC 2 OF 10 | COMMON TOPIC

Arrays, Hashing and Two Pointers

38 questions to study here, each on its own page, with the full interview answer and code

Nothing to skip: every question that fits this topic is covered here

8	26	4
EASY	MEDIUM	HARD

How to use

- Questions run from Easy to Medium to Hard, newest last-asked date first inside each level.
- PDFs run from the one with the most questions to the fewest. A question has pages in only one PDF; the others list it as SKIP
- Every page has < Prev | Contents | Next > at the bottom. Tap a title in the Contents to jump to it.
- Select code and press Cmd/Ctrl+C to copy it.

Go to Contents >

Contents

Arrays, Hashing and Two Pointers | 38 questions | tap a title to jump to it

E	1. Robot Status Message Deduplication Window	1Point3Acres	2026-09-10
E	2. Reverse a linked list.	Exponent	
E	3. Sort a binary array in linear time.	Exponent	
E	4. Is this a valid palindrome?	Exponent	
E	5. Find the Duplicates	Exponent	
E	6. Reverse a Sentence	Exponent	
E	7. Determine if two words are sorted in lexicographic order.	Exponent	
E	8. Parity Alternating Subarray Query	HackTheRounds	
M	9. Connected Crop Arrangement on a Grid	1Point3Acres	2026-10-04
M	10. Range Duplicate Check with Constant-Time Queries	1Point3Acres	2026-09-29
M	11. Temperature Monitoring Data Structure	1Point3Acres	2026-09-10
M	12. Longest Same-Color Grid Sequence	1Point3Acres	2026-09-08
M	13. Next-Word Predictor	1Point3Acres	2026-09-02
M	14. Piano Hand Movement Grouping	1Point3Acres	2026-07-14
M	15. Filter Repeated Robot Status Messages	PracHub	2026-07-05
M	16. Website Activity Analytics: Unique Users + Session Time	1Point3Acres	2026-06-26
M	17. Find a Horizontal Cut That Bisects Rectangle Area	PracHub	2026-06-04
M	18. Gold Chain Split into Two Equal Halves	1Point3Acres	2026-04-21
M	19. Solve several coding interview problems	PracHub	2026-03-04
M	20. Solve array-sum and city-community problems	PracHub	2026-02-20
M	21. Check pivot after removing one element	PracHub	2026-02-12
M	22. Min deletions to avoid overlap in first k	PracHub	2026-02-12
M	23. Sum of Good Subarrays (Adjacent Diff ±1)	1Point3Acres	2026-02-08
M	24. Run-Length Vector Storage + Dot Product	1Point3Acres	2026-01-21
M	25. Reconstruct original array from doubled shuffle	PracHub	2026-01-04
M	26. Maximize 1D deliveries within distance and minimize total distance	PracHub	2025-12-27
M	27. Two-Day Log Intersection under Memory Limit	1Point3Acres	2025-11-27
M	28. Add two big integers from digit lists	PracHub	2025-11-24
M	29. Minimize calls to find all bad test pairs	PracHub	2025-11-05
M	30. Rotate Any m×n 2D Matrix	1Point3Acres	
M	31. Find the minimum window substring.	Exponent	
M	32. Given positions of N kids and M ice cream sellers on a 1D line, find the smallest distance E such that every ki...	Exponent	
M	33. Shortest Subarray with K Distinct Integers	HackTheRounds	
M	34. Sum of Two Squares in Two Ways	HackTheRounds	
H	35. Maximum-Area Rectangle from Planar Points	1Point3Acres	2026-09-29
H	36. Smallest Missing Positive Integer in Linear Time and Constant Extra Space	PracHub	2026-09-09
H	37. Necklace Cut into Two Halves with Equal D/R Counts	1Point3Acres	2026-05-03
H	38. Find longest duplicated substring	PracHub	2025-12-08

STUDY HERE: topic 2, Arrays, Hashing and Two Pointers

Site tags: hashmap, simulation, easy

EASY | 1Point3Acres | onsite-coding | 1 of 38

Robot Status Message Deduplication Window

ASKED 2026-09-10 | 40 DAYS AGO | LAST 90 DAYS

STUDY: TYPE ALL TIME 51.2 30 DAYS 33 PRIORITY 34/228

ABOUT THIS QUESTION

Topic: Arrays, Hashing and Two Pointers

Optimal answer from: written and tested here for this exact question

Related in your app (a neighbouring problem, not this exact question): LeetCode #359 Logger Rate Limiter

LeetCode match: LC 359 Logger Rate Limiter (related)

Frequency: Low | Last asked: 2026-09-10 | Roles: SWE

Original: https://www.1point3acres.com/interview/problems/company/google/robot-status-message-deduplication

QUESTION

Process timestamped robot messages and display a message only when it has not been displayed within the prior 10 seconds. Hidden arrivals do not refresh the last-displayed timestamp, and the exact 10-second boundary must be clarified.

Process a stream of (time, message) status events from a robot.
If the same message was displayed within the past 10 seconds, hide the new event.
Otherwise display it and update that message's last-displayed time.
Track the last time each message was displayed, not the last time it was received. A hidden duplicate must not refresh the window.
Clarify whether an event exactly 10 seconds after the prior display is allowed: the comparison may be > 10 or >= 10.

Logger API variant

Implement shouldPrintMessage(timestamp, message), returning true when the message has not been printed in the past 10 seconds and false otherwise.
Handle timestamps arriving out of order, including timestamp 12 arriving before timestamp 10.

EXAMPLES

```
time=10, message="solar panel activated" -> show
time=13, message="solar panel activated" -> hide
time=21, message="solar panel activated" -> show
```

The event at 21 compares against the display at 10, not the hidden arrival at 13.

APPROACH

Implement the stream processor with per-message state, then trace repeated and interleaved messages by hand.
Write tests for the first occurrence, an event just inside the window, exactly at the boundary, just outside it, and a hidden event followed by an allowed display.

HOW TO THINK ABOUT IT

The state that matters is the last time each message was displayed, not the last time it arrived. A message that is hidden must not move that time, otherwise a message repeated every few seconds would be hidden forever. A hash map from message to last displayed time answers each event in O(1).

Basis: LeetCode 359, Logger Rate Limiter (a message may be shown again only after the window has passed).

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
shown = []  # (time, message) displayed
def on_event(t, msg):
    for t0, m0 in shown:
        if m0 == msg and t - t0 < 10:
            return False  # hidden
    shown.append((t, msg))
    return True  # displayed
```

TIME	SPACE
O(displayed) per event	O(displayed)

WHY IT IS SLOW

Each event scans the whole history. A map from message to its last displayed time answers each event in O(1).

OPTIMAL SOLUTION · Hash map of last displayed time

TIME	SPACE
O(1) per event	O(distinct messages)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Events can arrive out of order.

Decide the contract first: either a late event is judged against the stored time with abs(t - last), or you keep a sorted history per message and compare with the nearest displayed neighbours on both sides. The simple map is only correct for a chronological stream.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Robot Status Message Deduplication Window
#
# Process timestamped robot messages and display a message
# only when it has not been displayed within the prior 10
# seconds. Hidden arrivals do not refresh the last-
# displayed timestamp, and the exact 10-second boundary
# must be clarified. Process a stream of (time, message)
# status events from a robot. If the same message was
# displayed within the past 10 seconds, hide the new
# event. Otherwise display it and update that message's
# last-displayed time. Track the last time each message
# was displayed, not the last time it was received. A
# hidden duplicate must not refresh the window. Clarify
# whether an event exactly 10 seconds after the prior
# display is allowed: the comparison may be > 10 or >= 10.
# Logger API variant Implement
# shouldPrintMessage(timestamp, message), returning true
# when the message has not been printed in the past 10
# seconds and false otherwise.
#
# FOLLOW-UPS (answered below, in this order):
# 1. Events can arrive out of order.
#
# INTERVIEW NOTES
# The state that matters is the last time each message was
# displayed, not the last time it arrived. A message that
# is hidden must not move that time, otherwise a message
# repeated every few seconds would be hidden forever. A
# hash map from message to last displayed time answers
# each event in O(1).
#
# BRUTE FORCE (say it first): Keep every displayed time
# and scan them
# Store the list of every (time, message) shown so far and
# scan it for the same message within the window. Correct,
# but the scan grows with the whole history. Time
# O(displayed) per event. Space O(displayed).


from bisect import bisect_left, bisect_right, insort


# BRUTE FORCE (say it first): Keep every displayed time
# and scan them | time O(displayed) per event
class DedupBrute:
    def __init__(self, window=10):
        self.window, self.shown = window, []

    def show(self, t, msg):
        if any(
            m == msg and t - s < self.window
            for s, m in self.shown
        ):
            return False
        self.shown.append((t, msg))
        return True


# BASE QUESTION (optimal): Hash map of last displayed time
# Idea: If the message was displayed less than `window`
# seconds ago hide it without changing the stored time;
# otherwise display it and store the new time.
class Dedup:
    """Chronological stream: remember the last display."""

    def __init__(self, window=10, boundary_allowed=True):
        self.window, self.last = window, {}
        # True: exactly `window` seconds later may show
        self.boundary_allowed = boundary_allowed

    def too_soon(self, gap):
        # is this gap still inside the hiding window?
        if self.boundary_allowed:
            return gap < self.window
        return gap <= self.window

    def show(self, t, msg):
        # only a displayed event starts a new window
        last = self.last.get(msg)
        if last is not None and self.too_soon(t - last):
            return False
        self.last[msg] = t
        return True


# FOLLOW-UP 1: Events can arrive out of order.
# Answer: Decide the contract first: either a late event
# is judged against the stored time with abs(t - last), or
# you keep a sorted history per message and compare with
# the nearest displayed neighbours on both sides.
class DedupOutOfOrder:
    """Events may arrive late: look at both neighbours."""

    def __init__(self, window=10):
        self.window = window
        # message -> sorted list of displayed times
        self.shown = {}

    def show(self, t, msg):
        times = self.shown.setdefault(msg, [])
        # any display within `window` on either side?
        lo = bisect_right(times, t - self.window)
        hi = bisect_left(times, t + self.window)
        if lo < hi:
            return False
        insort(times, t)
        return True


if __name__ == "__main__":
    d = Dedup()
    # 13 is hidden and does not move the window
    assert [d.show(t, "solar") for t in (10, 13, 21)] == [
        True, False, True]
    # exactly 10 seconds later shows by default
    d = Dedup()
    d.show(0, "a")
    assert d.show(10, "a")
    # ... but is hidden if the boundary is not allowed
    d = Dedup(boundary_allowed=False)
    d.show(0, "a")
    assert not d.show(10, "a")
    o = DedupOutOfOrder()
    assert o.show(20, "x")
    # a late event close to a displayed one is hidden
    assert not o.show(15, "x")
    # a late event far enough from every display shows
    assert o.show(5, "x")
```

NOTES FROM CANDIDATES

The state transition occurs only on a displayed event. Updating on every receipt changes the semantics and can suppress a frequently repeated message forever.
Resolve the exact-boundary rule before coding and make the comparison match that contract.

For out-of-order arrival, clarify how the 10-second interval applies to late events and whether an earlier print decision can ever be revisited. These semantics are not specified by the logger variant; do not assume the chronological-stream contract resolves them.

PREPARATION

Implement the stream processor with per-message state, then trace repeated and interleaved messages by hand.
Write tests for the first occurrence, an event just inside the window, exactly at the boundary, just outside it, and a hidden event followed by an allowed display.

STUDY HERE: topic 2, Arrays, Hashing and Two Pointers

EASY | IN APP LC 206 | Exponent | 2 of 38

Reverse a linked list.

ASKED (ANY CO.) ~4 MONTHS AGO | LAST 6 MONTHS

STUDY: TYPE ALL TIME 59.8 30 DAYS 0 PRIORITY 36/228

ABOUT THIS QUESTION

Topic: Arrays, Hashing and Two Pointers

Optimal answer from: your app's interview approach (LeetCode #206)

LeetCode match: LC 206 Reverse Linked List

Last asked (any company): 4 months ago, about 2026-06 (Exponent shows only a rough age) | +921 interviews

Original: https://www.aced.io/questions/3247/reverse-linked-list

QUESTION

Given the head of a singly linked list, reverse the list and return its head. Implement a function reverseList that takes the head of the linked list as input and returns the new head of the reversed list. Refer to the ListNode class/struct and do not comment it out. Example Input: 1 -> 2 -> 3 -> 4 Output: 4 -> 3 -> 2 -> 1Challenge 1Can you come up with a solution with a time complexity of O(n)?Challenge 2Can you come up with a solution with a space complexity of O(1)?

APPROACH

Iterative: walk with prev/cur pointers flipping next; or recursion. O(n) time, O(1) space iterative.

HOW TO THINK ABOUT IT

Reversing means every next pointer has to flip, so hold three things: the node you are on, the one before it (what it will point to) and the one after it (so you do not lose the rest). Always save next before overwriting it. For 'reverse part of the list' remember the node before the part and the node that will end up last in it.

Basis: LeetCode 206 (Reverse Linked List), 92 (between positions) and 25 (in groups of k).

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def reverse_list(head):
    values = []
    node = head
    while node:
        values.append(node.val)
        node = node.next
    new_head = None
    for v in values:
        node = ListNode(v)
        node.next = new_head
        new_head = node  # push to front
    return new_head
```

BRUTE FORCE

TIME	SPACE
O(n)	O(n)

WHY IT IS SLOW

Time is O(n), but it allocates a whole new list, O(n) extra space. Flipping each node's next pointer in place with three variables needs O(1) space.

OPTIMAL SOLUTION · Flip the pointers in one pass

TIME	SPACE
O(n)	O(1)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Do it recursively.

Reverse the rest first, then make the next node point back at this one and cut this node's own pointer. An inner helper that carries prev (as coded) keeps it tail-recursive in spirit; the call stack is O(n), so mention the depth limit.

FOLLOW-UP 2 Reverse only positions left to right (LeetCode 92).

Walk to the node before the segment, reverse right - left + 1 nodes with the same loop, then reconnect: the old first node of the segment must point to what followed the segment, and the node before it points to the new first.

FOLLOW-UP 3 Reverse in groups of k and leave a short last group alone (LeetCode 25).

Check that k nodes exist before reversing a group, reverse them with the same loop, reconnect both ends, and move the group pointer to the old first node of the group.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Reverse a linked list.
#
# Given the head of a singly linked list, reverse the list
# and return its head. Implement a function reverseList
# that takes the head of the linked list as input and
# returns the new head of the reversed list. Refer to the
# ListNode class/struct and do not comment it out. Example
# Input: 1 -> 2 -> 3 -> 4 Output: 4 -> 3 -> 2 ->
# 1Challenge 1Can you come up with a solution with a time
# complexity of O(n)?Challenge 2Can you come up with a
# solution with a space complexity of O(1)?
#
# FOLLOW-UPS (answered below, in this order):
```

```
# 1. Do it recursively.
# 2. Reverse only positions left to right (LeetCode 92).
# 3. Reverse in groups of k and leave a short last group
# alone (LeetCode 25).
#
# INTERVIEW NOTES
# Reversing means every next pointer has to flip, so hold
# three things: the node you are on, the one before it
# (what it will point to) and the one after it (so you do
# not lose the rest). Always save next before overwriting
# it. For 'reverse part of the list' remember the node
# before the part and the node that will end up last in
# it.
#
# BRUTE FORCE (say it first): Copy the values to an array
# and rebuild
# Read all values, then build a new list in reverse order.
# Obviously correct, but it allocates a whole new list
# instead of changing the links. Time O(n). Space O(n).


# HELPERS for follow-up 2
class ListNode:
    def __init__(self, val=0, next=None):
        self.val, self.next = val, next


# BASE QUESTION (optimal): Flip the pointers in one pass
# Idea: Walk with prev and cur. Save cur.next, point cur
# at prev, then move both forward. When cur is None, prev
# is the new head. Saving next before the overwrite is the
# step people forget.
```

```
def reverse_list(head):
    # point every node back at the one before it
    prev = None
    while head:
        head.next, prev, head = prev, head, head.next
    return prev


# FOLLOW-UP 1: Do it recursively.
# Answer: Reverse the rest first, then make the next node
# point back at this one and cut this node's own pointer.
def reverse_recursive(head, prev=None):
    # the call stack is O(n): mention the depth limit
    if head is None:
        return prev
    nxt, head.next = head.next, prev
    return reverse_recursive(nxt, head)


# FOLLOW-UP 2: Reverse only positions left to right
# (LeetCode 92).
# Answer: Walk to the node before the segment, reverse
# right - left + 1 nodes with the same loop, then
# reconnect: the old first node of the segment must point
# to what followed the segment, and the node before it
# points to the new first.
def reverse_between(head, left, right):
    # LeetCode 92: reverse positions left..right (1-based)
    dummy = ListNode(0, head)
    before = dummy
    for _ in range(left - 1):
        before = before.next
    # reverse right - left + 1 nodes with the same loop
    prev, cur = None, before.next
    for _ in range(right - left + 1):
        cur.next, prev, cur = prev, cur, cur.next
    # reconnect: the old first node points past the
    # segment
    before.next.next = cur
    before.next = prev
```

```
    return dummy.next


# FOLLOW-UP 3: Reverse in groups of k and leave a short
# last group alone (LeetCode 25).
```

```
# Answer: Check that k nodes exist before reversing a
# group, reverse them with the same loop, reconnect both
# ends, and move the group pointer to the old first node
# of the group.
def reverse_k_group(head, k):
    # LeetCode 25: reverse each full group of k; a short
    # last
    # group is left alone
    dummy = ListNode(0, head)
    group_prev = dummy
    while True:
        # is there a full group of k nodes?
        end = group_prev
        for _ in range(k):
            end = end.next
            if end is None:
                return dummy.next
        first, after = group_prev.next, end.next
        prev, cur = after, first
        while cur is not after:
            cur.next, prev, cur = prev, cur, cur.next
        group_prev.next = prev
        group_prev = first


# HELPERS for the checks below
def make(values):
    dummy = tail = ListNode()
    for v in values:
        tail.next = ListNode(v)
        tail = tail.next
    return dummy.next


def to_list(node):
    out = []
    while node:
        out.append(node.val)
        node = node.next
    return out


if __name__ == "__main__":
    assert to_list(reverse_list(make([1, 2, 3, 4]))) == [
        4, 3, 2, 1]
```

```
    assert reverse_list(None) is None
    got = to_list(reverse_recursive(make([1, 2, 3])))
    assert got == [3, 2, 1]
    part = reverse_between(make([1, 2, 3, 4, 5]), 2, 4)
    assert to_list(part) == [1, 4, 3, 2, 5]
    got = to_list(reverse_between(make([1, 2]), 1, 2))
    assert got == [2, 1]
    grouped = reverse_k_group(make([1, 2, 3, 4, 5]), 2)
    assert to_list(grouped) == [2, 1, 4, 3, 5]
    got = to_list(reverse_k_group(make([1, 2, 3]), 3))
    assert got == [3, 2, 1]
    assert to_list(reverse_k_group(make([1]), 2)) == [1]
```

OFFICIAL PAGE TEXT

Given the head of a singly linked list, reverse the list and return its head. Implement a function reverseList that takes the head of the linked list as input and returns the new head of the reversed list. Refer to the ListNode class/struct and do not comment it out. Example

```
Input: 1 -> 2 -> 3 -> 4
Output: 4 -> 3 -> 2 -> 1
```

Challenge 1Can you come up with a solution with a time complexity of O(n)?Challenge 2Can you come up with a solution with a space complexity of O(1)?Our solution reverses the linked list by iterating through it while maintaining pointers to keep track of the previous node and the current node. For each node, it saves the next node, then reverses the link by pointing the current node's next to the previous node. It then moves the prev pointer to the current node and advances the current pointer to the saved next node. This process continues until the end of the list is reached. At the end, the prev pointer points to the new head of the reversed list, which is returned. The solution uses two pointers, prev and current, to achieve this reversal. This approach ensures that each node is processed exactly once, making it both efficient and straightforward. Time Complexity: The solution has a time complexity of O(n), where n is the number of nodes in the linked list. This is because we iterate through the list once, performing constant-time operations (pointer adjustments) for each node. Space Complexity: The solution uses O(1) space, as it only uses a fixed amount of extra space (three pointers: prev, current, and nextNode). No additional data structures are used, making it very space-efficient. NoteCheck out our lesson on the two pointer approach to learn more about what it is and how to use it!No answers

contributed by the community yet.

STUDY HERE: topic 2, Arrays, Hashing and Two Pointers

EASY | Exponent | 3 of 38

Sort a binary array in linear time.

ASKED (ANY CO.) ~5 MONTHS AGO | LAST 6 MONTHS

STUDY: TYPE | ALL TIME 53.6 | 30 DAYS 0 | PRIORITY 64/228

ABOUT THIS QUESTION

Topic: Arrays, Hashing and Two Pointers

Optimal answer from: written and tested here for this exact question

Related in your app (a neighbouring problem, not this exact question): LeetCode #75 Sort Colors

LeetCode match: LC 75 Sort Colors (binary version) (inferred from short statement)

Last asked (any company): 5 months ago, about 2026-05 (Exponent shows only a rough age)

Original: https://www.aced.io/questions/971/sort-binary-array-linear-time

QUESTION

Sort a binary array in linear time.

APPROACH

Two pointers swapping 0s to the front (or counting zeros); one pass, O(n), O(1).

INTERVIEW-DAY PLAN

CLARIFY FIRST

Ask whether it must be in place and stable (for bare 0 and 1 stability does not matter).

BRUTE FORCE TO SAY OUT LOUD

Use the library sort. O(n log n).

OPTIMAL IDEA

Two pointers: lo moves right past zeros, hi moves left past ones, and when lo points at a 1 and hi at a 0 swap them. One pass, O(n) time, O(1) space. Counting the zeros and refilling is the other simple option.

DRY-RUN THESE CASES

Empty array, all zeros, all ones, already sorted, reverse sorted.

LIKELY FOLLOW-UPS

Three values (Dutch national flag, LeetCode 75), or sort records by a binary key.

PITFALLS

Moving both pointers after a swap but forgetting the loop bound.

HOW TO THINK ABOUT IT

With only two values the sorted array is 'all zeros then all ones', so you only have to move zeros left and ones right. Two pointers from the two ends do it in one pass: swap whenever the left pointer sees a 1 and the right pointer a 0.

Basis: LeetCode 75, Sort Colors, with two values instead of three.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute(arr):
    return sorted(arr)  # O(n log n)
```

TIME	SPACE
O(n log n)	O(n)

WHY IT IS SLOW

A comparison sort is O(n log n) and needs a copy; with only two values two pointers (swap a 1 on the left with a 0 on the right) finish in one O(n) pass in place.

OPTIMAL SOLUTION · Two pointers swapping from both ends

TIME	SPACE
O(n)	O(1)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Why not just count zeros and fill?

Counting is also O(n) time and O(1) space and is fine for bare numbers. It needs two passes, and it cannot sort records that merely have a binary key, because it loses the objects. The swap version sorts the records themselves.

FOLLOW-UP 2 Count the zeros and rebuild (also O(n)).

Fine for bare numbers: one pass to count, one to fill. It cannot sort records that merely have a 0 or 1 key, because it loses the objects; the swapping version can.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Sort a binary array in linear time.
#
# Sort a binary array in linear time.
#
# FOLLOW-UPS (answered below, in this order):
# 1. Why not just count zeros and fill?
# 2. Count the zeros and rebuild (also O(n)).
#
# INTERVIEW NOTES
# With only two values the sorted array is 'all zeros then
# all ones', so you only have to move zeros left and ones
# right. Two pointers from the two ends do it in one pass:
# swap whenever the left pointer sees a 1 and the right
# pointer a 0.
#
# BRUTE FORCE (say it first): Use the library sort
# sorted(arr) is correct, but it is a comparison sort and
```

```
# ignores that the values are only 0 and 1. Time O(n log
# n). Space O(n).


# BRUTE FORCE (say it first): Use the library sort | time
# O(n log n)
def sort_binary_library(arr):
    return sorted(arr)


# BASE QUESTION (optimal): Two pointers swapping from both
# ends
# Idea: Keep lo at the next position for a 0 and hi at the
# next position for a 1. Advance lo past zeros, retreat hi
# past ones, and when both are stuck swap them. One pass,
# in place.
def sort_binary(arr):
    # two pointers: zeros collect on the left, ones on the
    # right; every step swaps or moves one pointer: O(n)
    lo, hi = 0, len(arr) - 1
    while lo < hi:
        if arr[lo] == 0:
            lo += 1
        elif arr[hi] == 1:
            hi -= 1
        else:
            # a 1 on the left and a 0 on the right: swap
            # them
```

```
            arr[lo], arr[hi] = arr[hi], arr[lo]
            lo += 1
            hi -= 1
    return arr


# RECORDS with a 0/1 key
def sort_records(items, key):
    # records that merely have a 0/1 key: swapping keeps
    # the
    # objects (counting would lose them). Not stable.
    lo, hi = 0, len(items) - 1
    while lo < hi:
        if key(items[lo]) == 0:
            lo += 1
        elif key(items[hi]) == 1:
            hi -= 1
        else:
            items[lo], items[hi] = items[hi], items[lo]
            lo += 1
            hi -= 1
    return items


# VARIANT: 0, 1 and 2 (Dutch flag)
def sort_three(arr):
    # the same idea for 0, 1, 2 (Dutch national flag):
    # [0, low) are 0, [high, n) are 2, the middle is 1
    low, mid, high = 0, 0, len(arr)
    while mid < high:
        if arr[mid] == 0:
            arr[low], arr[mid] = arr[mid], arr[low]
            low += 1
            mid += 1
        elif arr[mid] == 2:
            high -= 1
            arr[mid], arr[high] = arr[high], arr[mid]
        else:
            mid += 1
    return arr


# FOLLOW-UP 2: Count the zeros and rebuild (also O(n)).
# Answer: Fine for bare numbers: one pass to count, one to
```

```
# fill. It cannot sort records that merely have a 0 or 1
```

```
# key, because it loses the objects; the swapping version
# can.
def sort_by_counting(arr):
    # follow-up: count the zeros, then rewrite the array.
    # Fine for bare numbers, useless for records.
    zeros = arr.count(0)
    return [0] * zeros + [1] * (len(arr) - zeros)


if __name__ == "__main__":
    mixed = [1, 0, 1, 0, 1, 0]
    assert sort_binary(mixed) == [0, 0, 0, 1, 1, 1]
    assert sort_binary([]) == []
    assert sort_binary([1]) == [1]
    assert sort_binary([1, 1, 0]) == [0, 1, 1]
    for arr in ([0, 0], [1, 1], [1, 0], [0, 1, 0, 0, 1]):
        assert sort_binary(arr[:]) == sorted(arr)
        assert sort_by_counting(arr) == sorted(arr)
    people = [("ann", 1), ("bob", 0),
              ("cy", 1), ("di", 0)]
    done = sort_records(people[:], key=lambda p: p[1])
    keys = [p[1] for p in done]
    assert keys == [0, 0, 1, 1]
    # no record was lost: the same objects are still there
    assert sorted(done) == sorted(people)
    flag = sort_three([2, 0, 1, 2, 0, 1])
    assert flag == [0, 0, 1, 1, 2, 2]
    assert sort_three([2, 2, 0]) == [0, 2, 2]
```

STUDY HERE: topic 2, Arrays, Hashing and Two Pointers

EASY | IN APP LC 125 | Exponent | 4 of 38

Is this a valid palindrome?

ASKED (ANY CO.) ~6 MONTHS AGO | OLDER

STUDY: TYPE | ALL TIME 52.8 | 30 DAYS 51.4 | PRIORITY 22/228

ABOUT THIS QUESTION

Topic: Arrays, Hashing and Two Pointers

Optimal answer from: your app's interview approach (LeetCode #125)

LeetCode match: LC 125 Valid Palindrome

Last asked (any company): 6 months ago, about 2026-04 (Exponent shows only a rough age) | +612 interviews

Original: https://www.aced.io/questions/4056/validate-palindrome-check

QUESTION

Given a string s, return true if it is a palindrome, and false otherwise. A string is considered a palindrome if, after converting all uppercase letters into lowercase letters and removing all non-alphanumeric characters, it reads the same forward and backward. Write a function isPalindrome that implements this logic. Examples Input: s = "H123!@321h" Output: true Input: s = "race a car" Output: false Constraints 1 <= s.length <= 2 * 10^5 The input string s can include any printable ASCII characters. Challenge 1Can you come up with a solution that achieves O(n) time complexity?Challenge 2Can you come up with a solution that achieves O(1) space complexity?Solution 1: Filtering and Reversing Approach This solution processes the input string by first filtering out non-alphanumeric characters and converting all remaining characters to lowercase. It then checks if this filtered string reads the same forward and backward. Filtering: Iterate through the string and collect alphanumeric characters while converting them to lowercase. Reversing and Comparing: Create a reversed version of the filtered string and compare it with the original filtered string to check for palindrome properties. Time Complexity: The time complexity is O(n), where n is the length of the input string. This is because filtering the characters and creating the reversed string both require iterating through the input

once. Space Complexity: The space complexity is O(n) for storing the filtered characters and their

APPROACH

Two pointers from both ends skipping non-alphanumerics and comparing case-insensitively. O(n), O(1).

HOW TO THINK ABOUT IT

A palindrome is symmetric, so compare the two ends and move inward; there is no need to build the reversed string. Decide what to ignore (non-alphanumerics, case) and skip those characters with the pointers instead of copying a cleaned string.

Basis: LeetCode 125, Valid Palindrome, and 680 (Valid Palindrome II).

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def is_palindrome(s):
    t = [c.lower() for c in s if c.isalnum()]
    return t == t[::-1]
```

TIME	SPACE
O(n)	O(n)

WHY IT IS SLOW

O(n) time but two extra copies of the string, O(n) space. Two pointers moving inward, skipping punctuation, need O(1) space and stop at the first mismatch.

OPTIMAL SOLUTION · Two pointers that skip ignored characters

TIME	SPACE
O(n)	O(1)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 You may delete at most one character (LeetCode 680).

Move inward until the first mismatch, then the answer is whether skipping the left character or skipping the right character leaves a palindrome. That is two more linear checks, so O(n) overall.

FOLLOW-UP 2 Unicode, accents, or locale-specific case.

Normalise first (unicodedata.normalize and casefold) and decide what counts as a letter. str.isalnum handles many scripts, but combining marks need normalisation.

FOLLOW-UP 3 Longest palindromic substring.

Expand around each of the 2n - 1 centers in O(n^2), or use Manacher's algorithm in O(n). The two-pointer comparison is the building block.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Is this a valid palindrome?
#
# Given a string s, return true if it is a palindrome, and
# false otherwise. A string is considered a palindrome if,
# after converting all uppercase letters into lowercase
# letters and removing all non-alphanumeric characters, it
# reads the same forward and backward. Write a function
# isPalindrome that implements this logic. Examples Input:
# s = "H123!@321h" Output: true Input: s = "race a car"
# Output: false Constraints 1 <= s.length <= 2 * 10^5 The
# input string s can include any printable ASCII
# characters. Challenge 1Can you come up with a solution
# that achieves O(n) time complexity?Challenge 2Can you
# come up with a solution that achieves O(1) space
# complexity?Solution 1: Filtering and Reversing Approach
# This solution processes the input string by first
# filtering out non-alphanumeric characters and converting
# all remaining characters to lowercase.
#
# FOLLOW-UPS (answered below, in this order):
# 1. You may delete at most one character (LeetCode 680).
# 2. Unicode, accents, or locale-specific case.
# 3. Longest palindromic substring.
#
# INTERVIEW NOTES
# A palindrome is symmetric, so compare the two ends and
# move inward; there is no need to build the reversed
# string. Decide what to ignore (non-alphanumerics, case)
# and skip those characters with the pointers instead of
# copying a cleaned string.
#
# BRUTE FORCE (say it first): Clean the string and compare
# with its reverse
# Keep only letters and digits in lowercase, and compare
# with the reverse. Short and correct, with two extra
# copies of the string. Time O(n). Space O(n).


import unicodedata


# BRUTE FORCE (say it first): Clean the string and compare
# with its reverse | time O(n)
def is_palindrome_clean(s):
    t = [c.lower() for c in s if c.isalnum()]
    return t == t[::-1]


# HELPERS for the base question
# FOLLOW-UP 2: Unicode, accents, or locale-specific case.
# Answer: Normalise first (unicodedata.normalize and
# casefold) and decide what counts as a letter.
# str.isalnum handles many scripts, but combining marks
# need normalisation.
# NEEDED BY THE BASE QUESTION: it is also the answer to
# the follow-up below
def clean(s):
    # follow-up: unicode. Decompose accents, drop the
    # combining marks, fold case, keep letters and digits.
    text = unicodedata.normalize("NFKD", s)
    text = "".join(ch for ch in text
                   if not unicodedata.combining(ch))
    return [ch for ch in text.casefold() if ch.isalnum()]


# BASE QUESTION (optimal): Two pointers that skip ignored
# characters
# Idea: Move left and right toward each other, skipping
# characters that are not letters or digits, and compare
# the lowercase forms. The first mismatch ends it. Nothing
# is copied.
def is_palindrome(s):
    # compare the cleaned characters from both ends inward
    chars = clean(s)
    lo, hi = 0, len(chars) - 1
    while lo < hi:
        if chars[lo] != chars[hi]:
            return False
        lo += 1
        hi -= 1
    return True


# SAME check without a cleaned copy
def is_palindrome_in_place(s):
    # the same without building a cleaned copy: two
    # pointers
    # on the original string skip non-alphanumeric
    # characters
    lo, hi = 0, len(s) - 1
    while lo < hi:
        if not s[lo].isalnum():
            lo += 1
        elif not s[hi].isalnum():
            hi -= 1
        elif s[lo].lower() != s[hi].lower():
            return False
        else:
            lo += 1
            hi -= 1
    return True


# FOLLOW-UP 1: You may delete at most one character
# (LeetCode 680).
# Answer: Move inward until the first mismatch, then the
# answer is whether skipping the left character or
# skipping the right character leaves a palindrome. That
# is two more linear checks, so O(n) overall.
def almost_palindrome(s):
    # follow-up (LeetCode 680): at most one deletion. At
    # the
    # first mismatch try skipping the left or the right
    # character; each is one more linear check.
    chars = clean(s)

    def straight(lo, hi):
        while lo < hi:
            if chars[lo] != chars[hi]:
                return False
            lo += 1
            hi -= 1
        return True

    lo, hi = 0, len(chars) - 1
    while lo < hi:
        if chars[lo] != chars[hi]:
            skip_left = straight(lo + 1, hi)
            return skip_left or straight(lo, hi - 1)
        lo += 1
        hi -= 1
    return True


# FOLLOW-UP 3: Longest palindromic substring.
```

```
# Answer: Expand around each of the 2n - 1 centers in
# O(n^2), or use Manacher's algorithm in O(n). The two-
# pointer comparison is the building block.
def longest_palindromic_substring(s):
    # follow-up: expand around each of the 2n - 1 centres
    best = ""
    for centre in range(2 * len(s) - 1):
        i = centre // 2
        j = i + centre % 2
        while i >= 0 and j < len(s) and s[i] == s[j]:
            i -= 1
            j += 1
        if j - i - 1 > len(best):
            best = s[i + 1:j]
    return best


if __name__ == "__main__":
    assert is_palindrome("H123!@321h")
    assert not is_palindrome("race a car")
    assert is_palindrome("A man, a plan, a canal: Panama")
    assert is_palindrome("")
    assert is_palindrome(".,")
    assert is_palindrome("Été")
    assert is_palindrome_in_place("H123!@321h")
    assert not is_palindrome_in_place("race a car")
    assert almost_palindrome("abca")
    assert almost_palindrome("raceacar")
    assert not almost_palindrome("abcdef")
    assert almost_palindrome("a")
    assert longest_palindromic_substring("babad") in (
        "bab", "aba")
    assert longest_palindromic_substring("cbbd") == "bb"
    import random
    for _ in range(300):
        s = "".join(random.choice("aB,c ")
                    for _ in range(random.randint(0, 9)))
        kept = "".join(clean(s))
        assert is_palindrome(s) == (kept == kept[::-1])
        quick = is_palindrome_in_place(s)
        assert quick == is_palindrome(s)
        near = kept == kept[::-1]
        for i in range(len(kept)):
            rest = kept[:i] + kept[i + 1:]
            near = near or rest == rest[::-1]
```

```
        assert almost_palindrome(s) == near
```

OFFICIAL PAGE TEXT

Given a string s, return true if it is a palindrome, and false otherwise. A string is considered a palindrome if, after converting all uppercase letters into lowercase letters and removing all non-alphanumeric characters, it reads the same forward and backward. Write a function isPalindrome that implements this logic. Examples

```
Input: s = "H123!@321h"
Output: true

Input: s = "race a car"
Output: false
```

OFFICIAL PAGE TEXT

Constraints 1 <= s.length <= 2 * 10^5 The input string s can include any printable ASCII characters. Challenge 1Can you come up with a solution that achieves O(n) time complexity?Challenge 2Can you come up with a solution that achieves O(1) space complexity?Solution 1: Filtering and Reversing Approach This solution processes the input string by first filtering out non-alphanumeric characters and converting all remaining characters to lowercase. It then checks if this filtered string reads the same forward and backward. Filtering: Iterate through the string and collect alphanumeric characters while converting them to lowercase. Reversing and Comparing: Create a reversed version of the filtered string and compare it with the original filtered string to check for palindrome properties. Time Complexity: The time complexity is O(n), where n is the length of the input string. This is because filtering the characters and creating the reversed string both require iterating through the input once. Space Complexity: The space complexity is O(n) for storing the filtered characters and their reversed copy. This space is needed to hold the filtered string and its reversed version. Solution 2: Two-Pointer Approach This solution uses a two-pointer technique to efficiently check if the string is a palindrome. The idea is to use one pointer starting from the beginning of the string (left) and another pointer from the end (right).

The pointers move towards each other, skipping non-alphanumeric characters and comparing characters that are alphanumeric. Pointer Initialization: Set two pointers: left at the beginning and right at the end of the string. Skipping Non-Alphanumeric Characters: Move the left pointer to the right while skipping non-alphanumeric characters, and similarly move the right pointer to the left. Comparison: Compare the characters at left and right in a case-insensitive manner. If characters do not match, the string is not a palindrome. Update Pointers: Move the pointers closer to each other and repeat until the pointers meet or cross. Time Complexity: The time complexity is O(n), where n is the length of the string. Each character is processed at most twice (once by each pointer). Space Complexity: The space complexity is O(1) as it only uses a constant amount of extra space for the two pointers, regardless of the size of the input string. NoteCheck out our lesson on the two-pointer approach to learn more about it and its applications!No answers contributed by the community yet.

STUDY HERE: topic 2, Arrays, Hashing and Two Pointers

EASY | Exponent | 5 of 38

Find the Duplicates

ASKED (ANY CO.) ~7 MONTHS AGO | OLDER

STUDY: TALK | ALL TIME 28.2 | 30 DAYS 0 | PRIORITY 130/228

ABOUT THIS QUESTION

Topic: Arrays, Hashing and Two Pointers

Optimal answer from: written and tested here for this exact question

This question has several parts, so each part has its own answer below.

Related in your app (a neighbouring problem, not this exact question): LeetCode #350 Intersection of Two Arrays II

LeetCode match: LC 350 Intersection of Two Arrays II

Last asked (any company): 7 months ago, about 2026-03 (Exponent shows only a rough age) | +15 interviews

Original: https://www.aced.io/questions/3056/find-the-duplicates

QUESTION

Given two sorted arrays arr1 and arr2 of passport numbers, implement a function findDuplicates that returns an array of all passport numbers that are both arrays. Note that the output array should be sorted in ascending order. Let N and M be the lengths of arr1 and arr2, respectively. Solve for two cases and analyze the time & space complexities of your solutions: M ~ N - the array lengths are approximately the same M >> N - arr2 is much bigger than arr1. Example input: arr1 = [1, 2, 3, 5, 6, 7], arr2 = [3, 6, 7, 8, 20] output: [3, 6, 7]

since only these three values

are both in arr1 and arr2Hint 1Start with a brute force solution and optimize from there.Hint 2Note that the question asks for a sorted array. What techniques work well on sorted arrays?Hint 3If M >> N (case #2) is giving you trouble, consider how you might use the values of N, the shorter array, to increase the efficiency of your answer.Challenge 1Have you taken care to avoid array out-of-bound runtime errors?First, the brute force solution. Try two nested loops. For every value arr1[i] in the outer loop, we look it up in arr2 in the inner loop. The time complexity of this algorithm in the worst case scenario is O(N*M). Not very efficient. We can do better than that. Case 1 (M ~ N) We can use the fact the arrays are sorted to traverse both of them in an in-order manner at the same time. The general idea of the algorithm is to use two indices, i and j, for arr1 and arr2, respectively. Every time one of indices,

APPROACH

Two pointers when sizes are similar (O(N+M), O(1) space). When one array is much smaller, binary search each element of the small array in the large one (O(M log N)). Skip repeats to keep output unique.

INTERVIEW-DAY PLAN

CLARIFY FIRST

Ask whether duplicates within one array count once in the output, whether the output must be sorted (yes), the relative sizes N and M (the statement asks for two cases), and the value range.

BRUTE FORCE TO SAY OUT LOUD

Put one array in a set and scan the other. O(N + M) time, O(min(N, M)) space, fine but ignores the sorted input.

OPTIMAL IDEA

Similar sizes: two pointers, advancing the smaller value, collecting equal values and skipping repeats. O(N + M) time, O(1) extra space. M much bigger than N: for each element of the small array binary search the large one, O(N log M), which beats O(N + M) when N is small. Exponential (galloping) search from the previous position gives O(N log(M / N)).

DRY-RUN THESE CASES

No common values, all common, repeated values in one array, an empty array, one array a single element.

LIKELY FOLLOW-UPS

Three arrays, unsorted arrays, or arrays too big for memory (merge from files).

PITFALLS

Using O(M log N) for the second case: you search the large array N times, so it is O(N log M). Returning the same value twice.

HOW TO THINK ABOUT IT

Both arrays are sorted, so the question is how the two sizes compare. If they are about the same size, walk both with two pointers and advance the smaller one. If one is much smaller, do not walk the big one: binary search each element of the small array in the big array. Always say the cost in terms of both sizes N and M.

Basis: Intersection of two sorted arrays (LeetCode 349 / 350). Output: values present in both arrays, sorted, each once.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute(arr1, arr2):
    found = []
    for x in arr1:
        for y in arr2:
            if x == y and x not in found:
                found.append(x)
    return sorted(found)
```

TIME	SPACE
O(N * M)	O(1) extra

WHY IT IS SLOW

BRUTE FORCE

N * M comparisons. Two pointers on the sorted arrays are O(N + M), and binary searching the big array for each element of the small one is O(N log M).

OPTIMAL SOLUTION · Case 1 (M close to N): two pointers

TIME	SPACE
O(N + M)	O(1) extra

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

OPTIMAL SOLUTION · Case 2 (M much bigger than N): binary search the big array

TIME	SPACE
O(N log M)	O(1) extra

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Three or more arrays.

Intersect pairwise, smallest arrays first so the running result shrinks quickly: O(total size). With K arrays you can also keep K pointers and advance the smallest.

FOLLOW-UP 2 The arrays are too big for memory.

Read both as streams (two pointers need only the current element of each), or partition both by value range into files that fit in memory and intersect partition by partition. **Cost:** O(N + M) time and O(1) memory with two streaming pointers; partitioning by value range multiplies the disk passes by the number of partitions.

FOLLOW-UP 3 Which one do you choose when N = 1,000 and M = 10^9?

Binary search: N log M is about 30,000 steps while two pointers need 10^9. The break-even is when N log M is around M.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Find the Duplicates
#
# Given two sorted arrays arr1 and arr2 of passport
# numbers, implement a function findDuplicates that
# returns an array of all passport numbers that are both
# arrays. Note that the output array should be sorted in
# ascending order. Let N and M be the lengths of arr1 and
# arr2, respectively. Solve for two cases and analyze the
# time & space complexities of your solutions: M ~ N - the
# array lengths are approximately the same M >> N - arr2 i
s
# much bigger than arr1. Example input: arr1 = [1, 2, 3,
# 5, 6, 7], arr2 = [3, 6, 7, 8, 20] output: [3, 6, 7] #
# since only these three values are both in arr1 and
# arr2Hint 1Start with a brute force solution and optimize
# from there.Hint 2Note that the question asks for a
# sorted array.
#
# FOLLOW-UPS (answered below, in this order):
# 1. Three or more arrays.
# 2. The arrays are too big for memory.
# 3. Which one do you choose when N = 1,000 and M = 10^9?
#
# INTERVIEW NOTES
# Both arrays are sorted, so the question is how the two
# sizes compare. If they are about the same size, walk
# both with two pointers and advance the smaller one. If
# one is much smaller, do not walk the big one: binary
# search each element of the small array in the big array.
# Always say the cost in terms of both sizes N and M.
#
# BRUTE FORCE (say it first): Compare every pair
# For each element of the first array scan the whole
# second array. Correct and obvious, and far too slow.
# Time O(N * M). Space O(1) extra.


import heapq
from bisect import bisect_left
from math import log2


# BRUTE FORCE (say it first): Compare every pair | time
# O(N * M)
def duplicates_brute(a, b):
    return sorted({x for x in a for y in b if x == y})
```

```
# CASE 1: M close to N (two pointers)
def common_two_pointers(a, b):
    # M close to N: walk both sorted arrays together and
    # advance the smaller side. O(N + M) time, O(1) space
    # besides the output.
    out, i, j = [], 0, 0
    while i < len(a) and j < len(b):
        if a[i] < b[j]:
            i += 1
        elif a[i] > b[j]:
            j += 1
        else:
            if not out or out[-1] != a[i]:
                out.append(a[i])
            i += 1
            j += 1
    return out


# CASE 2: M much bigger than N (binary search)
def common_binary_search(small, big):
    # M much bigger than N: binary search every item of
    # the
    # short array in the long one. O(N log M); the search
    # starts where the previous one stopped.
    out, lo = [], 0
    for x in small:
        lo = bisect_left(big, x, lo)
        if lo == len(big):
```

```
            break
        if big[lo] == x and (not out or out[-1] != x):
            out.append(x)
    return out


# BASE QUESTION (optimal): Case 1 (M close to N): two
# pointers
# Idea: Advance whichever pointer points at the smaller
# value; when equal, record the value once (skip repeats)
# and advance both. Each element is passed at most once.
def common(a, b):
    # choose by cost: N log M against N + M
    small, big = (a, b) if len(a) <= len(b) else (b, a)
```

```
    if len(small) * log2(len(big) + 2) < len(a) + len(b):
        return common_binary_search(small, big)
    return common_two_pointers(a, b)


# FOLLOW-UP 1: Three or more arrays.
# Answer: Intersect pairwise, smallest arrays first so the
# running result shrinks quickly: O(total size). With K
# arrays you can also keep K pointers and advance the
# smallest.
def common_many(arrays):
    # follow-up: three or more arrays. Intersect pairwise,
    # smallest first, so the running result shrinks fast.
    ordered = sorted(arrays, key=len)
    result = ordered[0]
    for arr in ordered[1:]:
        result = common(result, arr)
        if not result:
            break
    return result


# FOLLOW-UP 2: The arrays are too big for memory.
# Answer: Read both as streams (two pointers need only the
# current element of each), or partition both by value
# range into files that fit in memory and intersect
# partition by partition.
def common_stream(a_iter, b_iter):
    # follow-up: arrays too big for memory. Only the
    # current
    # item of each stream is held.
    a_iter, b_iter = iter(a_iter), iter(b_iter)
    x, y = next(a_iter, None), next(b_iter, None)
    last = None
    while x is not None and y is not None:
        if x < y:
            x = next(a_iter, None)
        elif x > y:
            y = next(b_iter, None)
        else:
            if x != last:
                yield x
                last = x
            x, y = next(a_iter, None), next(b_iter, None)
```

```
if __name__ == "__main__":
    a = [1, 2, 3, 5, 6, 7]
    b = [3, 6, 7, 8, 20]
    assert common_two_pointers(a, b) == [3, 6, 7]
    assert common_binary_search(b, a) == [3, 6, 7]
    assert common(a, b) == [3, 6, 7]
    assert common([], [1]) == []
    assert common([1, 1, 2], [1, 1, 3]) == [1]
    assert common_many([[1, 2, 3], [7, 8], [3]]) == []
    assert common_many([[1, 2, 3, 4], [2, 3, 9],
                        [2, 3, 5, 6, 7]]) == [2, 3]
    streamed = list(common_stream(iter(a), iter(b)))
    assert streamed == [3, 6, 7]
    import random
    for _ in range(300):
        nx = random.randint(0, 9)
        ny = random.randint(0, 9)
        x = sorted(random.sample(range(40), nx))
        y = sorted(random.sample(range(40), ny))
        want = sorted(set(x) & set(y))
        assert common_two_pointers(x, y) == want
        assert common_binary_search(x, y) == want
        assert common(x, y) == want
        assert list(common_stream(x, y)) == want
    # one short array against a very long one
    tiny, huge = [5, 999999], list(range(0, 1000000, 3))
    assert common(tiny, huge) == [999999]
```

OFFICIAL PAGE TEXT

Given two sorted arrays arr1 and arr2 of passport numbers, implement a function findDuplicates that returns an array of all passport numbers that are both arrays. Note that the output array should be sorted in ascending order. Let N and M be the lengths of arr1 and arr2, respectively. Solve for two cases and analyze the time & space complexities of your solutions: M ~ N - the array lengths are approximately the same M >> N - arr2 is much bigger than arr1. Example

```
input:  arr1 = [1, 2, 3, 5, 6, 7], arr2 = [3, 6, 7, 8,
 20]

output: [3, 6, 7] # since only these three values are
both in arr1 and arr2
```

Hint 1Start with a brute force solution and optimize from there.Hint 2Note that the question asks for a sorted array. What techniques work well on sorted arrays?Hint 3If M >> N (case #2) is giving you trouble, consider how you might use the values of N, the shorter array, to increase the efficiency of your answer.Challenge 1Have you taken care to avoid array out-of-bound runtime errors?First, the brute force solution. Try two nested loops. For every value arr1[i] in the outer loop, we look it up in arr2 in the inner loop. The time complexity of this algorithm in the worst case scenario is O(N*M). Not very efficient. We can do better than that. Case 1 (M ~ N) We can use the fact the arrays are sorted to traverse both of them in an in-order manner at the same time. The general idea of the algorithm is to use two indices, i and j, for arr1 and arr2, respectively. Every time one of indices, let's say i without any loss of generality, points to a value that is smaller than the value pointed by other index, we increment i. If arr1[i] == arr2[j], we add the value to the output array and increment both indices. Pseudocode:

OFFICIAL PAGE TEXT

```
function findDuplicates(arr1, arr2):
   duplicates = []
   i = 0
   j = 0

   while (i < arr1.length AND j < arr2.length):
      if (arr1[i] == arr2[j]):
         duplicates.push(arr1[i])
         i = i + 1
         j = j + 1
      else if (arr1[i] < arr2[j]):
         i = i + 1
      else:
         j = j + 1

   return duplicates
```

Time Complexity: O(N+M) since in the worst case scenario we traverse both arrays once. Note that O(N+M) is a linear time complexity because the input size is O(N+M) as well. Space Complexity: the variable duplicates is the only dynamic auxiliary space we're using in the algorithm. In the worst case scenario, the size of duplicates is going to be as big as big as the smaller input array. For instance, when the smaller array is fully contained within the bigger one. The space complexity is therefore O(N), where N <= M. Case 2 (M >> N) When one array is substantially longer than the other, we should try to avoid traversing the longer one. Instead, we can traverse the shorter array and look up its values in the longer array by using the binary search algorithm. We explain why this approach is superior in this case to the previous one in the complexity analysis section. Pseudocode:

```
function findDuplicates(arr1, arr2):
    duplicates = []

    for number in arr1:
        if binarySearch(arr2, number) != -1:
            duplicates.push(number);

    return duplicates

function binarySearch(arr, num):
    begin = 0
    end = arr.length - 1

    while (begin <= end):
        mid = begin + floor((end-begin)/2)
        if arr[mid] < num:
            begin = mid + 1
        else if arr[mid] == num:
            return mid
        else:
            end = mid - 1

    return -1
```

Time Complexity: we running a binary search on arr2 N times. Hence the time complexity is O(N*log(M)). So why is this algorithm better than the previous one when M >> N? To demonstrate that let's analyze the case that M = N^C, where C is a constant such that C >= 2. In this case, the time complexity of the second algorithm is:

OFFICIAL PAGE TEXT

```
O(N*log(M)) = O(N*log(N^C)) = O(C*N*log(N)) = O(N*log(
N))
```

And the time complexity of the first algorithm is:

```
O(N + M) = O(N + N^C) = O(N^C)
```

As we all know O(N^C) > O(N*log(N)). Space Complexity: the variable duplicates is the only dynamic auxiliary space we're using in the algorithm. In the worst case scenario, the size of duplicates is going to be as big as big as the smaller input array. For instance, when the smaller array is fully contained within the bigger one. The space complexity is therefore O(N), where N <= M. No answers contributed by the community yet.

STUDY HERE: topic 2, Arrays, Hashing and Two Pointers

EASY | IN APP LC 186 | Exponent | 6 of 38

Reverse a Sentence

ASKED (ANY CO.) ~2 YEARS AGO | OLDER

STUDY: TALK NO LEETCODE FREQUENCY PRIORITY 183/228

ABOUT THIS QUESTION

Topic: Arrays, Hashing and Two Pointers

Optimal answer from: your app's interview approach (LeetCode #186)

LeetCode match: LC 186 Reverse Words in a String II

Last asked (any company): 2 years ago, about 2024-10 (Exponent shows only a rough age) | +38 interviews

Original: https://www.aced.io/questions/3062/reverse-a-sentence

QUESTION

You are given an array of characters arrthat consists of sequences of characters separated by space characters. Each space-delimited sequence of characters defines a word. Implement a function reverseWords that reverses the order of the words in the array in the most efficient manner. Explain your solution and analyze its time and space complexities. Example input: arr = ['p', 'e', 'r', 'f', 'e', 'c', 't', ' ', 'm', 'a', 'k', 'e', 's', ' ', 'p', 'r', 'a', 'c', 't', 'i', 'c', 'e'] output: ['p', 'r', 'a', 'c', 't', 'i', 'c', 'e', ' ', 'm', 'a', 'k', 'e', 's', ' ', 'p', 'e', 'r', 'f', 'e', 'c', 't']Hint 1Even if your language of choice enables this, don't be tempted to use standard functions. Consider how inefficient it is to separately create a string, split the words, and reverse the split.Challenge 1As a general rule, if you're tempted to use a standard or library function, be sure to explain that function's time and space complexity before committing to that solution. You may find a more efficient way to solve the problem yourself. Oftentimes in coding interviews, this is the whole point of the exercise.Challenge 2How have you handled edge cases? For example, empty arrays, arrays with nothing but spaces, arrays with single words only, or with multiple spaces between words, etc.Challenge 3Were you able to achieve a linear time complexity and a constant space complexity?Challenge 4Bonus: how might the mirrorReverse function be implemented with a single index?One possible s

APPROACH

Reverse the whole char array, then reverse each word in place. O(n) time, O(1) space.

HOW TO THINK ABOUT IT

Reversing the order of words but not the letters is the same as reversing everything and then reversing each word back. That is two passes of the same helper and needs no extra memory when the sentence is a mutable character array. First decide how to treat repeated spaces.

HOW TO THINK ABOUT IT

Basis: LeetCode 151 (Reverse Words in a String) and 557; the in-place version is the classic array trick.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def reverse_words(arr):
    sentence = "".join(arr)
    words = sentence.split()
    return list(" ".join(reversed(words)))
```

TIME	SPACE
O(n)	O(n)

WHY IT IS SLOW

O(n) time but extra space for the word list, and it collapses repeated spaces. Reversing the whole character array and then each word in place needs O(1) extra space.

OPTIMAL SOLUTION · Reverse the whole array, then reverse each word in place

TIME	SPACE
O(n)	O(1) extra on the character array

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 The input may have leading, trailing or repeated spaces.

The split-based version already normalises them; the in-place version keeps them, so say which you want. To normalise in place add a two-pointer pass that writes single spaces.

FOLLOW-UP 2 Reverse the letters of each word but keep the word order (LeetCode 557).

Split on single spaces and reverse each word, or reverse each run in place without the first full reversal.

FOLLOW-UP 3 The sentence is a linked list of characters.

Reverse each word segment, then reverse the whole list, using the pointer-flip routine for linked lists.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Reverse a Sentence
#
# You are given an array of characters arrthat consists of
```

```
# sequences of characters separated by space characters.
# Each space-delimited sequence of characters defines a
# word. Implement a function reverseWords that reverses
# the order of the words in the array in the most
# efficient manner. Explain your solution and analyze its
# time and space complexities. Example input: arr = [ 'p',
# 'e', 'r', 'f', 'e', 'c', 't', ' ', 'm', 'a', 'k', 'e',
# 's', ' ', 'p', 'r', 'a', 'c', 't', 'i', 'c', 'e' ]
# output: [ 'p', 'r', 'a', 'c', 't', 'i', 'c', 'e', ' ',
# 'm', 'a', 'k', 'e', 's', ' ', 'p', 'e', 'r', 'f', 'e',
# 'c', 't' ]Hint 1Even if your language of choice enables
# this, don't be tempted to use standard functions.
#
# FOLLOW-UPS (answered below, in this order):
# 1. The input may have leading, trailing or repeated
# spaces.
# 2. Reverse the letters of each word but keep the word
# order (LeetCode 557).
# 3. The sentence is a linked list of characters.
#
# INTERVIEW NOTES
# Reversing the order of words but not the letters is the
# same as reversing everything and then reversing each
# word back. That is two passes of the same helper and
# needs no extra memory when the sentence is a mutable
# character array. First decide how to treat repeated
# spaces.
#
# BRUTE FORCE (say it first): Split into words and join
# them backwards
# Split on whitespace, reverse the list, join with single
# spaces. Short and clear; it uses extra space for the
# list of words and also collapses repeated spaces. Time
# O(n). Space O(n).


# BRUTE FORCE (say it first): Split into words and join
# them backwards | time O(n)
def reverse_words_split(s):
    return " ".join(reversed(s.split()))
```

```
# HELPERS for the base question
def reverse_range(arr, lo, hi):
    # reverse arr[lo..hi] in place with two pointers
    while lo < hi:
        arr[lo], arr[hi] = arr[hi], arr[lo]
        lo += 1
        hi -= 1


def each_word(arr):
    # yield the (start, end) of every word: a run of
    # characters without a space
    i = 0
    while i < len(arr):
        if arr[i] == " ":
            i += 1
            continue
        j = i
        while j + 1 < len(arr) and arr[j + 1] != " ":
            j += 1
        yield i, j
        i = j + 1


# BASE QUESTION (optimal): Reverse the whole array, then
# reverse each word in place
# Idea: Reverse every character; each word is now
# backwards but in the right place. Walk the array and
# reverse each maximal run of non-space characters.
# Everything happens inside the same array.
def reverse_words(arr):
    # reverse the whole array, so the words are in the
    # right
    # order but spelled backwards; then reverse each word
    # back. O(n) time and O(1) extra space.
    reverse_range(arr, 0, len(arr) - 1)
    for start, end in list(each_word(arr)):
        reverse_range(arr, start, end)
    return arr


# FOLLOW-UP 1: The input may have leading, trailing or
# repeated spaces.
# Answer: The split-based version already normalises them;
# the in-place version keeps them, so say which you want.
```

```
# To normalise in place add a two-pointer pass that writes
# single spaces.
def normalise_spaces(arr):
    # follow-up: leading, trailing and repeated spaces. A
    # second pointer writes the words with single spaces;
    # the array is cut to the new length.
    write = 0
    for read in range(len(arr)):
        if arr[read] != " ":
            arr[write] = arr[read]
            write += 1
        elif write and arr[write - 1] != " ":
            arr[write] = " "
            write += 1
    if write and arr[write - 1] == " ":
        write -= 1
    del arr[write:]
    return arr


# FOLLOW-UP 2: Reverse the letters of each word but keep
# the word order (LeetCode 557).
# Answer: Split on single spaces and reverse each word, or
# reverse each run in place without the first full
# reversal.
def reverse_each_word(arr):
    # follow-up (LeetCode 557): keep the word order,
    # reverse
    # the letters of every word
    for start, end in list(each_word(arr)):
        reverse_range(arr, start, end)
    return arr


if __name__ == "__main__":
    text = list("perfect makes practice")
    flipped = "".join(reverse_words(text))
    assert flipped == "practice makes perfect"
    assert reverse_words([]) == []
    assert reverse_words(list("one")) == list("one")
    # spaces stay where they are in the in-place version
    spaced = reverse_words(list(" a  bc "))
    assert "".join(spaced) == " bc  a "
    letters = reverse_each_word(list("let's code"))
    assert "".join(letters) == "s'tel edoc"
```

```
    neat = normalise_spaces(list("  a   bc  "))
    assert "".join(neat) == "a bc"
    assert normalise_spaces(list("   ")) == []
    assert normalise_spaces(list("x")) == ["x"]
    messy = list("  one   two three ")
    tidy = reverse_words(normalise_spaces(messy))
    assert "".join(tidy) == "three two one"
    import random
    for _ in range(300):
        s = "".join(random.choice("ab c")
                    for _ in range(random.randint(0, 12)))
        parts = s.split(" ")
```

```
        got = "".join(reverse_words(list(s)))
        assert got == " ".join(reversed(parts))
```

OFFICIAL PAGE TEXT

You are given an array of characters arrthat consists of sequences of characters separated by space characters. Each space-delimited sequence of characters defines a word. Implement a function reverseWords that reverses the order of the words in the array in the most efficient manner. Explain your solution and analyze its time and space complexities. Example

```
input:  arr = [ 'p', 'e', 'r', 'f', 'e', 'c', 't', '
',
                'm', 'a', 'k', 'e', 's', '  ',
                'p', 'r', 'a', 'c', 't', 'i', 'c', 'e'
 ]

output: [ 'p', 'r', 'a', 'c', 't', 'i', 'c', 'e', '  '
,
          'm', 'a', 'k', 'e', 's', '  ',
          'p', 'e', 'r', 'f', 'e', 'c', 't' ]
```

Hint 1Even if your language of choice enables this, don't be tempted to use standard functions. Consider how inefficient it is to separately create a string, split the words, and reverse the split.Challenge 1As a general rule, if you're tempted to use a standard or library function, be sure to explain that function's time and space complexity before committing to that solution. You may find a more efficient way to solve the problem yourself. Oftentimes in coding interviews, this is the whole point of the exercise.Challenge 2How have you handled edge cases? For example, empty arrays, arrays with nothing but spaces, arrays with single words only, or with multiple spaces between words, etc.Challenge 3Were you able to achieve a linear time complexity and a constant space complexity?Challenge 4Bonus: how might the mirrorReverse function be implemented with a single index?One possible solution is doing a linear iteration on arr while pushing a copy of every word to a stack and then pulling them in reverse order while copying the words back into arr. Another approach is initializing a new array at the same length, iterating over arr from right to left, and copying any sequence of characters to the new array from left to right. Both approaches take O(N) time and at least O(N) space. A more elegant and efficient approach is to reverse all the characters in arr and then reverse the

characters in each word separately. While the first reverse gives us the words in the reverse order as we wanted, it also reverses the characters of each word. To fix that, we do the second reverse, which reverses each word separately. Reversing items in an array is done by a 'mirror' function, that swaps the items of every 2 indices with the same distance from the middle. Pseudocode:

OFFICIAL PAGE TEXT

```
function reverseWords(arr):
    # reverse all characters:
    n = arr.length
    mirrorReverse(arr, 0, n-1)

    # reverse each word:
    wordStart = null
    for i from 0 to n-1:
        if (arr[i] == ' '):
            if (wordStart != null):
                mirrorReverse(arr, wordStart, i-1)
                wordStart = null
        else if (i == n-1):
            if (wordStart != null):
                mirrorReverse(arr, wordStart, i)
        else:
            if (wordStart == null):
                wordStart = i

    return arr

# helper function - reverses the order of items in arr
# please note that this is language dependent:
# if are arrays sent by value, reversing should be don
e in place

function mirrorReverse(arr, start, end):
    tmp = null
    while (start < end):
        tmp = arr[start]
```

```
        arr[start] = arr[end]
        arr[end] = tmp
        start++
        end--
```

Time Complexity: traversing the array twice with a constant number of actions for each item is linear O(N). Space Complexity: using iteration indices and one temp variable takes constant O(1) memory. Answer to bonus question: The mirrorReverse function could be implemented with a single index with left to right linear iteration and swapping arr[i] and arr[n-1-i] as long as i < n-1-i. No answers contributed by the community yet.Related questionsReverse a linked list.Related coursesCourseCourseMachine Learning Engineer Interview PrepLand your dream machine learning role at Meta, Google, Amazon, Apple, Microsoft, Nvidia, and other top companies. Learn from mock interviews, frameworks, and advice from senior candidates. Explore ML system design, core concepts, coding, behavioral interviews, and more.Start learningCourseCourseData Science Interview PrepLand your dream data science role at Google, Amazon, Microsoft, Meta, Apple, and other top companies. Learn from mock interviews, frameworks, and advice from senior candidates. Practice statistics, experimentation, coding, SQL, machine learning, behavioral interviews, and more.Start learningPythonJavaScript C#JavaC++CGoRubySwiftPythonEditorShareLoading...OutputTest ResultsConsoleSign up to run code

STUDY HERE: topic 2, Arrays, Hashing and Two Pointers

EASY | IN APP LC 953 | Exponent | 7 of 38

Determine if two words are sorted in lexicographic order.

ASKED (ANY CO.) ~3 YEARS AGO | OLDER

STUDY: TALK | ALL TIME 8.1 | 30 DAYS 0 | PRIORITY 168/228

ABOUT THIS QUESTION

Topic: Arrays, Hashing and Two Pointers

Optimal answer from: your app's interview approach (LeetCode #953)

LeetCode match: LC 953 Verifying an Alien Dictionary (inferred from short statement)

Last asked (any company): 3 years ago, about 2023-10 (Exponent shows only a rough age)

Original: https://www.aced.io/questions/3431/check-lexicographic-order-two-words

QUESTION

Determine if two words are sorted in lexicographic order.

APPROACH

Compare the two words character by character using the given order map; shorter prefix sorts first. O(L).

HOW TO THINK ABOUT IT

Compare character by character until the first difference; whichever character comes first in the alphabet decides, and if one word is a prefix of the other the shorter word comes first. A custom alphabet is just a lookup table from letter to rank.

Basis: LeetCode 953, Verifying an Alien Dictionary (a custom alphabet makes it the general case).

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def sorted_pair(a, b, order):
    rank = {c: i for i, c in enumerate(order)}
    ranks_a = [rank[c] for c in a]
    ranks_b = [rank[c] for c in b]
    return ranks_a <= ranks_b
```

TIME	SPACE
O(L)	O(L)

WHY IT IS SLOW

It builds both full rank lists even when the first letters already decide the answer. Comparing character by character and stopping at the first difference is O(min(len a, len b)) with O(1) extra space.

OPTIMAL SOLUTION · Compare character by character, optionally with a custom alphabet

TIME	SPACE
O(min(len a, len b))	O(1)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 What about uppercase, accents, or case-insensitive order?

Normalize first (casefold, unicode normalization) or give the rank map entries for the extra characters. Python's own string comparison orders by code point, which is not locale order, so use locale.strxfrm or a collation library when real language order matters.

FOLLOW-UP 2 A whole list of words in an alien alphabet (LeetCode 953).

Check each adjacent pair; sortedness of a list is the AND of its adjacent pairs. Build the rank table once.

FOLLOW-UP 3 Case-insensitive or accented comparison.

Normalise (casefold, unicode normalisation) before building ranks, or use locale-aware collation keys; plain code-point order is not language order.

FOLLOW-UP 4 Prefix rule.

'app' comes before 'apple' but 'apple' does not come before 'app': compare lengths only after all shared positions are equal.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Determine if two words are sorted in
# lexicographic order.
#
# Determine if two words are sorted in lexicographic
# order.
#
# FOLLOW-UPS (answered below, in this order):
# 1. What about uppercase, accents, or case-insensitive
# order?
# 2. A whole list of words in an alien alphabet (LeetCode
# 953).
# 3. Case-insensitive or accented comparison.
# 4. Prefix rule.
#
# INTERVIEW NOTES
# Compare character by character until the first
# difference; whichever character comes first in the
# alphabet decides, and if one word is a prefix of the
# other the shorter word comes first. A custom alphabet is
```

```
# just a lookup table from letter to rank.
#
# BRUTE FORCE (say it first): Translate both words into
# ranks and compare the lists
# Map every letter to its position in the alphabet and
# compare the two rank lists with Python's list
# comparison. Correct and the oracle; it builds both lists
# even when the first letters already differ. Time O(L).
# Space O(L).


import unicodedata


# BRUTE FORCE (say it first): Translate both words into
# ranks and compare the lists | time O(L)
def sorted_pair_by_rank(
    a, b, alphabet="abcdefghijklmnopqrstuvwxyz"
):
    rank = {c: i for i, c in enumerate(alphabet)}
    return [rank[c] for c in a] <= [rank[c] for c in b]


# HELPERS for the base question
ALPHABET = "abcdefghijklmnopqrstuvwxyz"
```

```
# RANKS of the alphabet
def make_rank(order=ALPHABET):
    # the position of every letter in the alphabet
    return {ch: i for i, ch in enumerate(order)}


# BASE QUESTION (optimal): Compare character by character,
# optionally with a custom alphabet
# Idea: Find the first position where the words differ and
# compare the two characters' ranks. If one word is a
# prefix of the other, the shorter one comes first, so
# 'app' is before 'apple' but not the reverse.
def in_order(a, b, rank):
    # compare letter by letter by rank; the first
    # difference
    # decides. If one word is a prefix of the other, the
    # shorter one comes first. A character outside the
    # alphabet is ranked after all the known ones.
    def key(ch):
        return rank.get(ch, len(rank) + ord(ch))

    for x, y in zip(a, b):
        if x != y:
            return key(x) < key(y)
    return len(a) <= len(b)


# FOLLOW-UP 1: What about uppercase, accents, or case-
# insensitive order?
# Answer: Normalize first (casefold, unicode
# normalization) or give the rank map entries for the
# extra characters.
def fold(word):
    # follow-up: ignore case and accents before comparing
    text = unicodedata.normalize("NFKD", word.casefold())
    return "".join(ch for ch in text
                   if not unicodedata.combining(ch))


# FOLLOW-UP 2: A whole list of words in an alien alphabet
# (LeetCode 953).
# Answer: Check each adjacent pair; sortedness of a list
# is the AND of its adjacent pairs. Build the rank table
# once.
```

```
def words_sorted(words, order=ALPHABET, fold_case=False):
    # follow-up (LeetCode 953): a whole list in an alien
    # alphabet is sorted when every adjacent pair is
    rank = make_rank(order)
    if fold_case:
        words = [fold(w) for w in words]
    return all(in_order(a, b, rank)
               for a, b in zip(words, words[1:]))


if __name__ == "__main__":
    rank = make_rank()
    assert in_order("apple", "banana", rank)
    assert not in_order("banana", "apple", rank)
    # the prefix rule
    assert in_order("app", "apple", rank)
    assert not in_order("apple", "app", rank)
    assert in_order("same", "same", rank)
    assert in_order("", "a", rank)
    alien = "hlabcdefgijkmnopqrstuvwxyz"
    assert words_sorted(["hello", "leetcode"], alien)
    alien = "worldabcefghijkmnpqstuvxyz"
    mixed = ["word", "world", "row"]
    assert not words_sorted(mixed, alien)
    assert not words_sorted(["apple", "app"], ALPHABET)
    assert words_sorted([], ALPHABET)
    assert words_sorted(["x"], ALPHABET)
    # case and accents only matter when asked for
    assert not words_sorted(["A", "b"], ALPHABET)
    assert words_sorted(["A", "b"], fold_case=True)
    accents = ["éclair", "ecole"]
    assert words_sorted(accents, fold_case=True)
    import random
    for _ in range(500):
        a = "".join(random.choice("abc")
                    for _ in range(random.randint(0, 4)))
        b = "".join(random.choice("abc")
                    for _ in range(random.randint(0, 4)))
        assert in_order(a, b, rank) == (a <= b)
```

STUDY HERE: topic 2, Arrays, Hashing and Two Pointers

EASY | HackTheRounds | Onsite | 8 of 38

Parity Alternating Subarray Query

STUDY: TALK | ALL TIME 34.9 | 30 DAYS 0 | PRIORITY 112/228

ABOUT THIS QUESTION

Topic: Arrays, Hashing and Two Pointers

Optimal answer from: the interview site's own solution

Related in your app (a neighbouring problem, not this exact question): LeetCode #303 Range Sum Query - Immutable

LeetCode match: No direct match; prefix-sum range queries (idea of LC 303)

QUESTION

Given an array nums and multiple queries, each query asks whether a subarray nums[l:r] satisfies the condition that adjacent elements alternate in parity (odd-even-odd-even or even-odd-even-odd). Preprocess the array to answer each query in O(1) time.

EXAMPLES

Example 1	
Input	`nums = [1, 2, 3, 4, 5], queries = [[0, 4], [1, 3]]`
Output	`[true, true]`
Why	`All adjacent pairs alternate in parity.`

Example 2	
Input	`nums = [1, 3, 2, 4], queries = [[0, 1], [0, 3]]`
Output	`[false, false]`
Why	`nums[0]=1 and nums[1]=3 are both odd.`

CONSTRAINTS

- 1 <= nums.length <= 10^5
- 1 <= queries.length <= 10^5
- 0 <= l <= r < nums.length

APPROACH

Prefix count of adjacent pairs with equal parity; a query [l, r] alternates iff the count over (l..r-1) is 0. O(1) per query.

INTERVIEW-DAY PLAN

CLARIFY FIRST

Ask whether the query range is inclusive or exclusive (the statement writes nums[l:r]), whether a single element counts as alternating, and the number of queries.

BRUTE FORCE TO SAY OUT LOUD

For each query scan the range checking adjacent parities. O(length) per query.

INTERVIEW-DAY PLAN

OPTIMAL IDEA

Precompute a prefix count B where B[i] is the number of adjacent pairs (j, j + 1) with j < i whose parities are equal. A subarray from l to r (inclusive) alternates exactly when the number of equal-parity pairs among positions l to r - 1 is zero, i.e. B[r]

- B[l] == 0. O(n) preprocessing, O(1) per query.

DRY-RUN THESE CASES

A single-element range, a range with two equal-parity neighbors, the whole array, l equal r, negative numbers (use x % 2 carefully).

LIKELY FOLLOW-UPS

Update an element (Fenwick tree), or return the longest alternating subarray.

PITFALLS

Off-by-one on the inclusive end. Parity of negative numbers in languages where % can be negative.

HOW TO THINK ABOUT IT

A subarray alternates in parity exactly when it contains no pair of neighbours with the same parity. 'Count of bad pairs in a range' is a prefix sum question, so precompute once and answer every query with one subtraction. Mark each position i by whether (nums[i], nums[i + 1]) is a bad pair.

Basis: Prefix counts for range queries (compare LeetCode 303). Assumed rule: a query (l, r) is inclusive on both ends; a single element is alternating.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def alternates(nums, l, r):
    for i in range(l, r):
        if nums[i] % 2 == nums[i + 1] % 2:
            return False
    return True
```

TIME	SPACE
O(length) per query	O(1)

WHY IT IS SLOW

Each query scans up to n elements. A prefix count of equal-parity neighbour pairs answers every query in O(1) after one O(n) pass.

OPTIMAL SOLUTION · Prefix count of equal-parity neighbour pairs

TIME	SPACE
O(n) to build, O(1) per query	O(n)

OPTIMAL SOLUTION · Prefix count of equal-parity neighbour pairs

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 The array also changes (point updates).

Keep the bad-pair indicators in a Fenwick tree: an update changes at most two indicators, each an O(log n) update, and a query is two prefix sums.

FOLLOW-UP 2 Find the longest alternating subarray.

One pass with a running length that resets to 1 on a bad pair. O(n), O(1).

FOLLOW-UP 3 Offline queries, different property (for example all elements distinct).

The prefix-sum trick applies whenever the property is 'no violating neighbour pair'. Properties about distinctness need Mo's algorithm or an offline sweep with last-occurrence tracking. **Cost:** Mo's algorithm is O((n + q) sqrt(n)); the prefix trick for 'no violating neighbour pair' stays O(n + q).

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Parity Alternating Subarray Query
#
# Given an array nums and multiple queries, each query
# asks whether a subarray nums[l:r] satisfies the
# condition that adjacent elements alternate in parity
# (odd-even-odd-even or even-odd-even-odd). Preprocess the
# array to answer each query in O(1) time.
#
# FOLLOW-UPS (answered below, in this order):
# 1. The array also changes (point updates).
# 2. Find the longest alternating subarray.
# 3. Offline queries, different property (for example all
# elements distinct).
#
# INTERVIEW NOTES
# A subarray alternates in parity exactly when it contains
# no pair of neighbours with the same parity. 'Count of
# bad pairs in a range' is a prefix sum question, so
# precompute once and answer every query with one
# subtraction. Mark each position i by whether (nums[i],
# nums[i + 1]) is a bad pair.
#
# BRUTE FORCE (say it first): Scan the subarray for every
# query
# Check each adjacent pair in nums[l..r] for equal parity.
# Correct, and every query repeats work that other queries
# already did. Time O(length) per query. Space O(1).


def brute(nums, l, r):
    return all(nums[i] % 2 != nums[i + 1] % 2
               for i in range(l, r))
```

```
# BRUTE FORCE (say it first): Scan the subarray for every
# query | time O(length) per query
def alternating_brute(nums, l, r):
    return all(
        nums[i] % 2 != nums[i + 1] % 2
        for i in range(l, r)
    )


# HELPERS for follow-up 1
class Fenwick:
    def __init__(self, size):
        self.size = size
        self.tree = [0] * (size + 1)

    def add(self, i, delta):
        i += 1
        while i <= self.size:
            self.tree[i] += delta
            i += i & -i

    def prefix(self, i):
        # sum of the first i items
        total = 0
        while i > 0:
            total += self.tree[i]
            i -= i & -i
        return total


# BASE QUESTION (optimal): Prefix count of equal-parity
# neighbour pairs
# Idea: bad[i] is the number of positions j < i with
# nums[j] and nums[j + 1] of the same parity. The pairs
# inside nums[l..r] are j = l .. r - 1, so the subarray
# alternates when bad[r] - bad[l] == 0.
class ParityQueries:
    """Static array. bad[i] = number of neighbouring pairs
    with equal parity among the first i pairs. A range
    alternates when it contains no such pair."""

    def __init__(self, nums):
        self.bad = [0]
        for a, b in zip(nums, nums[1:]):
            same = a % 2 == b % 2
            self.bad.append(self.bad[-1] + same)

    def alternates(self, l, r):
        # pairs l .. r - 1 are inside the range [l, r]
        return self.bad[r] - self.bad[l] == 0


# FOLLOW-UP 1: The array also changes (point updates).
# Answer: Keep the bad-pair indicators in a Fenwick tree:
# an update changes at most two indicators, each an O(log
# n) update, and a query is two prefix sums.
```

```
class ParityWithUpdates:
    """Follow-up: point updates. The equal-parity flags
    live in a Fenwick tree; changing one number changes
    at most two flags, and a query is two prefix sums."""

    def __init__(self, nums):
        self.nums = list(nums)
        self.flags = [0] * max(len(nums) - 1, 0)
        self.tree = Fenwick(len(self.flags))
        for i in range(len(self.flags)):
            self.refresh(i)

    def refresh(self, i):
        # recompute the flag of the pair (i, i + 1)
```

```
        a, b = self.nums[i], self.nums[i + 1]
        now = int(a % 2 == b % 2)
        self.tree.add(i, now - self.flags[i])
        self.flags[i] = now

    def update(self, i, value):
        self.nums[i] = value
        for pair in (i - 1, i):
            if 0 <= pair < len(self.flags):
                self.refresh(pair)

    def alternates(self, l, r):
        inside = self.tree.prefix(r) - self.tree.prefix(l)
        return inside == 0


# FOLLOW-UP 2: Find the longest alternating subarray.
# Answer: One pass with a running length that resets to 1
# on a bad pair. O(n), O(1).
def longest_alternating(nums):
    # follow-up: one pass with a running length that
    # restarts at 1 after a bad pair
    best = run = 1 if nums else 0
    for a, b in zip(nums, nums[1:]):
        run = run + 1 if a % 2 != b % 2 else 1
        best = max(best, run)
    return best


if __name__ == "__main__":
    q = ParityQueries([1, 2, 3, 4, 5])
```

```
    assert q.alternates(0, 4) and q.alternates(1, 3)
    q = ParityQueries([1, 3, 2, 4])
    assert not q.alternates(0, 1)
    assert not q.alternates(0, 3)
    assert q.alternates(1, 2)
    # a single element always alternates
    assert q.alternates(2, 2)
    assert longest_alternating([1, 3, 2, 4]) == 2
    assert longest_alternating([1, 2, 3, 4, 5]) == 5
    assert longest_alternating([]) == 0
    assert longest_alternating([7]) == 1
    import random
    for _ in range(200):
        nums = [random.randint(0, 9)
                for _ in range(random.randint(1, 10))]
        fixed = ParityQueries(nums)
        live = ParityWithUpdates(nums)
        for _ in range(20):
            if random.random() < 0.4:
                i = random.randrange(len(nums))
                nums[i] = random.randint(0, 9)
                live.update(i, nums[i])
            l = random.randrange(len(nums))
            r = random.randrange(l, len(nums))
            want = brute(nums, l, r)
            assert live.alternates(l, r) == want
        fixed = ParityQueries(nums)
        l = random.randrange(len(nums))
        r = random.randrange(l, len(nums))
        assert fixed.alternates(l, r) == brute(nums, l, r)
```

ALSO REPORTED · 2 more

Answer parity-alternation range queries | PracHub | Technical Screen

https://prachub.com/coding-questions/answer-parity-alternation-range-queries

Answer range queries for alternating parity subarrays | PracHub | Technical Screen

ALSO REPORTED · 2 more

https://prachub.com/coding-questions/answer-range-queries-for-alternating-parity-subarrays

STUDY HERE: topic 2, Arrays, Hashing and Two Pointers

Site tags: grid, array, greedy, medium

MEDIUM | 1Point3Acres | phone-screen | 9 of 38

Connected Crop Arrangement on a Grid

ASKED 2026-10-04 | 16 DAYS AGO | LAST 30 DAYS

STUDY: READ **NO LEETCODE FREQUENCY** **PRIORITY 207/228**

ABOUT THIS QUESTION

Topic: Arrays, Hashing and Two Pointers

Optimal answer from: written and tested here for this exact question

Frequency: Single report | Last asked: 2026-10-04 | Roles: SWE

Original: https://www.1point3acres.com/interview/problems/company/google/connected-crop-arrangement-grid

QUESTION

Given an N x M grid and K crop counts summing to N*M, fill every cell so each crop's cells form one 4-connected region; return any valid arrangement. Follow-up: two adjacent square gardens (N x N and M x M) with counts summing to N^2 + M^2.

A garden is an N x M grid. There are K crop types labeled 1..K; crop i must occupy exactly count[i] cells.
Guaranteed: sum(count) = N * M, so every cell is used.
For every crop type, all of its cells must form one connected component under 4-directional adjacency (up, down, left, right; diagonals do not count).
Different crops have no relationship constraints with each other. Return any valid arrangement.

Follow-up: two square gardens

Instead of one rectangle, there are two square gardens placed directly next to each other: garden A is N x N, garden B is M x M; together they form the planting area.
Now sum(count) = N^2 + M^2. The same single-connected-region rule applies across the combined shape.

EXAMPLES

Follow-up shapes for N = 2, M = 3 (digits mark garden A = 1, garden B = 2):

```
11222
11222
  222
```

or

```
11
11
222
222
222
```

APPROACH

Write a traversal over an N x M grid in which every consecutive pair of visited cells shares an edge, then fill crops in order along it; argue why every crop's cells stay connected.
Extend the same traversal to an L-shaped region made of two squares, handling the transition cell between gardens.
Test with K = 1, many crops of size 1, a single row or column, and both follow-up layouts above.

HOW TO THINK ABOUT IT

Walk the grid in a snake (left to right on one row, right to left on the next) so that every cell is next to the one before it. If you give crop 1 the first count[1] cells of that walk, crop 2 the next count[2] cells and so on, each crop is a contiguous stretch of the walk, so its cells are connected. No search or backtracking is needed.

Basis: Written for this question: a constructive fill along a Hamiltonian path of the grid (a snake), no LeetCode problem matches it exactly.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def fill(crop):
    if crop == len(counts):
        return True
    size = counts[crop]
    for cells in combinations(free_cells, size):
        if is_connected(cells):
            place(cells, crop)
            if fill(crop + 1):
                return True
            remove(cells)
    return False
```

TIME	SPACE
exponential	O(n * m)

WHY IT IS SLOW

Exponential in the number of cells. Filling the crops along a snake path through the grid is O(n * m) and every crop stays connected by construction.

OPTIMAL SOLUTION · Fill crops along a snake path

TIME	SPACE
O(n * m)	O(n * m)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Two adjacent square gardens (N x N next to M x M).

Use a different Hamiltonian path of the combined L-shaped area: snake down and up the columns of the first garden so that the last column ends in the top row, step into the second garden there and snake through it. Hand out the crop counts along that path exactly as before. The code checks that this path visits every cell once with each step adjacent, for every size from 1 to 6.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Connected Crop Arrangement on a Grid
#
# Given an N x M grid and K crop counts summing to N*M,
# fill every cell so each crop's cells form one
# 4-connected region; return any valid arrangement.
# Follow-up: two adjacent square gardens (N x N and M x M)
# with counts summing to N^2 + M^2. A garden is an N x M
# grid. There are K crop types labeled 1..K; crop i must
# occupy exactly count[i] cells. Guaranteed: sum(count) =
# N * M, so every cell is used. For every crop type, all
# of its cells must form one connected component under
# 4-directional adjacency (up, down, left, right;
# diagonals do not count). Different crops have no
# relationship constraints with each other. Return any
# valid arrangement. Follow-up: two square gardens Instead
# of one rectangle, there are two square gardens placed
# directly next to each other: garden A is N x N, garden B
# is M x M; together they form the planting area. Now
# sum(count) = N^2 + M^2.
#
# FOLLOW-UPS (answered below, in this order):
# 1. Two adjacent square gardens (N x N next to M x M).
#
# INTERVIEW NOTES
# Walk the grid in a snake (left to right on one row,
# right to left on the next) so that every cell is next to
# the one before it. If you give crop 1 the first count[1]
# cells of that walk, crop 2 the next count[2] cells and
# so on, each crop is a contiguous stretch of the walk, so
# its cells are connected. No search or backtracking is
# needed.
#
# BRUTE FORCE (say it first): Backtrack over every
# assignment
# Try every way to place the counts and test connectivity.
# Correct and only possible on tiny grids, which makes it
# a good oracle for the constructive answer. Time
# exponential. Space O(n * m).


import itertools
from collections import defaultdict


# BRUTE FORCE (say it first): Backtrack over every
```

```
# assignment | time exponential
def crops_brute_possible(n, m, counts):
    cells = [(i, j) for i in range(n) for j in range(m)]

    def connected(group):
        group = set(group)
        if not group:
            return True
        stack, seen = [next(iter(group))], set()
        while stack:
            r, c = stack.pop()
            if (r, c) in seen:
                continue
            seen.add((r, c))
            stack += [
                x
                for x in (
                    (r + 1, c),
                    (r - 1, c),
                    (r, c + 1),
                    (r, c - 1),
                )
                if x in group
            ]
        return seen == group

    def go(idx, remaining):
        if idx == len(counts):
            return True
        for pick in itertools.combinations(
            remaining, counts[idx]
        ):
            if connected(pick) and go(
                idx + 1,
                [c for c in remaining if c not in pick],
            ):
                return True
        return False

    return go(0, cells)


# HELPERS for the base question
def snake_cells(n, m):
    # row 0 left to right, row 1 right to left, ...
    return [(i, j if i % 2 == 0 else m - 1 - j)
            for i in range(n) for j in range(m)]


def fill_along(grid, path, counts):
    # hand out the path to crop 1, crop 2, ... in turn
    k = 0
    for crop, cnt in enumerate(counts, 1):
        for _ in range(cnt):
            r, c = path[k]
            grid[r][c] = crop
            k += 1
    return grid


# HELPERS for follow-up 1
def two_gardens_path(n, m):
    # garden A is n x n; garden B is m x m on its right
    path = []
    # snake down and up the columns of garden A
    for c in range(n):
        # the last column must end in row 0
        down = (c % 2 == 0) == (n % 2 == 0)
        rows = range(n)
        if not down:
            rows = range(n - 1, -1, -1)
        path += [(r, c) for r in rows]
    # step into garden B at row 0 and snake through it
    for r in range(m):
        cols = range(n, n + m) if r % 2 == 0 \
            else range(n + m - 1, n - 1, -1)
        path += [(r, c) for c in cols]
    return path
```

```python
# BASE QUESTION (optimal): Fill crops along a snake path
# Idea: Generate the snake order of the cells and hand out
# count[0] cells to crop 1, the next count[1] to crop 2,
# and so on. Consecutive cells of the snake share an edge,
# so each crop is connected.
def arrange_crops(n, m, counts):
    # every cell is used exactly once
    assert sum(counts) == n * m
    grid = [[0] * m for _ in range(n)]
    # touching path cells keep each crop connected
    return fill_along(grid, snake_cells(n, m), counts)


# FOLLOW-UP 1: Two adjacent square gardens (N x N next to
# M x M).
# Answer: Use a different Hamiltonian path of the combined
# L-shaped area: snake down and up the columns of the
# first garden so that the last column ends in the top
# row, step into the second garden there and snake through
# it.
def arrange_two_gardens(n, m, counts):
    assert sum(counts) == n * n + m * m
    # the grid is wide enough for both gardens
    grid = [[0] * (n + m) for _ in range(max(n, m))]
    path = two_gardens_path(n, m)
    return fill_along(grid, path, counts)


# HELPERS for the checks below
def crops_valid(grid, counts):
    cells = defaultdict(set)
    for i, row in enumerate(grid):
        for j, v in enumerate(row):
            if v:
                cells[v].add((i, j))
    for crop, cnt in enumerate(counts, 1):
        group = cells[crop]
        # the right number of cells
        if len(group) != cnt:
            return False
        # flood fill from one cell must reach them all
        stack, seen = [next(iter(group))], set()
        while stack:
            r, c = stack.pop()
            if (r, c) in seen:
                continue
            seen.add((r, c))
            for x in ((r + 1, c), (r - 1, c),
                      (r, c + 1), (r, c - 1)):
                if x in group:
                    stack.append(x)
        if seen != group:
            return False
    return True
```

```python
if __name__ == "__main__":
    # a 2 x 3 field with two crops
    g = arrange_crops(2, 3, [2, 4])
    assert crops_valid(g, [2, 4])
    # many tiny crops still connect
    ones = [1] * 9
    assert crops_valid(arrange_crops(3, 3, ones), ones)
    # two gardens: 2 x 2 next to 3 x 3 (13 cells)
    counts = [5, 4, 4]
    g2 = arrange_two_gardens(2, 3, counts)
    assert crops_valid(g2, counts)
    # the path visits every cell once, step by step
    p = two_gardens_path(2, 3)
    assert len(set(p)) == 13
    assert all(abs(a[0] - b[0]) + abs(a[1] - b[1]) == 1
               for a, b in zip(p, p[1:]))
```

NOTES FROM CANDIDATES

Asked as the first-round online coding interview of an early-career L3 loop.
Because any valid arrangement is accepted, look for a constructive traversal order that keeps consecutive cells adjacent, rather than backtracking.
For the follow-up, the two gardens can be stacked side by side or one above the other; clarify how they are joined (which edges touch and the offset) before designing the traversal.

PREPARATION

Write a traversal over an N x M grid in which every consecutive pair of visited cells shares an edge, then fill crops in order along it; argue why every crop's cells stay connected.
Extend the same traversal to an L-shaped region made of two squares, handling the transition cell between gardens.
Test with K = 1, many crops of size 1, a single row or column, and both follow-up layouts above.

STUDY HERE: topic 2, Arrays, Hashing and Two Pointers

Site tags: array, sliding-window, hashmap, prefix-sum, medium

MEDIUM | 1Point3Acres | onsite-coding | 10 of 38

Range Duplicate Check with Constant-Time Queries

ASKED 2026-09-29 | 21 DAYS AGO | LAST 30 DAYS

STUDY: READ **NO LEETCODE FREQUENCY** **PRIORITY 211/228**

ABOUT THIS QUESTION

Topic: Arrays, Hashing and Two Pointers

Optimal answer from: written and tested here for this exact question

Frequency: Low | Last asked: 2026-09-29 | Roles: SWE, MLE

Original: https://www.1point3acres.com/interview/problems/company/google/range-duplicate-check-constant-time-queries

QUESTION

Decide whether the elements in a range of an array are all unique. Progression: set-based duplicate check -> sliding window -> many range queries where reads dominate, requiring O(1) per query after linear-time preprocessing / updates with linear space.

Base: given an array and a range [l, r], determine whether the elements in that range are all unique.
The problem is staged:

Easy: duplicate detection with a set.
Sliding-window duplicate checking.
Many queries over different ranges: optimize for a read-heavy, write-light workload.

Target for the final stage: O(1) per query, preprocessing (and updates, in one version) in linear time, linear space.

APPROACH

Precompute, for each index i, the previous index holding the same value (prev[i]) with a single hashmap pass.
Turn that into an O(1) query: a range [l, r] is duplicate-free exactly when every prev[j] < l for j in the range — precompute a running "furthest start that keeps [start, j] unique" array so each query is one comparison.
Explain the cost of an update to one element (which prev entries change) and why it stays linear.

HOW TO THINK ABOUT IT

A range [l, r] has a duplicate exactly when some index i in it has its next equal element at an index <= r. Let next[i] be the next index holding the same value and reach[l] the smallest next[i] over all i >= l. Then the range is all-unique iff reach[l] > r, a single comparison per query after one linear pass.

Basis: Written for this question (a precomputed next-occurrence array, similar in spirit to LeetCode 3, Longest Substring Without Repeating Characters).

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def all_unique(a, l, r):
    seen = set()
    for x in a[l:r + 1]:
        if x in seen:
            return False
        seen.add(x)
    return True
```

TIME	SPACE
O(r - l) per query	O(r - l)

WHY IT IS SLOW

Each query costs the length of the range. A next-occurrence array with suffix minimums answers each query with one comparison.

OPTIMAL SOLUTION · Next-occurrence array with suffix minimum

TIME	SPACE
O(n) preprocessing, O(1) per query	O(n)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 The array changes between queries.

Rebuild next and reach in O(n) after a batch of changes when reads dominate. If single-element updates are frequent, keep per-value sorted index lists and a segment tree of next-occurrence minima so an update and a query cost O(log n) instead.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Range Duplicate Check with Constant-Time
# Queries
#
# Decide whether the elements in a range of an array are
# all unique. Progression: set-based duplicate check ->
# sliding window -> many range queries where reads
# dominate, requiring O(1) per query after linear-time
# preprocessing / updates with linear space. Base: given
# an array and a range [l, r], determine whether the
# elements in that range are all unique. The problem is
# staged: Easy: duplicate detection with a set. Sliding-
# window duplicate checking. Many queries over different
# ranges: optimize for a read-heavy, write-light workload.
# Target for the final stage: O(1) per query,
# preprocessing (and updates, in one version) in linear
# time, linear space.
```

```
#
# FOLLOW-UPS (answered below, in this order):
# 1. The array changes between queries.
#
# INTERVIEW NOTES
# A range [l, r] has a duplicate exactly when some index i
# in it has its next equal element at an index <= r. Let
# next[i] be the next index holding the same value and
# reach[l] the smallest next[i] over all i >= l. Then the
# range is all-unique iff reach[l] > r, a single
# comparison per query after one linear pass.
#
# BRUTE FORCE (say it first): Put the range in a set
# Insert the elements of the range into a set and compare
# sizes. Correct; every query costs the length of the
# range. Time O(r - l) per query. Space O(r - l).


from collections import Counter


# BRUTE FORCE (say it first): Put the range in a set |
# time O(r - l) per query
def range_unique_brute(a, l, r):
    return len(set(a[l : r + 1])) == r - l + 1


# HELPERS for the base question
INF = float("inf")
```

```
# EASY VERSION (say it first)
def unique_with_set(a, l, r):
    # easy version: put the range in a set
    return len(set(a[l:r + 1])) == r - l + 1


# STAGE 2: sliding window
def longest_unique_window(a):
    # stage 2: sliding window with a counter
    seen, left, best = Counter(), 0, 0
    for right, x in enumerate(a):
        seen[x] += 1
        # shrink until x occurs only once
        while seen[x] > 1:
            seen[a[left]] -= 1
            left += 1
        best = max(best, right - left + 1)
    return best


# BASE QUESTION (optimal): Next-occurrence array with
# suffix minimum
# Idea: Scan right to left with a map from value to its
# latest index to fill next[i] (infinity if none). Then
# reach[i] = min(next[i], reach[i + 1]). Query(l, r) is
# reach[l] > r.
# FOLLOW-UP 1: The array changes between queries.
# Answer: Rebuild next and reach in O(n) after a batch of
# changes when reads dominate.
class RangeUnique:
    """O(n) build, then O(1) per query."""

    def __init__(self, a):
        self.a = list(a)
        self.rebuild()

    def rebuild(self):
        a, n = self.a, len(self.a)
        # nxt[i] = next index holding the same value
        nxt, last = [INF] * n, {}
        for i in range(n - 1, -1, -1):
            nxt[i] = last.get(a[i], INF)
            last[a[i]] = i
```

```
        # reach[i] = earliest repeat from index i onward
        self.reach = [INF] * (n + 1)
        for i in range(n - 1, -1, -1):
            self.reach[i] = min(nxt[i], self.reach[i + 1])

    def unique(self, l, r):
        # no repeat starts in [l, r] and ends inside it
        return self.reach[l] > r

    # FOLLOW-UP 1: the array changes between queries
    def update(self, i, value):
        # reads dominate, so a linear rebuild is fine
        self.a[i] = value
        self.rebuild()


if __name__ == "__main__":
    a = [1, 2, 3, 1, 4]
    q = RangeUnique(a)
    for l in range(len(a)):
        for r in range(l, len(a)):
            plain = unique_with_set(a, l, r)
            assert q.unique(l, r) == plain
    assert q.unique(0, 2) and not q.unique(0, 3)
    assert longest_unique_window(a) == 4
    # an update can create or remove a duplicate
    q.update(3, 9)
    assert q.unique(0, 4)
    q.update(0, 9)
    assert not q.unique(0, 3)
```

NOTES FROM CANDIDATES

In one L4 onsite, this was the second question of a round that opened with weighted job scheduling; the interviewer asked specifically for the precomputation that makes each query O(1). The interviewer states the complexity targets explicitly, so the round is judged on hitting them, not on a working brute force. Clarify whether the array can change between queries (one version mentions linear-time updates) and whether ranges are inclusive.

PREPARATION

Precompute, for each index i, the previous index holding the same value (prev[i]) with a single hashmap pass.
Turn that into an O(1) query: a range [l, r] is duplicate-free exactly when every prev[j] < l for j in the range — precompute a running "furthest start that keeps [start, j] unique" array so each query is one comparison.
Explain the cost of an update to one element (which prev entries change) and why it stays linear.

STUDY HERE: topic 2, Arrays, Hashing and Two Pointers

Site tags: data-structure, streaming, sliding-window

MEDIUM | DESIGN-STYLE (skippable) | 1Point3Acres | onsite-coding | 11 of 38

Temperature Monitoring Data Structure

ASKED 2026-09-10 | 40 DAYS AGO | LAST 90 DAYS

STUDY: READ **NO LEETCODE FREQUENCY** **PRIORITY 223/228**

ABOUT THIS QUESTION

Topic: Arrays, Hashing and Two Pointers

Optimal answer from: written and tested here for this exact question

Frequency: Single report | Last asked: 2026-09-10 | Roles: SWE

Original: https://www.1point3acres.com/interview/problems/company/google/temperature-monitor-moving-window

QUESTION

Implement temperature insertion, a moving average, and a constant-time maximum query for a recent window. Clarify whether the window is based on reading count or elapsed days. SWEdata-structurestreamingsliding-windowFrequencySingle reportLast asked2026-09-10Stageonsite-coding

APPROACH

Implement all three reading-count operations using a running sum and deques; compare outputs with direct window scans. Test N=1, duplicate maxima, negative readings, expiration of the maximum, and partially filled windows under an explicitly chosen contract; explain amortized versus worst-case insertion cost.

INTERVIEW-DAY PLAN

CLARIFY FIRST

Ask whether the window is the last N readings or the last N days, what moving average should return before the window fills, and the number of readings.

BRUTE FORCE TO SAY OUT LOUD

Store everything in a list and compute the window average and max by scanning. O(window) per query.

OPTIMAL IDEA

For a count window keep a deque of the last N readings and a running sum for the average, and a monotonic decreasing deque for the maximum: pop smaller values from the back on insert and drop the front when it expires. All operations O(1) amortized. For time windows store (time, value) and evict by timestamp.

DRY-RUN THESE CASES

N = 1, duplicate maxima, negative readings, the maximum expiring, fewer readings than the window.

LIKELY FOLLOW-UPS

Minimum too, median over a window, or out-of-order timestamps.

PITFALLS

Removing the wrong element from the max deque when equal values repeat. Floating-point drift in the running sum.

HOW TO THINK ABOUT IT

Three operations on the last N readings. The average only needs a running sum, and the maximum needs a monotonic deque that drops smaller values from the back as larger ones arrive, so the front is always the max of the window. Decide first whether the window is a count of readings or a span of time.

Basis: Written; a sliding window with an average (LeetCode 346) and a window maximum (LeetCode 239), here as one data structure.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
readings = []
def insert(x):
    readings.append(x)
def average():
    window = readings[-N:]
    return sum(window) / len(window)
def maximum():
    return max(readings[-N:])
```

TIME	SPACE
O(N) per average or max	O(N)

WHY IT IS SLOW

Every query scans the window, O(N). A running sum gives the average in O(1), and a deque of decreasing values gives the maximum in O(1) amortised.

OPTIMAL SOLUTION · Running sum plus a monotonic deque

TIME	SPACE
O(1) amortized per operation	O(N)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 The window is the last D days (elapsed time), not the last N readings.

Store (timestamp, value) pairs and evict from the front while the timestamp is at or before now minus the span, in both the window and the max deque. Everything else is unchanged.

FOLLOW-UP 2 Also support minimum, or the median.

Minimum uses a second monotonic deque with the opposite comparison. A median needs two heaps with lazy deletion or a sorted container, O(log N) per update, because it is not decomposable like sum and max.

FOLLOW-UP 3 Floating-point drift in the running sum.

Recompute the sum from the window every so often (say every N inserts), or use math.fsum or integer hundredths of a degree to avoid drift.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Temperature Monitoring Data Structure
#
# Implement temperature insertion, a moving average, and a
# constant-time maximum query for a recent window. Clarify
# whether the window is based on reading count or elapsed
# days.
#
# FOLLOW-UPS (answered below, in this order):
# 1. The window is the last D days (elapsed time), not the
# last N readings.
# 2. Also support minimum, or the median.
# 3. Floating-point drift in the running sum.
#
# INTERVIEW NOTES
# Three operations on the last N readings. The average
# only needs a running sum, and the maximum needs a
# monotonic deque that drops smaller values from the back
# as larger ones arrive, so the front is always the max of
# the window. Decide first whether the window is a count
# of readings or a span of time.
#
# BRUTE FORCE (say it first): Keep a list and scan the
# window on every query
# Store every reading and compute sum and max over the
# last N each time. Correct and the right oracle; the cost
# is a full window scan for each query. Time O(N) per
# average or max. Space O(N).


from collections import deque


# BRUTE FORCE (say it first): Keep a list and scan the
# window on every query | time O(N) per average or max
class MonitorBrute:
    def __init__(self, n):
        self.n, self.data = n, []

    def insert(self, x):
        self.data.append(x)

    def average(self):
        w = self.data[-self.n :]
        return sum(w) / len(w)
```

```
    def maximum(self):
        return max(self.data[-self.n :])
```

```
# HELPERS for the base question
class MonotoneQueue:
    """Best window value is kept at the front."""

    def __init__(self, better):
        self.items = deque()
        # better(a, b): a is at least as good as b
        self.better = better

    def push(self, key, value):
        # values the new one beats can never be best
        while self.items and self.better(
                value, self.items[-1][1]):
            self.items.pop()
        self.items.append((key, value))

    def drop_while(self, too_old):
        # forget entries that left the window
        while self.items and too_old(self.items[0][0]):
            self.items.popleft()

    def best(self):
        return self.items[0][1]


# FOLLOW-UP 2: Also support minimum, or the median.
# Answer: Minimum uses a second monotonic deque with the
# opposite comparison. A median needs two heaps with lazy
# deletion or a sorted container, O(log N) per update,
# because it is not decomposable like sum and max.
# NEEDED BY THE BASE QUESTION: it is also the answer to
# the follow-up below
class SlidingStats:
    def __init__(self):
        self.items = deque()
        self.total = 0
        self.maxq = MonotoneQueue(lambda a, b: a >= b)
        self.minq = MonotoneQueue(lambda a, b: a <= b)

    def add(self, key, value):
        self.items.append((key, value))
```

```
        self.total += value
        self.maxq.push(key, value)
        self.minq.push(key, value)

    def expire(self, too_old):
        # drop readings that left the window everywhere
        while self.items and too_old(self.items[0][0]):
            self.total -= self.items.popleft()[1]
        self.maxq.drop_while(too_old)
        self.minq.drop_while(too_old)

    def average(self):
        return self.total / len(self.items)

    def maximum(self):
        return self.maxq.best()

    def minimum(self):
        return self.minq.best()


# BASE QUESTION (optimal): Running sum plus a monotonic
# deque
# Idea: Keep the window in a deque and a running sum; on
# insert add the value and drop the oldest when the window
# is full.
class Monitor(SlidingStats):
    """Window = the last n readings."""

    def __init__(self, n):
        super().__init__()
```

```
        self.n, self.seen = n, 0

    def insert(self, x):
        # the reading number is its key
        self.add(self.seen, x)
        self.seen += 1
        # keep only the newest n readings
        limit = self.seen - self.n
        self.expire(lambda key: key < limit)


# FOLLOW-UP 1: The window is the last D days (elapsed
# time), not the last N readings.
# Answer: Store (timestamp, value) pairs and evict from
```

```
# the front while the timestamp is at or before now minus
# the span, in both the window and the max deque.
# Everything else is unchanged.
class TimedMonitor(SlidingStats):
    """Window = readings from the last `span` days."""

    def __init__(self, span):
        super().__init__()
        self.span = span

    def insert(self, day, x):
        self.add(day, x)
        # drop readings at or before day - span
        cutoff = day - self.span
        self.expire(lambda key: key <= cutoff)


if __name__ == "__main__":
    m = Monitor(3)
    for x in [5, 1, 4, 2]:
        m.insert(x)
    # window is [1, 4, 2]
    assert m.maximum() == 4 and m.minimum() == 1
    assert m.average() == 7 / 3
    m.insert(0)
    # window is [4, 2, 0]
    assert m.maximum() == 4 and m.minimum() == 0
    t = TimedMonitor(2)
    for day, x in [(1, 9), (2, 3), (3, 5)]:
        t.insert(day, x)
    # day 1 is two days old, so it left the window
    assert t.maximum() == 5 and t.average() == 4
```

NOTES FROM CANDIDATES

The introductory description refers to the most recent N days, while the API requirements refer to the most recent N readings. Clarify the window definition before coding; the day-based variant does not specify how timestamps enter the interface. Empty-window behavior and insertion complexity are unspecified.

For the reading-count window with positive N, retain the last N readings in a FIFO queue and maintain their sum. Add the new reading and subtract any expired reading. Agree whether a partially filled window returns its current mean or is unavailable. Maintain a second deque of reading indices with nonincreasing temperatures. Expire old indices at the front and remove smaller or equal temperatures from the back before adding the new index. The front holds the current maximum; indices distinguish duplicate readings.

Average and maximum queries are O(1). Each reading enters and leaves each deque at most once, so insertion is amortized O(1) but can take O(N) for one update; auxiliary space is O(N). This implements the reading-count contract. A day-based contract additionally needs timestamps, expiration timing, and an ordering rule.

PREPARATION

Implement all three reading-count operations using a running sum and deques; compare outputs with direct window scans. Test N=1, duplicate maxima, negative readings, expiration of the maximum, and partially filled windows under an explicitly chosen contract; explain amortized versus worst-case insertion cost.

STUDY HERE: topic 2, Arrays, Hashing and Two Pointers

Site tags: array, grid

MEDIUM | 1Point3Acres | phone-screen | 12 of 38

Longest Same-Color Grid Sequence

ASKED 2026-09-08 | 42 DAYS AGO | LAST 90 DAYS

STUDY: TALK | **ALL TIME 19.6** | **30 DAYS 0** | **PRIORITY 148/228**

ABOUT THIS QUESTION

Topic: Arrays, Hashing and Two Pointers

Optimal answer from: written and tested here for this exact question

LeetCode match: LC 562 Longest Line of Consecutive One in Matrix (related)

Frequency: Single report | Last asked: 2026-09-08 | Roles: SWE

Original: https://www.1point3acres.com/interview/problems/company/google/longest-same-color-grid-line

QUESTION

Given a grid of colors, return the longest consecutive same-color sequence along a horizontal, vertical, or diagonal line.

Input is a two-dimensional grid of colors such as R, G, and B. Return the length of the longest consecutive sequence of a single color horizontally, vertically, or diagonally.

EXAMPLES

```
R R R G
R R B B
G R R R
B G B R

Answer: 4
```

APPROACH

Implement the four-direction DP in 15 minutes and trace the given grid to identify its longest line.
Compare the DP with a direct straight-line scan on small grids; include one row, one column, alternating colors, and both diagonal orientations.

HOW TO THINK ABOUT IT

A straight line can run in four directions (right, down, down-right, down-left). For each cell and each direction keep the length of the same-colour run that ends there: it is the predecessor's run plus one if the colours match, otherwise 1. Because every run only looks one cell back in a fixed direction, one pass in reading order is enough.

Basis: LeetCode 562, Longest Line of Consecutive One in Matrix, generalised to any colour.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute(grid):
    best = 0
    for r, c in all_cells(grid):
        for dr, dc in DIRECTIONS:
            n = 1
            while same(grid, r, c,
                       r + n * dr, c + n * dc):
                n += 1
            best = max(best, n)
    return best
```

TIME	SPACE
O(m * n * max(m, n))	O(1)

WHY IT IS SLOW

Re-walks the same cells in every line, O(m * n * max(m, n)). Keeping the run length ending at each cell in each direction is O(m * n).

OPTIMAL SOLUTION · Four run lengths per cell

TIME	SPACE
O(m * n)	O(n)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 The line may also bend (a connected region).

Then it is a different problem: the largest connected same-colour region, found with BFS/DFS or union-find in O(m * n). The run-length DP only handles straight lines because each run depends on exactly one predecessor.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Longest Same-Color Grid Sequence
#
# Given a grid of colors, return the longest consecutive
# same-color sequence along a horizontal, vertical, or
# diagonal line. Input is a two-dimensional grid of colors
# such as R, G, and B. Return the length of the longest
# consecutive sequence of a single color horizontally,
# vertically, or diagonally. Examples: R R R G R R B B G R
# R R B G B R Answer: 4
#
# FOLLOW-UPS (answered below, in this order):
# 1. The line may also bend (a connected region).
#
# INTERVIEW NOTES
# A straight line can run in four directions (right, down,
# down-right, down-left). For each cell and each direction
# keep the length of the same-colour run that ends there:
# it is the predecessor's run plus one if the colours
# match, otherwise 1. Because every run only looks one
```

```
# cell back in a fixed direction, one pass in reading
# order is enough.
#
# BRUTE FORCE (say it first): Extend a line from every
# cell in every direction
# Start at each cell and walk in each of the four
# directions while the colour matches. Correct; it re-
# walks the same cells many times. Time O(m * n * max(m,
# n)). Space O(1).


# BRUTE FORCE (say it first): Extend a line from every
# cell in every direction | time O(m * n * max(m, n))
def longest_line_brute(g):
    best = 0
    for i in range(len(g)):
        for j in range(len(g[0])):
            for di, dj in (
                (0, 1),
                (1, 0),
                (1, 1),
                (1, -1),
            ):
                k, a, b = 0, i, j
                while (
                    0 <= a < len(g)
                    and 0 <= b < len(g[0])
                    and g[a][b] == g[i][j]
                ):
                    k += 1
                    a += di
                    b += dj
                best = max(best, k)
    return best


# BASE QUESTION (optimal): Four run lengths per cell
# Idea: Keep, for the previous row, the run length ending
# at each column in the vertical and both diagonal
# directions, and a running value for the horizontal run
# in the current row.
def longest_same_color_line(g):
    if not g or not g[0]:
        return 0
    n, best = len(g[0]), 0
    # run lengths ending in the previous row:
    # straight down, down-left, down-right
    up, dl, dr = [0] * n, [0] * n, [0] * n
    for i, row in enumerate(g):
        nu, nl, nr = [1] * n, [1] * n, [1] * n
        left = 0
        for j, color in enumerate(row):
            # horizontal run from the cell on the left
            same = j and row[j - 1] == color
            left = left + 1 if same else 1
            if i:
                # extend the run from the cell above
                if g[i - 1][j] == color:
                    nu[j] = up[j] + 1
                # from above-left (down-right diagonal)
                if j and g[i - 1][j - 1] == color:
                    nr[j] = dr[j - 1] + 1
                # from above-right (down-left diagonal)
                if j + 1 < n and g[i - 1][j + 1] == color:
                    nl[j] = dl[j + 1] + 1
            best = max(best, left, nu[j], nl[j], nr[j])
        up, dl, dr = nu, nl, nr
    return best


# FOLLOW-UP 1: The line may also bend (a connected
# region).
# Answer: Then it is a different problem: the largest
# connected same-colour region, found with BFS/DFS or
# union-find in O(m * n). The run-length DP only handles
# straight lines because each run depends on exactly one
# predecessor.
def largest_region(g):
    # follow-up: the line may bend (connected region)
    seen, best = set(), 0
    for r in range(len(g)):
        for c in range(len(g[0])):
            if (r, c) in seen:
                continue
            seen.add((r, c))
            stack, size = [(r, c)], 0
            while stack:
                x, y = stack.pop()
                size += 1
                for a, b in ((x + 1, y), (x - 1, y),
                             (x, y + 1), (x, y - 1)):
                    if a < 0 or b < 0:
                        continue
                    if a >= len(g) or b >= len(g[0]):
                        continue
                    if (a, b) not in seen \
                            and g[a][b] == g[r][c]:
                        seen.add((a, b))
                        stack.append((a, b))
            best = max(best, size)
    return best


if __name__ == "__main__":
    grid = ["RRRG", "RRBB", "GRRR", "BGBR"]
    assert longest_same_color_line(grid) == 4
    assert longest_same_color_line([]) == 0
    # one column and alternating colours
    assert longest_same_color_line(["R", "R", "R"]) == 3
    assert longest_same_color_line(["RGR", "GRG"]) == 2
    # a bent region is longer than any straight line
    bent = ["RRR", "GGR"]
    assert longest_same_color_line(bent) == 3
    assert largest_region(["RR", "GR"]) == 3
```

NOTES FROM CANDIDATES

Track the length of a same-color run ending at each cell in four directions: horizontal, vertical, down-right diagonal, and down-left diagonal. For each direction, extend the predecessor's run by one only when its color equals the current cell; otherwise start at one.

Process rows from top to bottom and columns left to right. Horizontal state reads the current row's previous column; the other directions read the previous row. Take the maximum over all cells and directions. This checks straight consecutive lines, not connected regions or paths that turn.

Four constant-time transitions per cell give O(rows * columns) time. Keeping all states uses O(rows * columns) space; rolling previous/current rows reduces auxiliary space to O(columns). Clarify empty-grid behavior before coding.

PREPARATION

Implement the four-direction DP in 15 minutes and trace the given grid to identify its longest line.
Compare the DP with a direct straight-line scan on small grids; include one row, one column, alternating colors, and both diagonal orientations.

STUDY HERE: topic 2, Arrays, Hashing and Two Pointers

Site tags: string, hashmap, data-structure

MEDIUM | DESIGN-STYLE (skippable) | 1Point3Acres | onsite-coding | 13 of 38

Next-Word Predictor

ASKED 2026-09-02 | 48 DAYS AGO | LAST 90 DAYS

STUDY: READ **NO LEETCODE FREQUENCY** **PRIORITY 224/228**

ABOUT THIS QUESTION

Topic: Arrays, Hashing and Two Pointers

Optimal answer from: written and tested here for this exact question

Frequency: Single report | Last asked: 2026-09-02 | Roles: SWE

Original: https://www.1point3acres.com/interview/problems/company/google/next-word-frequency-predictor

QUESTION

Process training data and predict the most frequent next word for a supplied word. Maintain prediction state during processing so prediction can run in constant time.SWEstringhashmapdata-structureFrequencySingle reportLast asked2026-09-02Stageonsite-coding

APPROACH

Implement pair counting and best-successor maintenance during processing, then compare predictions with a full scan of each successor counter.

Trace repeated pairs, a tie, a new leader, an unseen word, and separate training sequences; write down the tie and sequence-boundary conventions before testing.

INTERVIEW-DAY PLAN

CLARIFY FIRST

Ask how sentences are split and cased, how ties are broken, what to return for an unseen word, and whether training is incremental.

BRUTE FORCE TO SAY OUT LOUD

Store all sentences and scan them for each prediction. O(corpus) per query.

OPTIMAL IDEA

Keep a dict from a word to a dict of next-word counts, and a separate best[word] updated whenever a count changes (if the new count beats the current best, or ties and wins the tie-break). Prediction is a dict lookup, O(1). Training is O(total words).

DRY-RUN THESE CASES

A repeated pair, a tie, a new leader taking over, a word with no successor, the last word of a sentence.

LIKELY FOLLOW-UPS

Top-k predictions (keep a heap per word), bigram versus trigram context, or decay for recent data.

INTERVIEW-DAY PLAN

PITFALLS

Updating best only on strict increase and ignoring the tie rule. Counting across sentence boundaries.

HOW TO THINK ABOUT IT

Prediction in O(1) means the answer must already be sitting in a table when the query arrives, so do the work while training. Keep counts per (word, next word) and, for each word, the current best successor. Each count only increases by one, so the best can be updated by comparing just the one count that changed.

Basis: Written; a bigram language model. Assumed rules (confirm them): words are split on spaces and compared case-sensitively; pairs never cross sentence boundaries; ties go to the alphabetically smaller word; an unseen word predicts None.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
sentences = []
def train(sentence):
    sentences.append(sentence.split())
def predict(word):
    counts = Counter()
    for words in sentences:
        for a, b in zip(words, words[1:]):
            if a == word:
                counts[b] += 1
    if not counts:
        return None
    return max(counts, key=counts.get)
```

TIME	SPACE
O(corpus size) per prediction	**O(corpus size)**

WHY IT IS SLOW

Each prediction is a full pass over the corpus. Counting pairs while training and keeping the best successor per word makes prediction O(1).

OPTIMAL SOLUTION · Counts plus a maintained best successor

TIME	SPACE
O(1) per pair when training and per prediction	**O(number of distinct pairs)**

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Return the top K predictions.

Keep the Counter per word and use heapq.nsmallest on (-count, word): O(d log K) per query, or keep a sorted structure per word if queries dominate.

FOLLOW-UP 2 Use two words of context (trigrams).

The key becomes the pair (previous two words) and everything else is identical, at the price of more memory. Back off to the bigram table when the pair was never seen.

FOLLOW-UP 3 The corpus keeps changing and old text should fade.

Use an exponentially decayed count (multiply by a factor per training step, or store a timestamp and decay on read), and recheck the best when a count drops below another.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Next-Word Predictor
#
# Process training data and predict the most frequent next
# word for a supplied word. Maintain prediction state
# during processing so prediction can run in constant
# time.
#
# FOLLOW-UPS (answered below, in this order):
# 1. Return the top K predictions.
# 2. Use two words of context (trigrams).
# 3. The corpus keeps changing and old text should fade.
#
# INTERVIEW NOTES
# Prediction in O(1) means the answer must already be
# sitting in a table when the query arrives, so do the
# work while training. Keep counts per (word, next word)
# and, for each word, the current best successor. Each
# count only increases by one, so the best can be updated
# by comparing just the one count that changed.
#
# BRUTE FORCE (say it first): Scan the whole corpus on
# every prediction
# Keep the sentences and count successors of the query
# word from scratch on each call. Simple and correct, but
# every query is as slow as one full pass over the data.
# Time O(corpus size) per prediction. Space O(corpus
# size).


import heapq
from collections import Counter, defaultdict


# BRUTE FORCE (say it first): Scan the whole corpus on
# every prediction | time O(corpus size) per prediction
class PredictorBrute:
    def __init__(self):
        self.sentences = []

    def train(self, sentence):
        self.sentences.append(sentence.split())

    def predict(self, word):
        c = Counter(
            s[i + 1]
            for s in self.sentences
            for i in range(len(s) - 1)
            if s[i] == word
        )
        return (
            min(c, key=lambda w: (-c[w], w))
            if c
            else None
        )


# BASE QUESTION (optimal): Counts plus a maintained best
# successor
# Idea: counts[word][next] is incremented for every
# adjacent pair. After the increment compare that one
# count with the current best for the word (higher count
# wins, then the smaller word).
# FOLLOW-UP 1: Return the top K predictions.
# Answer: Keep the Counter per word and use
# heapq.nsmallest on (-count, word): O(d log K) per query,
# or keep a sorted structure per word if queries dominate.
# FOLLOW-UP 2: Use two words of context (trigrams).
# Answer: The key becomes the pair (previous two words)
# and everything else is identical, at the price of more
# memory. Back off to the bigram table when the pair was
# never seen.
class Predictor:
    # FOLLOW-UP 2: orders=(2, 1) means two words of
    # context
    def __init__(self, orders=(2, 1)):
        # orders: how many previous words are the context
        self.orders = sorted(orders, reverse=True)
        # order -> context tuple -> Counter of next words
        self.counts = {k: defaultdict(Counter)
                       for k in orders}
        # order -> context tuple -> best next word so far
        self.best = {k: {} for k in orders}

    def train(self, sentence):
        words = sentence.split()
        for k in self.orders:
            for i in range(len(words) - k):
                ctx = tuple(words[i:i + k])
                nxt = words[i + k]
                self.update(k, ctx, nxt)

    def update(self, k, ctx, nxt):
        table = self.counts[k][ctx]
        table[nxt] += 1
        cur = self.best[k].get(ctx)
        # more counts win; a tie goes to the smaller word
        better = cur is None or table[nxt] > table[cur]
        tie = cur is not None and table[nxt] == table[cur]
        if better or (tie and nxt < cur):
            self.best[k][ctx] = nxt

    def predict(self, *context):
        # O(1): try the longest context, then back off
        for k in self.orders:
            if len(context) >= k:
                ctx = tuple(context[len(context) - k:])
                if ctx in self.best[k]:
                    return self.best[k][ctx]
        return None

    # FOLLOW-UP 1: the top K predictions
    def top_k(self, word, k):
        # k most likely next words after one word
```

```
        by_ctx = self.counts.get(1, {})
        table = by_ctx.get((word,), Counter())
        return heapq.nsmallest(
            k, table, key=lambda w: (-table[w], w))


if __name__ == "__main__":
    p = Predictor()
    p.train("i love dogs")
    p.train("i love cats")
    p.train("i love cats and dogs")
    # "love" is followed by cats twice, dogs once
    assert p.predict("love") == "cats"
    # two words of context are tried first
    assert p.predict("i", "love") == "cats"
    # an unseen pair backs off to the last word
    assert p.predict("we", "love") == "cats"
    assert p.predict("zebra") is None
    assert p.top_k("love", 2) == ["cats", "dogs"]
    # a tie goes to the alphabetically smaller word
    q = Predictor()
    q.train("a z")
```

```
    q.train("a b")
    assert q.predict("a") == "b"
```

NOTES FROM CANDIDATES

Exact method signatures, tokenization rules, frequency ties, and behavior for an unseen word are unspecified.
The candidate used a dictionary and stored the current best successor during processing. Calling this state a cache caused a misunderstanding: the interviewer initially thought updates would happen during prediction. Explain the update timing explicitly.
After agreeing on tokenization and sequence boundaries, increment a counter for each adjacent pair (previous_word, next_word). For each previous word, retain a successor whose count is maximal. Compare the updated successor with the retained best immediately during processing; prediction only reads that stored best.
With insertion-only counts, a pair whose count did not change cannot newly overtake the best. Under a practice convention that retains the incumbent on ties, replace the best only when the updated count is strictly larger. If a different tie rule is required, apply it explicitly on equal counts.
Hash-table updates per observed pair and prediction are expected O(1), excluding tokenization/string hashing costs. Store O(P + V) state for P distinct pairs and V predecessor words. Clarify unseen-word results and whether separate training sequences create cross-boundary pairs.

PREPARATION

Implement pair counting and best-successor maintenance during processing, then compare predictions with a full scan of each successor counter.
Trace repeated pairs, a tie, a new leader, an unseen word, and separate training sequences; write down the tie and sequence-boundary conventions before testing.

STUDY HERE: topic 2, Arrays, Hashing and Two Pointers

Site tags: array, simulation, sliding-window, medium

MEDIUM | 1Point3Acres | onsite-coding | 14 of 38

Piano Hand Movement Grouping

ASKED 2026-07-14 | 98 DAYS AGO | LAST 6 MONTHS

STUDY: READ **NO LEETCODE FREQUENCY** **PRIORITY 205/228**

ABOUT THIS QUESTION

Topic: Arrays, Hashing and Two Pointers

Optimal answer from: written and tested here for this exact question

Frequency: Low | Last asked: 2026-07-14 | Roles: SWE

Original: https://www.1point3acres.com/interview/problems/company/google/piano-hand-movement-grouping

QUESTION

EXAMPLES

```
With notes = [1, 3, 5, 7, 9, 11] and k = 4, the
    stated output is 2.
Earlier examples use [1,2,3,4,5] -> 0 and [5,9,1] ->
    1, without stating the reach parameter.
```

APPROACH

Practice turning the physical reach rule into an explicit interval invariant before writing code.
Implement a one-pass grouping scan, then extend it to emit group boundaries without changing the core state transition.

INTERVIEW-DAY PLAN

CLARIFY FIRST

The statement is ambiguous, so ask: does the hand cover a fixed interval of keys [p, p + k], is the interval fixed until the hand lifts, does the first placement count as a move, and do notes arrive in the given order. Notice the examples are not all consistent, so ask which one is the reference.

BRUTE FORCE TO SAY OUT LOUD

Try every starting position for each group and take the minimum. Exponential or quadratic.

OPTIMAL IDEA

Greedy one-pass scan: start a group at the first note with the hand covering [note, note + k]. For each next note, if it is inside the interval keep going; if not, start a new group at that note. The answer is the number of groups (or groups minus one if the first placement is free). Group boundaries come from the same scan. O(n).

DRY-RUN THESE CASES

A single note, the example [1, 3, 5, 7, 9, 11] with k = 4, notes going down and then up, k = 0.

LIKELY FOLLOW-UPS

A hand that can slide, two hands, or return the group boundaries.

INTERVIEW-DAY PLAN

PITFALLS

Counting the initial placement inconsistently. Assuming notes are sorted.

HOW TO THINK ABOUT IT

Turn the physical rule into an invariant: the current group is valid while (largest note - smallest note) <= k, because you can place the hand at the smallest. The first note that breaks it must start a new group, and starting later never helps, so a single greedy pass is optimal. Ask whether the first placement counts as a move.

Basis: Written; interval grouping, like LeetCode 452 and 1326. Assumed rule (confirm it, the examples disagree): a hand placed on key p covers p to p + k inclusive and stays put until it is lifted; the notes come in order; the answer is the minimum number of placements, and the number of lifts is one fewer.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
best = [INF] * (n + 1)
best[0] = 0
for j in range(1, n + 1):
    for i in range(j):
        part = notes[i:j]
        if max(part) - min(part) <= k:
            best[j] = min(best[j], best[i] + 1)
answer = best[n]
```

TIME	SPACE
O(n^2)	O(n)

WHY IT IS SLOW

Every end tries every start and re-checks the range, O(n^2). A greedy pass that starts a new group only when the next note no longer fits is O(n).

OPTIMAL SOLUTION · Greedy scan, tracking the group's smallest and largest note

TIME	SPACE
O(n)	O(1)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Return where each group starts and where to place the hand.

Record the start index when a group opens and emit (start, end, smallest note) when it closes. The state transition is unchanged.

FOLLOW-UP 2 Count lifts, not placements.

Lifts are placements minus one if the first placement is free; the examples differ on this, so ask and state it. In code it is a single subtraction on the group count.

FOLLOW-UP 3 The hand may slide without a lift (costing something).

Then it becomes a DP over positions with a cost per slide distance, solved with the O(n * range) DP or by greedy if the cost is monotone. Ask for the cost model.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Piano Hand Movement Grouping
#
# Examples: With notes = [1, 3, 5, 7, 9, 11] and k = 4,
# the stated output is 2. Earlier examples use [1,2,3,4,5]
# -> 0 and [5,9,1] -> 1, without stating the reach
# parameter.
#
# FOLLOW-UPS (answered below, in this order):
# 1. Return where each group starts and where to place the
# hand.
# 2. Count lifts, not placements.
# 3. The hand may slide without a lift (costing
# something).
#
# INTERVIEW NOTES
# Turn the physical rule into an invariant: the current
# group is valid while (largest note - smallest note) <=
# k, because you can place the hand at the smallest. The
# first note that breaks it must start a new group, and
# starting later never helps, so a single greedy pass is
# optimal. Ask whether the first placement counts as a
# move.
#
# BRUTE FORCE (say it first): DP over every way to cut the
# notes into consecutive groups
# best[j] is the fewest groups for the first j notes: try
# every start i of the last group and require max - min of
# notes[i:j] to be at most k. Quadratic but easy to trust.
# Time O(n^2). Space O(n).


# BRUTE FORCE (say it first): DP over every way to cut the
# notes into consecutive groups | time O(n^2)
def groups_brute(notes, k):
    n = len(notes)
    INF = float("inf")
    best = [0] + [INF] * n
    for j in range(1, n + 1):
        for i in range(j):
            if max(notes[i:j]) - min(notes[i:j]) <= k:
                best[j] = min(best[j], best[i] + 1)
    return best[n]


# HELPERS for the base question
```

```
def hand_positions(notes, k):
    # split the notes into groups that one hand position
    # (a span of at most k) can play, as early as possible
    groups = []
    for i, x in enumerate(notes):
        if groups:
            start, lo, hi = groups[-1]
            # does the note fit the current span?
            lo2, hi2 = min(lo, x), max(hi, x)
            if hi2 - lo2 <= k:
                groups[-1] = (start, lo2, hi2)
                continue
        # otherwise the hand must move: a new group starts
        groups.append((i, x, x))
    return groups


# BASE QUESTION (optimal): Greedy scan, tracking the
# group's smallest and largest note
# Idea: Keep lo and hi of the current group. Extend while
# hi - lo stays within k, otherwise close the group and
# start a new one at this note.
def groups(notes, k):
    # how many hand placements are needed
    return len(hand_positions(notes, k))


# FOLLOW-UP 1: Return where each group starts and where to
# place the hand.
# Answer: Record the start index when a group opens and
# emit (start, end, smallest note) when it closes. The
# state transition is unchanged.
def plan(notes, k):
    # where each group starts and ends, and where to put
    # the hand (on the lowest note of the group)
    out, spots = [], hand_positions(notes, k)
    for n, (start, lo, hi) in enumerate(spots):
        end = spots[n + 1][0] - 1 if n + 1 < len(spots) \
            else len(notes) - 1
        out.append((start, end, lo))
    return out


# FOLLOW-UP 2: Count lifts, not placements.
# Answer: Lifts are placements minus one if the first
# placement is free; the examples differ on this, so ask
# and state it. In code it is a single subtraction on the
# group count.
def lifts(notes, k):
    # the first placement is free; every move is a lift
    return max(groups(notes, k) - 1, 0)


if __name__ == "__main__":
    # the stated example: 2 placements for reach 4
    assert groups([1, 3, 5, 7, 9, 11], 4) == 2
    # the earlier examples, counted as lifts
    assert lifts([1, 2, 3, 4, 5], 4) == 0
    assert lifts([5, 9, 1], 4) == 1
    assert groups([], 4) == 0
    assert plan([1, 3, 5, 7, 9, 11], 4) == [(0, 2, 1),
                                            (3, 5, 7)]
    # a reach of 0 means a new placement for every change
    assert groups([2, 2, 3], 0) == 2
```

NOTES FROM CANDIDATES

Clarify whether a placement is fixed until the hand lifts, whether the covered interval may slide without counting as a lift, and whether the initial placement counts. These details materially change the answer.
The six-note example's explanation says a placement at 7 covers keys 7–11 but still counts another lift for 9 and 11. Treat that as an ambiguity to resolve before coding rather than silently assuming the output convention.
Maintain the feasible placement interval for the current group. The follow-up printout should reuse the same grouping state instead of duplicating the movement logic.

PREPARATION

Practice turning the physical reach rule into an explicit interval invariant before writing code.
Implement a one-pass grouping scan, then extend it to emit group boundaries without changing the core state transition.

STUDY HERE: topic 2, Arrays, Hashing and Two Pointers

MEDIUM | PracHub | Technical Screen | 15 of 38

★ TOP 75: rank 51 in the Medium and Hard priority list

★ Filter Repeated Robot Status Messages

POSTED 2026-07-05 | 107 DAYS AGO | LAST 6 MONTHS

STUDY: TYPE | ALL TIME 51.2 | 30 DAYS 33 | PRIORITY 32/228

ABOUT THIS QUESTION

Topic: Arrays, Hashing and Two Pointers

Optimal answer from: the interview site's own solution

Related in your app (a neighbouring problem, not this exact question): LeetCode #359 Logger Rate Limiter

LeetCode match: LC 359 Logger Rate Limiter (inferred from overview)

Posted on PracHub: 2026-07-05 (updated 2026-07-15)

Original: https://prachub.com/coding-questions/filter-repeated-robot-status-messages

QUESTION

Overview: Filter a timestamped robot status stream so a message reappears only when strictly more than ten seconds have passed since its last displayed occurrence. Preserve input order, track each exact case-sensitive message independently, and ensure hidden repeats do not reset the interval.

APPROACH

Hash map message -> last displayed timestamp; display only if now - last > 10 (strictly), then update. Preserve input order. O(1) per message.

INTERVIEW-DAY PLAN

CLARIFY FIRST

Ask for the exact rule: show a message again only if more than 10 seconds have passed since it was last displayed (strictly more), whether messages are compared case-sensitively (yes), and whether hidden repeats extend the interval (no).

BRUTE FORCE TO SAY OUT LOUD

For each message scan the already displayed ones for the last occurrence. O(n^2).

OPTIMAL IDEA

Keep a dict from message to the time it was last displayed. For each message in order: if it is not in the dict or now - last > 10, display it and set last = now; otherwise skip it and do not update last. Output preserves input order. O(1) per message.

DRY-RUN THESE CASES

A repeat at exactly 10 seconds (hidden), at 11 seconds (shown), a hidden repeat followed by another repeat 10 seconds after the first display (shown, since hidden repeats do not reset), different casing (different messages), the first message.

INTERVIEW-DAY PLAN

LIKELY FOLLOW-UPS

Bounded memory (evict messages older than the interval), or a per-robot map.

PITFALLS

Updating the last time on a hidden message. Using at least instead of strictly more than.

HOW TO THINK ABOUT IT

The decision for a message depends only on when that exact message was last displayed, so a dictionary from message to last display time is all the state you need. The detail that decides correctness: the stored time changes only when a message is shown (a hidden repeat must not reset the clock), and the comparison is strict (> 10).

Basis: Written (compare LeetCode 359, Logger Rate Limiter). Rule: show a message again only when strictly more than 10 seconds have passed since it was last shown; hidden repeats do not extend the interval; messages are case-sensitive; input order is kept.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
shown = []  # (time, message)
def on_message(t, msg):
    last = None
    for t0, m0 in shown:
        if m0 == msg:
            last = t0  # most recent display
    if last is None or t - last > 10:
        shown.append((t, msg))
        return True
    return False
```

TIME	SPACE
O(n^2)	O(n)

WHY IT IS SLOW

Each message searches the whole history, O(n^2). A dictionary from message to its last displayed time makes each decision O(1).

OPTIMAL SOLUTION · Dictionary from message to the time it was last shown

TIME	SPACE
O(n)	O(distinct messages)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 The stream never ends; memory must be bounded.

A message older than the gap can never be blocked again, so evict it. Keep a min-heap of (time shown, message) and pop entries older than the gap before each decision; delete the dictionary key only if it still points at that time.

FOLLOW-UP 2 Several robots send messages.

Use (robot id, message) as the key, or one filter per robot. The same logic applies per key.

FOLLOW-UP 3 The interval depends on the message severity.

Look up the gap from a severity table and use it in the comparison; the state (last shown time) is unchanged.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```python
# MAIN QUESTION: Filter Repeated Robot Status Messages
#
# Filter a timestamped robot status stream so a message
# reappears only when strictly more than ten seconds have
# passed since its last displayed occurrence. Preserve
# input order, track each exact case-sensitive message
# independently, and ensure hidden repeats do not reset
# the interval.
#
# FOLLOW-UPS (answered below, in this order):
# 1. The stream never ends; memory must be bounded.
# 2. Several robots send messages.
# 3. The interval depends on the message severity.
#
# INTERVIEW NOTES
# The decision for a message depends only on when that
# exact message was last displayed, so a dictionary from
# message to last display time is all the state you need.
# The detail that decides correctness: the stored time
# changes only when a message is shown (a hidden repeat
# must not reset the clock), and the comparison is strict
# (> 10).
#
# BRUTE FORCE (say it first): Scan the already shown
# messages for each new one
# Keep the list of shown (time, message) pairs and, for
# each new message, look backward for its last shown time.
# Correct and the oracle; each message searches the whole
# history. Time O(n^2). Space O(n).


import heapq


# BRUTE FORCE (say it first): Scan the already shown
# messages for each new one | time O(n^2)
def filter_status_brute(stream):
    shown, out = [], []
    for ts, msg in stream:
        last = next(
            (t for t, m in reversed(shown) if m == msg),
            None,
        )
        if last is None or ts - last > 10:
            shown.append((ts, msg))
            out.append((ts, msg))
    return out


# BASE QUESTION (optimal): Dictionary from message to the
# time it was last shown
# Idea: For each message in order: if it was never shown
# or now - last > 10, emit it and set last = now;
# otherwise drop it and leave last unchanged. One
# dictionary lookup per message.
def filter_status(stream, gap=10):
    # show a message only if MORE than `gap` seconds
    # passed
    # since it was last shown; hidden repeats change
    # nothing
    last, shown = {}, []
    for ts, msg in stream:
        if msg not in last or ts - last[msg] > gap:
            last[msg] = ts
            shown.append((ts, msg))
    return shown


# FOLLOW-UP 1: The stream never ends; memory must be
# bounded.
# Answer: A message older than the gap can never be
# blocked again, so evict it. Keep a min-heap of (time
# shown, message) and pop entries older than the gap
# before each decision; delete the dictionary key only if
# it still points at that time.
class BoundedFilter:
    """An endless stream: forget messages that can no
    longer be blocked, so memory stays small."""

    def __init__(self, gap=10):
        self.gap = gap
        self.last = {}
        # (time shown, message), oldest first
        self.heap = []

    def show(self, ts, msg):
        # drop entries older than the gap
        while self.heap:
            if ts - self.heap[0][0] <= self.gap:
                break
            old_ts, old_msg = heapq.heappop(self.heap)
            # delete only if it was not shown again since
            if self.last.get(old_msg) == old_ts:
                del self.last[old_msg]
        if msg in self.last:
            return False
        self.last[msg] = ts
        heapq.heappush(self.heap, (ts, msg))
        return True


# FOLLOW-UP 2: Several robots send messages.
# Answer: Use (robot id, message) as the key, or one
# filter per robot. The same logic applies per key.
class PerRobotFilter:
    """Several robots; the gap depends on severity."""

    def __init__(self, gaps, default=10):
        self.gaps, self.default = gaps, default
        self.last = {}

    def show(self, robot, ts, msg, severity=None):
        key = (robot, msg)
        gap = self.gaps.get(severity, self.default)
```

```
        last = self.last.get(key)
        if last is not None and ts - last <= gap:
            return False
        self.last[key] = ts
        return True


if __name__ == "__main__":
    stream = [(0, "a"), (5, "a"), (10, "a"), (11, "a"),
              (12, "b"), (22, "a")]
    # 10 is hidden (not MORE than 10 after 0); 11 shows
    assert filter_status(stream) == [(0, "a"), (11, "a"),
                                     (12, "b"), (22, "a")]
    # messages are case sensitive
    assert filter_status([(0, "A"), (1, "a")]) == [
        (0, "A"), (1, "a")]
    b = BoundedFilter()
    shows = [b.show(ts, msg) for ts, msg in stream]
    assert shows == [True, False, False, True, True, True]
    assert len(b.last) <= 2
    r = PerRobotFilter({"high": 2})
```

```
    assert r.show("r1", 0, "x")
    assert not r.show("r1", 9, "x")
    # another robot, and a stricter severity
    assert r.show("r2", 1, "x")
    assert r.show("r1", 1, "y", "high")
    assert r.show("r1", 4, "y", "high")
```

FUNCTION AND TEST CASES

Signature

```
def filter_messages(events: list[tuple[int, str]]) -> list
[tuple[int, str]]:
```

Sample call

```
filter_messages([])
```

Test cases

Input	Expected
`([],)`	`[]`
`([[10, "solar panel activated"], [13, "solar panel activated"], [20, "solar panel activated"], [21, "solar panel activated"]],)`	`[[10, "solar panel activated"], [21, "solar panel activated"]]`

ALSO REPORTED · 1 more

Robot Status Message Deduplication Window | 1Point3Acres | onsite-coding | asked 2026-09-10 | Low

https://www.1point3acres.com/interview/problems/company/google/robot-status-message-deduplication

ALSO REPORTED · 1 more

The state transition occurs only on a displayed event. Updating on every receipt changes the semantics and can suppress a frequently repeated message forever.
Resolve the exact-boundary rule before coding and make the comparison match that contract.
For out-of-order arrival, clarify how the 10-second interval applies to late events and whether an earlier print decision can ever be revisited. These semantics are not specified by the logger variant; do not assume the chronological-stream contract resolves them.

Preparation: Implement the stream processor with per-message state, then trace repeated and interleaved messages by hand. Write tests for the first occurrence, an event just inside the window, exactly at the boundary, just outside it, and a hidden event followed by an allowed display.

STUDY HERE: topic 2, Arrays, Hashing and Two Pointers

Site tags: python, hashmap, data-analysis, sql, medium

MEDIUM | 1Point3Acres | phone-screen | 16 of 38

Website Activity Analytics: Unique Users + Session Time

ASKED 2026-06-26 | 116 DAYS AGO | LAST 6 MONTHS

STUDY: READ **NO LEETCODE FREQUENCY** **PRIORITY 217/228**

ABOUT THIS QUESTION

Topic: Arrays, Hashing and Two Pointers

Optimal answer from: written and tested here for this exact question

Frequency: Single report | Last asked: 2026-06-26 | Roles: DS

Original: https://www.1point3acres.com/interview/problems/company/google/website-activity-analytics-unique-users-session-time

QUESTION

Given website action records with **`user_id`**, timestamp, and activity, compute per-activity unique-user counts and average time spent on the site per user in seconds. The screen paired this Python task with a lighter SQL section and emphasized edge-case discussion over perfect final code.

Input is a collection of user-activity records. Each record contains:

user_id: numeric user identifier
timestamp: timestamp string such as 2024-07-26 10:00:00
activity: action name such as login, view, purchase, or logout

Implement two analytics tasks

Return a mapping from each activity type to the number of unique users who performed that activity.
Calculate the average time spent on the website per user in seconds.

Clarify session semantics before coding. The prompt uses login/logout-style events, but the exact treatment of users without an explicit logout, multiple sessions per user, and out-of-order records must be agreed with the interviewer.

EXAMPLES

```
user_activity = [
    {'user_id': 1, 'timestamp': '2024-07-26
        10:00:00', 'activity': 'login'},
    {'user_id': 2, 'timestamp': '2024-07-26
        10:05:00', 'activity': 'login'},
    {'user_id': 1, 'timestamp': '2024-07-26
        10:10:00', 'activity': 'view'},
    {'user_id': 1, 'timestamp': '2024-07-26
        10:15:00', 'activity': 'purchase'},
    {'user_id': 2, 'timestamp': '2024-07-26
        10:20:00', 'activity': 'view'},
    {'user_id': 3, 'timestamp': '2024-07-26
        10:25:00', 'activity': 'login'},
    {'user_id': 1, 'timestamp': '2024-07-26
        10:30:00', 'activity': 'logout'},
]
```

APPROACH

Practice writing small Python analytics functions in a plain document without IDE help.
Drill grouping records by user and by activity with dictionaries / sets.
Rehearse explaining incomplete edge-case handling and dry-running a small event stream.

HOW TO THINK ABOUT IT

Unique users per activity is a map from activity to a set of user ids. Time on site needs a definition: pair each login with the next logout of the same user, and say what you do for a login without a logout (here: the session ends at that user's last event). Parse timestamps once, sort each user's events by time, and agree on out-of-order rows and repeated logins before coding.

Basis: Written for this question (a grouping and aggregation task; the SQL half of the screen is the same idea with GROUP BY and COUNT(DISTINCT)).

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
for activity in activities:
    users = set()
    for rec in records:
        if rec.activity == activity:
            users.add(rec.user)
    unique[activity] = len(users)
for user in all_users:
    mine = [e for e in records if e.user == user]
    events = sorted(mine)
    spent = 0
    for login, logout in pairs(events):
        spent += logout.time - login.time
    total[user] = spent
average = sum(total.values()) / len(total)
```

TIME	SPACE
O(U * N + A * N)	O(U + A)

WHY IT IS SLOW

Rescans the whole log for every activity and every user. One pass that groups by activity and by user, then sorts each user's events once, is O(N log N).

OPTIMAL SOLUTION · One pass to group, then sessions per user

TIME	SPACE
O(N log N)	O(N)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Users with several sessions, missing logouts or out-of-order rows.

Sort each user's events by time before pairing; a second login before a logout is ignored (or closes the old session, depending on the contract you agree). A login with no logout ends at the user's last event, or is dropped if you decide it is unreliable. Say the choice out loud: the interviewer accepted a strong discussion over perfect code.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Website Activity Analytics: Unique Users
# + Session Time
#
# Given website action records with user_id, timestamp,
# and activity, compute per-activity unique-user counts
# and average time spent on the site per user in seconds.
# The screen paired this Python task with a lighter SQL
# section and emphasized edge-case discussion over perfect
# final code. Input is a collection of user-activity
# records. Each record contains: user_id: numeric user
# identifier timestamp: timestamp string such as
# 2024-07-26 10:00:00 activity: action name such as login,
# view, purchase, or logout Implement two analytics tasks:
# Return a mapping from each activity type to the number
# of unique users who performed that activity. Calculate
# the average time spent on the website per user in
# seconds. Clarify session semantics before coding.
#
# FOLLOW-UPS (answered below, in this order):
# 1. Users with several sessions, missing logouts or out-
# of-order rows.
#
# INTERVIEW NOTES
# Unique users per activity is a map from activity to a
# set of user ids. Time on site needs a definition: pair
# each login with the next logout of the same user, and
# say what you do for a login without a logout (here: the
# session ends at that user's last event). Parse
# timestamps once, sort each user's events by time, and
# agree on out-of-order rows and repeated logins before
# coding.
#
# BRUTE FORCE (say it first): Rescan all records for every
# user and every activity
# For each activity loop over all records to collect
# users, and for each user loop over all records again to
# compute the session time. Correct, but it rescans the
# data many times. Time O(U * N + A * N). Space O(U + A).


from collections import defaultdict
from datetime import datetime


# BRUTE FORCE (say it first): Rescan all records for every
```

```
# user and every activity | time O(U * N + A * N)
def activity_brute(records):
    from datetime import datetime

    ts = lambda r: datetime.strptime(
        r["timestamp"], "%Y-%m-%d %H:%M:%S"
    )
    users = sorted({r["user_id"] for r in records})
    unique = {
        a: len(
            {
                r["user_id"]
                for r in records
                if r["activity"] == a
            }
        )
        for a in sorted(
            {r["activity"] for r in records}
        )
    }
    total = 0
    for u in users:
        evs = sorted(
            (ts(r), r["activity"])
            for r in records
            if r["user_id"] == u
        )
        start, secs = None, 0
        for t, act in evs:
            if act == "login" and start is None:
                start = t
            elif act == "logout" and start is not None:
                secs += (t - start).total_seconds()
                start = None
        if start is not None:
            secs += (evs[-1][0] - start).total_seconds()
        total += secs
    return unique, (
        total / len(users) if users else 0.0
    )


# HELPERS for the base question
FORMAT = "%Y-%m-%d %H:%M:%S"
```

```
def when(record):
    # timestamp text -> datetime
    return datetime.strptime(record["timestamp"], FORMAT)


# BASE QUESTION (optimal): One pass to group, then
# sessions per user
# Idea: Parse each record once. unique[activity] collects
# user ids in a set.
def unique_users_by_activity(records):
    users = defaultdict(set)
    for r in records:
        # a set counts every user once per activity
        users[r["activity"]].add(r["user_id"])
    return {a: len(u) for a, u in sorted(users.items())}


# BASE QUESTION (optimal): One pass to group, then
# sessions per user
# Idea: Parse each record once. unique[activity] collects
# user ids in a set.
# FOLLOW-UP 1: Users with several sessions, missing
# logouts or out-of-order rows.
# Answer: Sort each user's events by time before pairing;
# a second login before a logout is ignored (or closes the
# old session, depending on the contract you agree).
def session_seconds(events):
    # events: one user's (time, activity), any order
    total, start = 0, None
    # FOLLOW-UP 1: sorted: rows may arrive out of order
    for t, act in sorted(events):
        if act == "login" and start is None:
            # a second login before a logout is ignored
            start = t
```

```
        elif act == "logout" and start is not None:
            total += (t - start).total_seconds()
            start = None
    # a login with no logout ends at the last event seen
    if start is not None:
        last = max(t for t, _ in events)
        total += (last - start).total_seconds()
    return total
```

```
# AVERAGE time on the site per user
def average_time_per_user(records):
    events = defaultdict(list)
    for r in records:
        item = (when(r), r["activity"])
        events[r["user_id"]].append(item)
    if not events:
        return 0.0
    # state the denominator: every user who appears
    per_user = events.values()
    total = sum(session_seconds(e) for e in per_user)
    return total / len(events)


if __name__ == "__main__":
    def rec(user, minute, activity):
        stamp = f"2024-07-26 10:{minute:02}:00"
        return {"user_id": user, "activity": activity,
                "timestamp": stamp}

    data = [rec(1, 0, "login"), rec(2, 5, "login"),
            rec(1, 10, "view"), rec(1, 15, "purchase"),
            rec(2, 20, "view"), rec(3, 25, "login"),
            rec(1, 30, "logout")]
    assert unique_users_by_activity(data) == {
        "login": 3, "logout": 1, "purchase": 1, "view": 2}
    # user 1: 1800 s, user 2: 900 s (no logout), user 3: 0
    assert average_time_per_user(data) == 900
    assert average_time_per_user([]) == 0.0
    # out-of-order rows are sorted first
    shuffled = [rec(1, 30, "logout"), rec(1, 0, "login")]
    assert average_time_per_user(shuffled) == 1800
    # two sessions add up
    two = [rec(1, 0, "login"), rec(1, 5, "logout"),
           rec(1, 10, "login"), rec(1, 20, "logout")]
    assert average_time_per_user(two) == 900
```

NOTES FROM CANDIDATES

SQL in the same screen was described as simple and covered joins / aggregation style work, but the exact SQL prompt did not leak.
The interviewer accepted a strong draft when all edge cases were discussed, even though the final code did not cover every case.
Spend time up front on timestamp parsing, ordering, repeated login/logout events, and users with missing terminal actions.

PREPARATION

Practice writing small Python analytics functions in a plain document without IDE help.
Drill grouping records by user and by activity with dictionaries / sets.
Rehearse explaining incomplete edge-case handling and dry-running a small event stream.

STUDY HERE: topic 2, Arrays, Hashing and Two Pointers

MEDIUM | PracHub | Technical Screen | 17 of 38

Find a Horizontal Cut That Bisects Rectangle Area

POSTED 2026-06-04 | 138 DAYS AGO | LAST 6 MONTHS

STUDY: TALK | ALL TIME 8.1 | 30 DAYS 0 | PRIORITY 169/228

ABOUT THIS QUESTION

Topic: Arrays, Hashing and Two Pointers

Optimal answer from: the interview site's own solution

Related in your app (a neighbouring problem, not this exact question): LeetCode #850 Rectangle Area II

LeetCode match: No direct match; related LC 850 Rectangle Area II (inferred from overview)

Posted on PracHub: 2026-06-04 (updated 2026-08-07)

Original: https://prachub.com/coding-questions/find-a-horizontal-cut-that-bisects-rectangle-area

QUESTION

Overview: Solve a computational-geometry problem that asks for the horizontal line dividing the combined area of many disjoint rectangles in half. The exercise emphasizes numerical precision, large coordinate ranges, performance at 200,000 rectangles, and careful treatment of partially intersected shapes.

APPROACH

Compute total area and the covered width per y-slab via a sweep; walk slabs accumulating area until half, then interpolate inside the slab. Use exact arithmetic for precision.

INTERVIEW-DAY PLAN

CLARIFY FIRST

Ask whether the rectangles are disjoint (the overview says so), the coordinate range, whether the answer must be exact (use fractions) and which y to return if a gap makes many cuts valid (the lowest).

BRUTE FORCE TO SAY OUT LOUD

Binary search on y, summing the rectangle area below y each time. O(n log precision), but floating point error is a risk.

OPTIMAL IDEA

Because the rectangles are disjoint, the area below y is a piecewise linear function whose slope is the sum of the widths of the rectangles spanning y. Create events at each rectangle's bottom (slope increases by its width) and top (slope decreases), sort by y, and walk the slabs accumulating area. When the next slab would pass half of the total, solve inside it: y = y0 + (half - accumulated) / slope, using integer or Fraction arithmetic. O(n log n) for 200,000 rectangles.

DRY-RUN THESE CASES

INTERVIEW-DAY PLAN

One rectangle (the middle), two rectangles separated by a gap (slope zero there), the half exactly at a rectangle boundary, huge coordinates.

LIKELY FOLLOW-UPS

Overlapping rectangles (LeetCode 850, needs a segment tree over x), or a vertical cut.

PITFALLS

Floating point error when comparing areas. Not handling slabs with slope zero.

HOW TO THINK ABOUT IT

Area below a line y = c is a piecewise linear function of c, and between two consecutive rectangle edges its slope is just the total width of rectangles crossing that height (they are disjoint, so widths add). Walk the slabs between events, accumulate area, and when half the area falls inside a slab solve a linear equation. Use exact fractions so the answer is exact.

Basis: Written; a sweep over y events (compare LeetCode 850, Rectangle Area II, which is the harder overlapping version). Assumed rule: axis-aligned rectangles (x1, y1, x2, y2) that do not overlap; return the lowest y with half the total area below it.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute(rects):
    total = sum(area(r) for r in rects)
    lo = lowest_edge(rects)
    hi = highest_edge(rects)
    for _ in range(100):
        mid = (lo + hi) / 2
        if area_below(rects, mid) >= total / 2:
            hi = mid
        else:
            lo = mid
    return hi
```

TIME	SPACE
O(R * iterations)	O(1)

WHY IT IS SLOW

Each step sums over all rectangles and the answer is only as precise as floating point allows. A sweep over sorted edges with exact fractions finds the slab and the cut directly in O(R log R).

OPTIMAL SOLUTION · Sweep the y events and interpolate inside one slab

TIME	SPACE
O(R log R)	O(R)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Rectangles may overlap (LeetCode 850).

The slope is then the union width of the rectangles crossing a height, not the sum. Maintain it with a segment tree over compressed x coordinates (cover counts), O(R log R) overall.

FOLLOW-UP 2 Coordinates up to 10^9 and 200,000 rectangles.

Use integers or Fractions (areas reach 10^18, which fits in Python ints but not in 32-bit). The sweep is O(R log R); never use floats for the comparisons that decide the slab.

FOLLOW-UP 3 A vertical cut instead, or a cut at a given ratio.

Swap the axes for a vertical cut. For a ratio p, replace half by p * total in the slab test; the interpolation formula is unchanged.

FOLLOW-UP 4 A cut that leaves a given fraction of the area below it, not exactly half.

Replace 'half of the total' by p * total in the slab test; the interpolation formula inside the slab is the same. The code returns an exact Fraction and is checked by computing the area below the returned cut.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Find a Horizontal Cut That Bisects
# Rectangle Area
#
# Solve a computational-geometry problem that asks for the
# horizontal line dividing the combined area of many
# disjoint rectangles in half. The exercise emphasizes
# numerical precision, large coordinate ranges,
# performance at 200,000 rectangles, and careful treatment
# of partially intersected shapes.
#
# FOLLOW-UPS (answered below, in this order):
# 1. Rectangles may overlap (LeetCode 850).
# 2. Coordinates up to 10^9 and 200,000 rectangles.
# 3. A vertical cut instead, or a cut at a given ratio.
# 4. A cut that leaves a given fraction of the area below
# it, not exactly half.
#
# INTERVIEW NOTES
# Area below a line y = c is a piecewise linear function
# of c, and between two consecutive rectangle edges its
# slope is just the total width of rectangles crossing
# that height (they are disjoint, so widths add). Walk the
# slabs between events, accumulate area, and when half the
# area falls inside a slab solve a linear equation. Use
# exact fractions so the answer is exact.
#
# BRUTE FORCE (say it first): Binary search on the cut
# height
# The area below y is non-decreasing in y, so bisect y
# between the lowest and highest edge until the area below
# is half the total. It is simple and general, but it
# works in floating point (limited precision) and costs a
# full pass over all rectangles per step. Time O(R *
# iterations). Space O(1).


from collections import defaultdict
from fractions import Fraction


# BRUTE FORCE (say it first): Binary search on the cut
# height | time O(R * iterations)
def _area_below(rects, y):
    return sum(
        (x2 - x1) * max(0, min(y, y2) - y1)
        for x1, y1, x2, y2 in rects
    )


# HELPERS for the base question
def total_area(rects):
    # rects are disjoint (x1, y1, x2, y2)
    return sum((x2 - x1) * (y2 - y1)
               for x1, y1, x2, y2 in rects)


def bisect_cut_search(rects, iterations=100):
    total = sum(
        (x2 - x1) * (y2 - y1)
        for x1, y1, x2, y2 in rects
    )
    lo = min(r[1] for r in rects)
    hi = max(r[3] for r in rects)
    for _ in range(iterations):
        mid = (lo + hi) / 2
        if 2 * _area_below(rects, mid) >= total:
            hi = mid
        else:
            lo = mid
    return hi


# BASE QUESTION (optimal): Sweep the y events and
# interpolate inside one slab
# Idea: Make events (y, +width) at each rectangle's bottom
# and (y, -width) at its top and sort them. Between
# consecutive event heights the slope is the running sum
# of widths.
# FOLLOW-UP 4: A cut that leaves a given fraction of the
# area below it, not exactly half.
# Answer: Replace 'half of the total' by p * total in the
# slab test; the interpolation formula inside the slab is
# the same. The code returns an exact Fraction and is
# checked by computing the area below the returned cut.
def cut_height(rects, share=Fraction(1, 2)):
    # FOLLOW-UP 4: share= leaves any fraction below the
    # cut
    # the height with `share` of the total area below it
    # (1/2 = bisect). Exact Fractions, never floats.
    want = share * total_area(rects)
    # width gained when the sweep passes each height
    events = defaultdict(int)
    for x1, y1, x2, y2 in rects:
        events[y1] += x2 - x1
        events[y2] -= x2 - x1
    ys = sorted(events)
    width, so_far = 0, Fraction(0)
    for y0, y1 in zip(ys, ys[1:]):
        # inside this slab the covered width is constant
        width += events[y0]
        gain = width * (y1 - y0)
        if width and so_far + gain >= want:
```

```
            # the cut is inside this slab: interpolate
            return y0 + (want - so_far) / width
        so_far += gain
    return Fraction(ys[-1])


# HELPERS for the checks below
def area_below(rects, y):
    # exact area under the line at height y (used to
    # check)
    total = Fraction(0)
    for x1, y1, x2, y2 in rects:
        top = min(Fraction(y), y2)
        if top > y1:
            total += (x2 - x1) * (top - y1)
    return total


if __name__ == "__main__":
    stacked = [(0, 0, 2, 1), (0, 1, 2, 2)]
    assert cut_height(stacked) == 1
    # one tall rectangle: halfway up
    assert cut_height([(0, 0, 1, 4)]) == 2
    # a gap between rectangles: the cut sits at the first
    # top
    gap = [(0, 0, 1, 1), (0, 5, 1, 6)]
    assert cut_height(gap) == 1
    # the cut can fall inside a slab with a fractional
    # height
    odd = [(0, 0, 3, 1), (0, 1, 1, 2)]
    y = cut_height(odd)
    assert area_below(odd, y) * 2 == total_area(odd)
    # a different share: 1/4 of the area below
```

```
    q = cut_height(stacked, Fraction(1, 4))
    below = area_below(stacked, q)
    assert below * 4 == total_area(stacked)
    # large coordinates stay exact
    big = [(0, 0, 10 ** 9, 10 ** 9)]
    assert cut_height(big) == Fraction(10 ** 9, 2)
```

FUNCTION AND TEST CASES

Signature

```
def horizontal_bisector(rectangles: list[list[float]]) ->
float:
```

Sample call

```
horizontal_bisector([[0, 0, 4, 2]])
```

Test cases

Input	Expected
`([[0, 0, 4, 2]],)`	`1.0`
`([[0, 0, 2, 4], [10, 1, 2, 2]],)`	`2.0`

ALSO REPORTED · 1 more

Find where to cut cake into equal areas | PracHub | Technical Screen

https://prachub.com/coding-questions/find-where-to-cut-cake-into-equal-areas

STUDY HERE: topic 2, Arrays, Hashing and Two Pointers

Site tags: array, hashmap, medium

MEDIUM | 1Point3Acres | onsite-coding | 18 of 38

Gold Chain Split into Two Equal Halves

ASKED 2026-04-21 | 182 DAYS AGO | OLDER

STUDY: READ | NO LEETCODE FREQUENCY | PRIORITY 201/228

ABOUT THIS QUESTION

Topic: Arrays, Hashing and Two Pointers

Optimal answer from: written and tested here for this exact question

Frequency: Single report | Last asked: 2026-04-21 | Roles: SWE

Original: https://www.1point3acres.com/interview/problems/company/google/gold-chain-split-equal

QUESTION

Onsite coding interview 1: given a gold chain represented by an array of segment weights, remove exactly one segment and check whether the remaining segments — when reconnected — can be cut once more to give two halves of equal total weight. Follow-up: enumerate all valid (remove, cut) pairs.SWEArrayhashmapmediumFrequencySingle reportLast asked2026-04-21Stageonsite-coding

APPROACH

Drill The equal-subset-sum and partition-to-k-equal-sum-subsets problems are the standard drill partners.

Practice enumerating all valid cut positions, not just one — interviewers commonly add the follow-up after the boolean version works.

Watch the boundary: remove + cut should not yield an empty side.

INTERVIEW-DAY PLAN

CLARIFY FIRST

Ask whether removing a segment keeps the rest as a chain joined end to end, whether the cut after removal must leave two non-empty parts, and whether to return a boolean, one answer, or all valid pairs.

BRUTE FORCE TO SAY OUT LOUD

For each removed index, rebuild the array and try every cut with prefix sums. O(n^2).

OPTIMAL IDEA

Compute the total S and prefix sums P. Removing index r leaves total S - a[r], which must be even; the target is half = (S - a[r]) / 2. In the remaining chain a cut is valid when some non-empty prefix sums to half. A prefix that stops before r has sum P[j], and one that includes the segments after r has sum P[j] - a[r]. So for each r check whether half appears among the prefix sums before r, or whether half + a[r] appears among the prefix sums after r (keep a hash set of prefix sums seen so far, and one for those still ahead). Boolean answer in O(n); listing all valid (remove, cut) pairs is output-bound, up to O(n^2).

DRY-RUN THESE CASES

A chain of two segments, a removal at the first or last index, a total that becomes odd, equal weights.

LIKELY FOLLOW-UPS

Enumerate all valid (remove, cut) pairs, or allow removing k segments.

PITFALLS

Allowing an empty half. Forgetting the shift of indices after the removed segment.

INTERVIEW-DAY PLAN

Compute the total S and prefix sums P. Removing index r leaves total S - a[r], which must be even; the target is half = (S - a[r]) / 2. In the remaining chain a cut is valid when some non-empty prefix sums to half. A prefix that stops before r has sum P[j], and one that includes the segments after r has sum P[j] - a[r]. So for each r check whether half appears among the prefix sums before r, or whether half + a[r] appears among the prefix sums after r (keep a hash set of prefix sums seen so far, and one for those still ahead). Boolean answer in O(n); listing all valid (remove, cut) pairs is output-bound, up to O(n^2).

DRY-RUN THESE CASES

A chain of two segments, a removal at the first or last index, a total that becomes odd, equal weights.

LIKELY FOLLOW-UPS

Enumerate all valid (remove, cut) pairs, or allow removing k segments.

PITFALLS

Allowing an empty half. Forgetting the shift of indices after the removed segment.

HOW TO THINK ABOUT IT

After removing segment r the target half is (S - a[r]) / 2, and a cut means some prefix of the remaining chain has exactly that weight. A prefix of the remaining chain is either an ordinary prefix of the original array (stops before r) or an original prefix minus a[r] (passes over r). Each is one value to look up in a hash of prefix sums, so every r costs O(1).

Basis: Written; prefix sums with a hash lookup (like LeetCode 724 and 560). Assumed rule (confirm it): remove exactly one segment, reconnect the remaining chain end to end, then cut it once into two non-empty parts of equal total weight.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute(w):
    for r in range(len(w)):
        chain = w[:r] + w[r + 1:]
        half, run = sum(chain) / 2, 0
        for x in chain:
            run += x
            if run == half:
                return True
    return False
```

TIME	SPACE
O(n^2)	O(n)

WHY IT IS SLOW

n removals, each rebuilt and scanned: O(n^2). Prefix sums with a lookup of the needed prefix value answer each removal in O(1).

OPTIMAL SOLUTION · Prefix sums with first and last occurrence maps

TIME	SPACE
O(n)	O(n)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 List every valid (removed segment, cut position) pair.

The answer itself can be quadratic, so a simple loop is acceptable. With positive weights prefix sums are strictly increasing, so each target has at most one cut position and the listing is O(n) after the hash lookups; with zeros or negatives a value can repeat, so keep a list of indices per prefix value.

FOLLOW-UP 2 The chain is a circle (cut anywhere).

Then a cut is two cut points on the circle and the removed segment just shortens it; use the circular window idea (duplicate the array and slide a window whose sum is half).

FOLLOW-UP 3 Remove k segments instead of one.

The remaining total depends on which k you remove, which becomes a subset-sum style search; ask for the limits before promising a fast solution.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Gold Chain Split into Two Equal Halves
#
# Onsite coding interview 1: given a gold chain
# represented by an array of segment weights, remove
# exactly one segment and check whether the remaining
# segments — when reconnected — can be cut once more to
# give two halves of equal total weight. Follow-up:
# enumerate all valid (remove, cut) pairs.
#
# FOLLOW-UPS (answered below, in this order):
# 1. List every valid (removed segment, cut position)
# pair.
# 2. The chain is a circle (cut anywhere).
# 3. Remove k segments instead of one.
#
# INTERVIEW NOTES
# After removing segment r the target half is (S - a[r]) /
# 2, and a cut means some prefix of the remaining chain
# has exactly that weight. A prefix of the remaining chain
# is either an ordinary prefix of the original array
# (stops before r) or an original prefix minus a[r]
# (passes over r). Each is one value to look up in a hash
# of prefix sums, so every r costs O(1).
#
# BRUTE FORCE (say it first): For each removal rebuild the
# chain and try every cut
# Remove each segment, then test every cut position with a
# running sum. Correct, with a quadratic loop that is the
# baseline to improve. Time O(n^2). Space O(n).


# BRUTE FORCE (say it first): For each removal rebuild the
# chain and try every cut | time O(n^2)
def gold_chain_brute(a):
    n = len(a)
    for r in range(n):
        rest = a[:r] + a[r + 1 :]
        total = sum(rest)
        run = 0
        for c in range(len(rest) - 1):
            run += rest[c]
            if run * 2 == total:
                return True
    return False


# HELPERS for the base question
def prefix_sums(a):
    # P[j] = sum of the first j segments
    P = [0]
    for x in a:
        P.append(P[-1] + x)
    return P


# BASE QUESTION (optimal): Prefix sums with first and last
# occurrence maps
# Idea: Let P[j] be the sum of the first j segments.
def gold_chain(a):
    # remove exactly one segment, then cut once into
    # two non-empty halves of equal weight
    n = len(a)
    if n < 3:
        return False
    P = prefix_sums(a)
    total = P[-1]
    # first and last position of every prefix sum (cuts
    # sit between segments, so positions 1..n-1)
    first, last = {}, {}
    for j in range(1, n):
        first.setdefault(P[j], j)
        last[P[j]] = j
    for r in range(n):
        left_over = total - a[r]
        if left_over % 2:
            continue
        half = left_over // 2
        # cut before the removed segment: P[j] == half
        if half in first and first[half] <= min(r, n - 2):
            return True
        # cut after it: P[j] - a[r] == half
        if half + a[r] in last and last[half + a[r]] > r:
            return True
    return False


# FOLLOW-UP 1: List every valid (removed segment, cut
# position) pair.
# Answer: The answer itself can be quadratic, so a simple
# loop is acceptable.
def valid_pairs(a):
    # follow-up: every (removed index, cut position) pair;
    # the answer can be quadratic, so a loop is fine
    pairs = []
    for r in range(len(a)):
        rest = a[:r] + a[r + 1:]
        total, run = sum(rest), 0
        # cut after `cut` segments of the reconnected
```

```
        # chain
        for cut in range(1, len(rest)):
            run += rest[cut - 1]
            if run * 2 == total:
                pairs.append((r, cut))
    return pairs


if __name__ == "__main__":
    # remove the middle 1, then cut 5 | 5
    assert gold_chain([5, 1, 5])
    assert gold_chain([2, 7, 2])
    assert not gold_chain([1, 2, 3])
    assert not gold_chain([1, 1])
    assert not gold_chain([1, 2, 4])
    assert (1, 1) in valid_pairs([1, 2, 1])
    # both methods agree on many small chains
    import itertools
    for chain in itertools.product(range(1, 5), repeat=5):
        assert gold_chain(list(chain)) == bool(
            valid_pairs(list(chain)))
```

NOTES FROM CANDIDATES

Precompute prefix sums; for each candidate removed index r, the remaining total is S - a[r]. If that is odd, skip. Otherwise check if there is a split point achieving (S - a[r]) / 2.
Use a hashmap from prefix-sum value to index for O(1) lookups -> overall O(N) for the boolean answer, $O(N^2)$ worst case for enumerating all pairs (output-bound).
Watch out for cut positions adjacent to the removed link — must still respect contiguous-chain semantics.

PREPARATION

Drill The equal-subset-sum and partition-to-k-equal-sum-subsets problems are the standard drill partners.
Practice enumerating all valid cut positions, not just one — interviewers commonly add the follow-up after the boolean version works.
Watch the boundary: remove + cut should not yield an empty side.

STUDY HERE: topic 2, Arrays, Hashing and Two Pointers

MEDIUM | PracHub | Technical Screen | 19 of 38

Solve several coding interview problems

POSTED 2026-03-04 | 230 DAYS AGO | OLDER

STUDY: TALK | ALL TIME 38.3 | 30 DAYS 0 | PRIORITY 98/228

ABOUT THIS QUESTION

Topic: Arrays, Hashing and Two Pointers

Optimal answer from: written and tested here for this exact question

This question has several parts, so each part has its own answer below.

Related in your app (a neighbouring problem, not this exact question): LeetCode #160 Intersection of Two Linked Lists

LeetCode match: No direct match (multi-part); related LC 415 Add Strings, LC 160 Intersection of Two Linked Lists (inferred from overview)

Posted on PracHub: 2026-03-04 (updated 2026-04-19)

Original: https://prachub.com/coding-questions/solve-several-coding-interview-problems

QUESTION

Overview: This set of problems evaluates proficiency in string processing and combinatorial parsing, arbitrary-precision arithmetic using string representations, pointer/ancestor-chain intersection detection, and computational geometry for overlap counting, emphasizing algorithmic design, correctness, and performance.

APPROACH

Multiple small tasks: string parsing, big-number string arithmetic, ancestor-chain intersection (hash set or depth alignment), etc. Solve each independently.

INTERVIEW-DAY PLAN

CLARIFY FIRST

My copy lists four small tasks: string parsing with combinatorial counting, adding two big integers given as strings, finding where two ancestor chains intersect, and counting overlaps in a geometry setting. Confirm each task's exact input and output, because the overview is the only statement I have.

BRUTE FORCE TO SAY OUT LOUD

Do each directly: parse and count with recursion, add digit by digit with a carry, compare ancestor lists, and test every pair of shapes.

OPTIMAL IDEA

Big integers: walk both digit strings from the right with a carry and build the result (O(max(m, n))), as in LeetCode 415. Ancestor chains: put one chain in a set and walk the other, or align by depth and step both up until they meet, as in LeetCode 160; O(n) time, O(1) space for the aligned version. Overlap counting: a sweep line over start and end events, taking the running maximum or counting pairs depending on the question. String parsing and counting: define the state and use memoized recursion.

DRY-RUN THESE CASES

Big integers with different lengths, a final carry, leading zeros. Chains that never meet, meet at the head, are identical. Overlaps that touch at a boundary.

LIKELY FOLLOW-UPS

Subtraction or multiplication of big integers, lowest common ancestor with parent pointers, or counting overlapping rectangles.

PITFALLS

Starting to code before the four statements are confirmed. Forgetting the final carry.

HOW TO THINK ABOUT IT

A set of unrelated small tasks: split them, restate each contract, and solve the smallest one first. Big numbers: add digit by digit with a carry. Two chains that meet: walk both and switch to the other head at the end so the lengths cancel. Overlap counting: sort start and end events and keep a running count.

Basis: Three of the small tasks the overview lists, with the standard versions (my copy has only the overview): adding two big integers given as strings (LeetCode 415), intersection of two chains such as ancestors or linked lists (LeetCode 160), and the largest number of overlapping intervals.

BRUTE FORCE · Big numbers: convert to integers and add

BRUTE FORCE · PSEUDO-CODE

```
def add_strings(a, b):
    return str(int(a) + int(b))
```

TIME	SPACE
O(n^2)	O(n)
for the conversions	

WHY IT IS SLOW

Relies on arbitrary-precision integers (not in every language) and the string-to-integer conversions are quadratic for very long inputs. Adding digit by digit from the right with a carry is O(max length).

BRUTE FORCE · Chains that meet: store one chain in a set

BRUTE FORCE · PSEUDO-CODE

```
def intersection(a, b):
    seen = set()
    node = a
    while node:
        seen.add(node)
        node = node.parent
    node = b
    while node:
        if node in seen:
            return node
        node = node.parent
    return None
```

TIME	SPACE
O(m + n)	O(m)

WHY IT IS SLOW

O(m + n) time but O(m) extra memory. Two pointers that switch to the other chain's head at the end meet at the shared node with O(1) memory.

BRUTE FORCE · Overlap: test every time point

BRUTE FORCE · PSEUDO-CODE

```
def max_overlap(intervals):
    lo = min(s for s, e in intervals)
    hi = max(e for s, e in intervals)
    best = 0
    for t in range(lo, hi):
        busy = 0
        for s, e in intervals:
            if s <= t < e:
                busy += 1
        best = max(best, busy)
    return best
```

TIME	SPACE
O(range * n)	O(1)

WHY IT IS SLOW

Depends on the length of the time range, O(range * n). Sorting start and end events and keeping a running count is O(n log n).

OPTIMAL SOLUTION · Big numbers: digit by digit from the right with a carry

TIME	SPACE
O(max(m, n))	O(max(m, n))

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

OPTIMAL SOLUTION · Chains that meet: two pointers that switch heads at the end

TIME	SPACE
O(m + n)	O(1)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

OPTIMAL SOLUTION · Overlap: sweep over sorted start and end events

TIME	SPACE
O(n log n)	O(n)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Ancestors in a tree with parent pointers, not a linked list.

The same two-pointer trick works on parent chains (LeetCode 1650); with depths known, lift the deeper node first and then move both up together.

FOLLOW-UP 2 Add two big numbers stored as linked lists.

If the lists store the least significant digit first, walk both together with the carry (LeetCode 2); if most significant first, reverse them or use stacks.

FOLLOW-UP 3 Count overlapping rectangles instead of intervals.

Sweep over x with a data structure over y (a segment tree with range add and max) to find the largest point depth in O(n log n); the pairwise count is O(n^2) unless you use a sweep with an interval tree.

FOLLOW-UP 4 Add two big numbers stored as linked lists (least significant digit first).

Walk both lists together with a carry and build the result list as you go; no reversal needed. O(max(m, n)) time. If the most significant digit comes first, reverse the lists or use stacks.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Solve several coding interview problems
#
# This set of problems evaluates proficiency in string
# processing and combinatorial parsing, arbitrary-
# precision arithmetic using string representations,
# pointer/ancestor-chain intersection detection, and
```

```
# computational geometry for overlap counting, emphasizing
# algorithmic design, correctness, and performance.
#
# FOLLOW-UPS (answered below, in this order):
# 1. Ancestors in a tree with parent pointers, not a
# linked list.
# 2. Add two big numbers stored as linked lists.
# 3. Count overlapping rectangles instead of intervals.
# 4. Add two big numbers stored as linked lists (least
# significant digit first).
#
# INTERVIEW NOTES
# A set of unrelated small tasks: split them, restate each
# contract, and solve the smallest one first. Big numbers:
# add digit by digit with a carry. Two chains that meet:
# walk both and switch to the other head at the end so the
# lengths cancel. Overlap counting: sort start and end
# events and keep a running count.
#
# BRUTE FORCE (say it first): Big numbers: convert to
# integers and add
# int(a) + int(b), then str. Valid in Python, but it hides
# the algorithm and relies on arbitrary-precision
# integers. Time O(n^2) for the conversions. Space O(n).


# BRUTE FORCE (say it first): Big numbers: convert to
# integers and add | time O(n^2) for the conversions
def add_strings_int(a, b):
    return str(int(a) + int(b))


# HELPERS for follow-up 2
class ListNode:
    def __init__(self, val=0, next=None):
        self.val, self.next = val, next


# BASE QUESTION (optimal): Big numbers: digit by digit
```

```
# from the right with a carry
# Idea: Walk both strings from the right, add the two
# digits and the carry, append total % 10, carry total //
# 10; after the loop add a final carry digit if needed and
# reverse.
def add_strings(a, b):
    # big numbers kept as digit strings, added from the
    # right
    i, j, carry, out = len(a) - 1, len(b) - 1, 0, []
    while i >= 0 or j >= 0 or carry:
        total = carry
        if i >= 0:
            total += int(a[i])
        if j >= 0:
            total += int(b[j])
        out.append(str(total % 10))
        carry = total // 10
        i, j = i - 1, j - 1
    return "".join(out[::-1])


# BASE QUESTION (optimal): Big numbers: digit by digit
# from the right with a carry
# Idea: Walk both strings from the right, add the two
# digits and the carry, append total % 10, carry total //
# 10; after the loop add a final carry digit if needed and
# reverse.
def intersection(head_a, head_b):
    # two chains meeting: each pointer walks its chain and
    # then
    # the other one, so both walk the same total length
    a, b = head_a, head_b
    while a is not b:
```

```
        a = a.next if a else head_b
        b = b.next if b else head_a
    return a


# BASE QUESTION (optimal): Big numbers: digit by digit
# from the right with a carry
# Idea: Walk both strings from the right, add the two
# digits and the carry, append total % 10, carry total //
# 10; after the loop add a final carry digit if needed and
# reverse.
def overlap_depth(rects):
```

```
    # rects (x1, y1, x2, y2), half-open: the largest
    # number of
    # rectangles sharing one point. The best point can be
    # a
    # rectangle's lower-left corner, so test those.
    best = 0
    for x, y, _, _ in rects:
        inside = [r for r in rects if r[0] <= x < r[2]
                  and r[1] <= y < r[3]]
        best = max(best, len(inside))
    return best


# FOLLOW-UP 2: Add two big numbers stored as linked lists.
# Answer: If the lists store the least significant digit
# first, walk both together with the carry (LeetCode 2);
# if most significant first, reverse them or use stacks.
# FOLLOW-UP 4: Add two big numbers stored as linked lists
# (least significant digit first).
# Answer: Walk both lists together with a carry and build
# the result list as you go; no reversal needed. O(max(m,
# n)) time. If the most significant digit comes first,
# reverse the lists or use stacks.
def add_lists(a, b):
    # digits stored least significant first: add with a
    # carry
    dummy = tail = ListNode()
    carry = 0
    while a or b or carry:
        total = carry
        total += a.val if a else 0
        total += b.val if b else 0
        tail.next = ListNode(total % 10)
        tail = tail.next
        carry = total // 10
        a = a.next if a else None
        b = b.next if b else None
    return dummy.next


# FOLLOW-UP 3: Count overlapping rectangles instead of
# intervals.
# Answer: Sweep over x with a data structure over y (a
# segment tree with range add and max) to find the largest
# point depth in O(n log n); the pairwise count is O(n^2)
```

```
# unless you use a sweep with an interval tree.
def overlapping_pairs(rects):
    # how many pairs of rectangles overlap?
    count = 0
    for i in range(len(rects)):
        for j in range(i + 1, len(rects)):
            a, b = rects[i], rects[j]
            if a[0] < b[2] and b[0] < a[2] \
                    and a[1] < b[3] and b[1] < a[3]:
                count += 1
    return count


# HELPERS for the checks below
```

```
def to_list(node):
    out = []
    while node:
        out.append(node.val)
        node = node.next
    return out


if __name__ == "__main__":
    assert add_strings("999", "1") == "1000"
    assert add_strings("0", "0") == "0"
    big = add_strings("123456789123456789", "987654321")
    assert big == str(123456789123456789 + 987654321)
    shared = ListNode(8, ListNode(9))
    a = ListNode(1, ListNode(2, shared))
    b = ListNode(5, shared)
    assert intersection(a, b) is shared
    # no meeting: both end at None
    assert intersection(ListNode(1), ListNode(2)) is None
    # 342 + 465 = 807 with digits reversed
    x = ListNode(2, ListNode(4, ListNode(3)))
    y = ListNode(5, ListNode(6, ListNode(4)))
    assert to_list(add_lists(x, y)) == [7, 0, 8]
    rects = [(0, 0, 4, 4), (2, 2, 6, 6), (3, 3, 5, 5)]
    assert overlap_depth(rects) == 3
    assert overlapping_pairs(rects) == 3
    assert overlap_depth([]) == 0
```

FUNCTION AND TEST CASES

Signature

```
def solution(s) -> list[str]:
```

Sample call

```
solution('.-')
```

Test cases

Input	Expected
`('.-',)`	`['A', 'ET']`
`('...',)`	`['EEE', 'EI', 'IE', 'S']`

STUDY HERE: topic 2, Arrays, Hashing and Two Pointers

MEDIUM | PracHub | Technical Screen | 20 of 38

★ TOP 75: rank 13 in the Medium and Hard priority list

★ Solve array-sum and city-community problems

POSTED 2026-02-20 | 242 DAYS AGO | OLDER

STUDY: TYPE | ALL TIME 66 | 30 DAYS 46.9 | PRIORITY 7/228

ABOUT THIS QUESTION

Topic: Arrays, Hashing and Two Pointers

Optimal answer from: written and tested here for this exact question

This question has several parts, so each part has its own answer below.

Related in your app (a neighbouring problem, not this exact question): LeetCode #560 Subarray Sum Equals K, LeetCode #200 Number of Islands

LeetCode match: LC 560 Subarray Sum Equals K + LC 200 Number of Islands (inferred from overview)

Posted on PracHub: 2026-02-20 (updated 2026-04-19)

Original: https://prachub.com/coding-questions/solve-array-sum-and-city-community-problems

QUESTION

Overview: This question evaluates array-processing and grid/graph connectivity skills, covering counting subarrays with a target sum and identifying connected building communities in a grid; it belongs to the Coding & Algorithms domain and involves streaming/online algorithm reasoning as well as grid traversal concepts.

APPROACH

Prefix sum + hash map of counts for the subarray target; communities = connected components via DFS/union-find on the grid.

INTERVIEW-DAY PLAN

CLARIFY FIRST

Two parts. Array: count subarrays whose sum equals K; are values negative (decides the method) and can the input be a stream? Grid: building communities as connected components, is adjacency four-way, and what marks a building. The scraped solution covers only the array part.

BRUTE FORCE TO SAY OUT LOUD

Array: all subarrays with running sums, O(n^2). Grid: none needed, a flood fill is the baseline.

OPTIMAL IDEA

Array: prefix sums with a dict of how many times each prefix has occurred; for each element add the count of (prefix - K), then record the prefix. O(n) and it works online for a stream. A sliding window fails with negative values. Grid: DFS/BFS (iterative) or Union-Find to count connected components of building cells.

INTERVIEW-DAY PLAN

DRY-RUN THESE CASES

Array: [1,1,1] with K = 2 gives 2, [1,2,3] with K = 3 gives 2, negatives, K = 0 with zeros, an empty array. Grid: no buildings, one building, diagonal buildings (separate).

LIKELY FOLLOW-UPS

Array: longest subarray with sum K, or counting in a stream. Grid: the largest community, or communities as buildings are added.

PITFALLS

Not seeding the prefix dict with {0: 1}. Using a window technique with negative numbers.

HOW TO THINK ABOUT IT

Subarray sums with negatives allowed cannot use a sliding window; use prefix sums: a subarray ending here sums to K exactly when an earlier prefix equals (current prefix - K), so a dictionary of prefix counts answers it in one pass and works for streams. Communities are connected components, so flood fill or union-find over the building cells.

Basis: Two tasks: count subarrays with sum equal to K (LeetCode 560) and count connected building communities in a grid (LeetCode 200 idea). Assumed rule: buildings are '1' cells, communities connect horizontally and vertically.

BRUTE FORCE · Task 1: try every subarray

BRUTE FORCE · PSEUDO-CODE

```
def brute(nums, K):
    count = 0
    for i in range(len(nums)):
        total = 0
        for j in range(i, len(nums)):
            total += nums[j]
            if total == K:
                count += 1
    return count
```

TIME	SPACE
O(n^2)	O(1)

WHY IT IS SLOW

O(n^2). A dictionary of prefix sums (how many earlier prefixes equal current - K) is O(n) and also works on a stream and with negative numbers.

BRUTE FORCE · Task 2: relabel cells until the labels stop changing

BRUTE FORCE · PSEUDO-CODE

BRUTE FORCE · Task 2: relabel cells until the labels stop changing

```
label = {}
for i, cell in enumerate(buildings):
    label[cell] = i
changed = True
while changed:
    changed = False
    for cell in buildings:
        group = [cell] + neighbours(cell)
        best = min(label[x] for x in group)
        if best < label[cell]:
            label[cell] = best
            changed = True
answer = len(set(label.values()))
```

TIME	SPACE
O((m * n)^2) worst case	O(m * n)

WHY IT IS SLOW

Labels spread one step per round, so a long snake needs about m * n rounds of O(m * n): O((m * n)^2). Union-Find or one flood fill is near-linear.

OPTIMAL SOLUTION · Task 1: prefix sums in a dictionary

TIME	SPACE
O(n)	O(n)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

OPTIMAL SOLUTION · Task 2: Union-Find over building cells

TIME	SPACE
O(m * n * alpha(m * n))	O(m * n)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 The numbers are all positive.

A sliding window works in O(1) memory: grow the right end, shrink the left while the sum exceeds K. With negatives the window is not monotone, which is why prefix sums are needed.

FOLLOW-UP 2 Longest subarray with sum K.

Store the first index of each prefix sum instead of a count, and take the longest distance to prefix - K.

FOLLOW-UP 3 Buildings are added over time and you report the number of communities after each addition.

Union-Find with a counter: +1 for the new building, -1 for each union with a different neighbouring community (LeetCode 305).

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Solve array-sum and city-community
# problems
#
# This question evaluates array-processing and grid/graph
# connectivity skills, covering counting subarrays with a
# target sum and identifying connected building
# communities in a grid; it belongs to the Coding &
# Algorithms domain and involves streaming/online
# algorithm reasoning as well as grid traversal concepts.
#
# FOLLOW-UPS (answered below, in this order):
# 1. The numbers are all positive.
# 2. Longest subarray with sum K.
# 3. Buildings are added over time and you report the
# number of communities after each addition.
#
# INTERVIEW NOTES
# Subarray sums with negatives allowed cannot use a
# sliding window; use prefix sums: a subarray ending here
# sums to K exactly when an earlier prefix equals (current
# prefix - K), so a dictionary of prefix counts answers it
# in one pass and works for streams. Communities are
# connected components, so flood fill or union-find over
# the building cells.
#
# BRUTE FORCE (say it first): Task 1: try every subarray
# Extend each start with a running sum and count the ends
# where it equals K. Correct and the oracle; quadratic.
# Time O(n^2). Space O(1).


# BRUTE FORCE (say it first): Task 1: try every subarray |
# time O(n^2)
def subarray_sum_brute(nums, k):
    count = 0
    for i in range(len(nums)):
        total = 0
        for j in range(i, len(nums)):
            total += nums[j]
            count += total == k
    return count


# HELPERS for the base question
STEPS = ((1, 0), (-1, 0), (0, 1), (0, -1))
```

```
# BASE QUESTION (optimal): prefix sums in a dictionary
# Idea: Walk the array keeping the prefix sum.
def count_subarrays(nums, k):
    # prefix sums: a subarray ending here has sum k if
    # some
    # earlier prefix equals (prefix - k)
    seen, prefix, count = {0: 1}, 0, 0
    for x in nums:
        prefix += x
```

```
        count += seen.get(prefix - k, 0)
        seen[prefix] = seen.get(prefix, 0) + 1
    return count


# BASE QUESTION (optimal): prefix sums in a dictionary
# Idea: Walk the array keeping the prefix sum.
def communities(grid):
    # connected groups of buildings (1) in a grid
    seen, count = set(), 0
    for r in range(len(grid)):
        for c in range(len(grid[0])):
            if grid[r][c] != 1 or (r, c) in seen:
                continue
            count += 1
            seen.add((r, c))
            stack = [(r, c)]
            while stack:
                x, y = stack.pop()
                for dx, dy in STEPS:
                    a, b = x + dx, y + dy
                    if not (0 <= a < len(grid)):
                        continue
                    if not (0 <= b < len(grid[0])):
                        continue
                    land = grid[a][b] == 1
                    if land and (a, b) not in seen:
                        seen.add((a, b))
                        stack.append((a, b))
    return count


# FOLLOW-UP 1: The numbers are all positive.
# Answer: A sliding window works in O(1) memory: grow the
```

```
# right end, shrink the left while the sum exceeds K. With
# negatives the window is not monotone, which is why
# prefix sums are needed.
def count_positive(nums, k):
    # all numbers positive: a sliding window, O(1) memory
    count = left = total = 0
    for right, x in enumerate(nums):
        total += x
        while total > k and left <= right:
            total -= nums[left]
            left += 1
        if total == k:
            count += 1
    return count


# FOLLOW-UP 2: Longest subarray with sum K.
# Answer: Store the first index of each prefix sum instead
# of a count, and take the longest distance to prefix - K.
def longest_subarray(nums, k):
    # store the FIRST index of each prefix sum
    first, prefix, best = {0: -1}, 0, 0
    for i, x in enumerate(nums):
        prefix += x
        if prefix - k in first:
            best = max(best, i - first[prefix - k])
        first.setdefault(prefix, i)
    return best


# FOLLOW-UP 3: Buildings are added over time and you
# report the number of communities after each addition.
# Answer: Union-Find with a counter: +1 for the new
# building, -1 for each union with a different
# neighbouring community (LeetCode 305).
def communities_online(rows, cols, additions):
    # buildings added one by one: count after each
    # (LeetCode 305)
```

```
    parent, count, out = {}, 0, []

    def find(x):
        while parent[x] != x:
            parent[x] = parent[parent[x]]
            x = parent[x]
        return x
```

```
    for r, c in additions:
        if (r, c) not in parent:
            parent[(r, c)] = (r, c)
            count += 1
            for dr, dc in STEPS:
                nb = (r + dr, c + dc)
                if nb in parent:
                    a, b = find((r, c)), find(nb)
                    if a != b:
                        parent[a] = b
                        # two communities became one
                        count -= 1
        out.append(count)
    return out


if __name__ == "__main__":
    assert count_subarrays([1, 1, 1], 2) == 2
    assert count_subarrays([1, -1, 0], 0) == 3
    assert count_subarrays([], 0) == 0
    assert longest_subarray([1, -1, 5, -2, 3], 3) == 4
    assert longest_subarray([1, 2], 9) == 0
    assert count_positive([1, 2, 3, 2, 1], 3) == 3
    grid = [[1, 1, 0], [0, 0, 1], [1, 0, 1]]
    assert communities(grid) == 3
    adds = [(0, 0), (0, 1), (1, 2), (2, 2), (1, 1)]
    counts = communities_online(3, 3, adds)
    assert counts == [1, 1, 2, 2, 1]
```

FUNCTION AND TEST CASES

Signature

```
def solution(nums: list[int], k: int) -> int:
```

Sample call

```
solution([1, 1, 1], 2)
```

Test cases

Input	Expected
([1, 1, 1], 2)	2
([1, 2, 3], 3)	2

STUDY HERE: topic 2, Arrays, Hashing and Two Pointers

MEDIUM | PracHub | Technical Screen | 21 of 38

Check pivot after removing one element

POSTED 2026-02-12 | 250 DAYS AGO | OLDER

STUDY: TALK | ALL TIME 33.2 | 30 DAYS 0 | PRIORITY 116/228

ABOUT THIS QUESTION

Topic: Arrays, Hashing and Two Pointers

Optimal answer from: the interview site's own solution

Related in your app (a neighbouring problem, not this exact question): LeetCode #724 Find Pivot Index

LeetCode match: LC 724 Find Pivot Index (variant: one removal) (inferred from overview)

Posted on PracHub: 2026-02-12 (updated 2026-05-04)

Original: https://prachub.com/coding-questions/check-pivot-after-removing-one-element

QUESTION

Overview: This question evaluates array-manipulation skills, cumulative-sum/prefix-sum reasoning, and robustness in handling negative values and duplicates, testing algorithmic problem-solving competency in the Coding & Algorithms domain and focusing on practical application rather than purely conceptual understanding.

APPROACH

Prefix sums plus a hash map of left-side sums; for each removal candidate, check whether some index has equal left and right sums after the removal.

INTERVIEW-DAY PLAN

CLARIFY FIRST

The statement is not complete in my copy, so confirm: after removing exactly one element (or at most one), does there exist an index where the sum to the left equals the sum to the right (the pivot itself excluded)? Are negatives and duplicates allowed?

BRUTE FORCE TO SAY OUT LOUD

For each element removed, rebuild the array and test every pivot with prefix sums. O(n^2).

OPTIMAL IDEA

Let P[i] be the sum before index i and S the total. If the removed element r is to the right of the pivot i, the two sides are equal exactly when a[r] = S - 2 * P[i] - a[i]. If r is to the left of i, the condition is a[r] = 2 * P[i] + a[i]

- S. So for every pivot candidate the value that must be removed is determined; look it up in a dict of value to last index (needs an index greater than i) and a dict of value to first index (needs an index less than i). One pass, O(n) time and space. Say the brute force first, and mention you checked the formulas against it on random small arrays.

INTERVIEW-DAY PLAN

DRY-RUN THESE CASES

A one-element array, two elements, negatives, duplicate values, a pivot that exists without any removal, removing the pivot itself (not allowed by most readings).

LIKELY FOLLOW-UPS

Allow at most one removal or remove a pivot candidate, or count all pairs.

PITFALLS

Letting the removed element be the pivot. Forgetting that the prefix sums shift after a removal.

HOW TO THINK ABOUT IT

Use prefix sums. For a pivot at original index i and a removed element on one side, the balance equation fixes the value that has to be removed: it is a closed form in the prefix sum, the total and a[i]. So for every candidate pivot you only have to ask 'does that value occur on the correct side?', which a dictionary of first and last positions answers in O(1).

Basis: Variant of LeetCode 724 (Find Pivot Index). Assumed rule (confirm it): remove exactly one element; is there then an index where the sum on its left equals the sum on its right (the pivot excluded)?

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute(a):
    for r in range(len(a)):
        b = a[:r] + a[r + 1:]
        for i in range(len(b)):
            if sum(b[:i]) == sum(b[i + 1:]):
                return True
    return False
```

TIME	SPACE
O(n^3)	O(n)
O(n^2) with prefix sums	

WHY IT IS SLOW

n removals times n pivots times a sum: O(n^3) (O(n^2) with prefix sums). For each pivot the value to remove is determined by prefix sums, so a dictionary lookup makes it O(n).

OPTIMAL SOLUTION · Closed-form value to remove, found in a dictionary of positions

TIME	SPACE
O(n)	O(n)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Without any removal (LeetCode 724).

One pass: the pivot is the index where left sum equals total minus left minus the element. O(n), O(1).

FOLLOW-UP 2 At most one removal (removal optional).

Answer true if pivot_index finds a pivot, otherwise run the removal test above.

FOLLOW-UP 3 Return which element to remove and the pivot.

Record i together with the index of the matching value (first or last occurrence) when the lookup succeeds, instead of returning True.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Check pivot after removing one element
#
# This question evaluates array-manipulation skills,
# cumulative-sum/prefix-sum reasoning, and robustness in
# handling negative values and duplicates, testing
# algorithmic problem-solving competency in the Coding &
# Algorithms domain and focusing on practical application
# rather than purely conceptual understanding.
#
# FOLLOW-UPS (answered below, in this order):
# 1. Without any removal (LeetCode 724).
# 2. At most one removal (removal optional).
# 3. Return which element to remove and the pivot.
#
# INTERVIEW NOTES
# Use prefix sums. For a pivot at original index i and a
# removed element on one side, the balance equation fixes
# the value that has to be removed: it is a closed form in
# the prefix sum, the total and a[i]. So for every
# candidate pivot you only have to ask 'does that value
# occur on the correct side?', which a dictionary of first
# and last positions answers in O(1).
#
# BRUTE FORCE (say it first): Try every removal and every
# pivot
# For each removed index build the shorter array and test
# every pivot with sums. Obviously correct and the oracle.
# Time O(n^3), O(n^2) with prefix sums. Space O(n).


# BRUTE FORCE (checker)
def brute(a):
    # try every removal and every pivot: the demo's check
    for i in range(len(a)):
        rest = a[:i] + a[i + 1:]
        for j in range(len(rest)):
            if sum(rest[:j]) == sum(rest[j + 1:]):
                return True
    return False


# BASE QUESTION (optimal): Closed-form value to remove,
# found in a dictionary of positions
# Idea: Let P be the prefix sum before i and S the total.
def pivot_after_removal(a):
    # remove ONE element, then is there a pivot: an
    # element
    # whose left sum equals its right sum? Returns
    # (removed
    # index, pivot index) or None. O(n) with two scans.
    n, total = len(a), sum(a)
    prefix = [0]
    for x in a:
        prefix.append(prefix[-1] + x)
    # key[j]: left + left + a[j] when nothing is removed
    key = [2 * prefix[j] + a[j] for j in range(n)]
    # pivot j AFTER the removed index i:
    #   2 * prefix[j] - a[i] = total - a[j]  -> a[i] = key
    # - total
    where, ok = {}, None
    for j in range(n):
        i = where.get(key[j] - total)
        if i is not None:
            return i, j
        where[a[j]] = j
    # pivot j BEFORE the removed index i:
    #   2 * prefix[j] + a[j] = total - a[i]  -> a[i] =
    # total - key
    where = {}
    for j in range(n - 1, -1, -1):
        i = where.get(total - key[j])
        if i is not None:
            return i, j
        where[a[j]] = j
    return ok


# FOLLOW-UP 1: Without any removal (LeetCode 724).
# Answer: One pass: the pivot is the index where left sum
# equals total minus left minus the element. O(n), O(1).
def pivot_index(a):
    # LeetCode 724: no removal, O(1) extra space
    total, left = sum(a), 0
    for i, x in enumerate(a):
        if left == total - left - x:
            return i
        left += x
    return -1
```

```
# FOLLOW-UP 2: At most one removal (removal optional).
# Answer: Answer true if pivot_index finds a pivot,
# otherwise run the removal test above.
def has_pivot(a, allow_removal=True):
    # follow-up: removal is optional
    if pivot_index(a) != -1:
        return True
    if not allow_removal:
        return False
    return pivot_after_removal(a) is not None


if __name__ == "__main__":
    # [1,7,3,6,5,6] has a pivot at index 3 without
    # removal,
    # but removing one element never gives another
    assert pivot_after_removal([1, 7, 3, 6, 5, 6]) is None
    # [1, 1, 1, 1] has no pivot, but removing the element
    # at
    # index 1 leaves [1, 1, 1] whose middle element is one
    assert pivot_index([1, 1, 1, 1]) == -1
    assert pivot_after_removal([1, 1, 1, 1]) == (1, 2)
    assert pivot_index([1, 7, 3, 6, 5, 6]) == 3
    assert pivot_index([1, 2, 3]) == -1
    assert not has_pivot([1, 2, 3], allow_removal=False)
    # check against the brute force on many small arrays
    import itertools
    for n in range(1, 6):
        every = itertools.product([-2, 0, 1, 3], repeat=n)
```

```
        for a in every:
            a = list(a)
            found = pivot_after_removal(a) is not None
            assert found == brute(a)
    assert has_pivot([1, 1, 1, 1])
    ones = [1, 1, 1, 1]
    assert not has_pivot(ones, allow_removal=False)
```

FUNCTION AND TEST CASES

Signature

```
def solution(nums: list[int]) -> bool:
```

Sample call

```
solution([1, 2])
```

Test cases

Input	Expected
`([1, 2],)`	`True`
`([2, 1, 1, 2],)`	`True`

STUDY HERE: topic 2, Arrays, Hashing and Two Pointers

MEDIUM | PracHub | Technical Screen | 22 of 38

Min deletions to avoid overlap in first k

POSTED 2026-02-12 | 250 DAYS AGO | OLDER

STUDY: TALK | NO LEETCODE FREQUENCY | PRIORITY 195/228

ABOUT THIS QUESTION

Topic: Arrays, Hashing and Two Pointers

Optimal answer from: the interview site's own solution

LeetCode match: No direct match (inferred) (inferred from overview)

Posted on PracHub: 2026-02-12 (updated 2026-04-22)

Original: https://prachub.com/coding-questions/min-deletions-to-avoid-overlap-in-first-k

QUESTION

Overview: This question evaluates array and sequence manipulation skills, set-based reasoning about value overlap, and the ability to compute minimal edits while preserving relative order constraints.

APPROACH

Likely: compare the first k elements with the rest via sets and delete the minimum elements to remove value overlap while preserving order; confirm details on the PracHub page.

INTERVIEW-DAY PLAN

CLARIFY FIRST

This prompt is vague, so ask for the exact rule first. A common reading: the first k elements and the remaining elements must share no value, delete the fewest elements. Ask whether deleted elements must be removed from the first part or either part, and whether order matters.

BRUTE FORCE TO SAY OUT LOUD

Try every subset of deletions. Exponential.

OPTIMAL IDEA

Under that reading values are independent. For each value that appears in both parts, you must remove all its copies from one side, so you delete the smaller of its two counts. Sum over shared values. Count with two hash maps. O(n).

DRY-RUN THESE CASES

No shared values (0), every value shared, k = 0 or k = n, repeated values on one side only.

LIKELY FOLLOW-UPS

Delete only from the first part, weights on elements, or k given as a range.

PITFALLS

Starting to code before the rule is confirmed. Deleting elements one at a time greedily.

HOW TO THINK ABOUT IT

Values do not interact with each other, so each value can be decided on its own. A value that appears on both sides must disappear from one side entirely, and the cheapest side is the one with fewer copies. The answer is the sum, over values that appear on both sides, of the smaller of the two counts.

Basis: The statement is vague in my copy. Assumed rule (confirm it): given an array and k, the first k elements and the remaining elements must share no value; delete the fewest elements (from either part) to achieve that.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute(a, k):
    for size in range(len(a) + 1):
        ids = range(len(a))
        for gone in combinations(ids, size):
            keep = [i for i in ids
                    if i not in gone]
            head = {a[i] for i in keep if i < k}
            tail = {a[i] for i in keep if i >= k}
            if not head & tail:
                return size
```

TIME	SPACE
O(2^n * n)	O(n)

WHY IT IS SLOW

2^n sets to try: exponential. Each shared value must disappear from one side, so the cost is the sum over shared values of the smaller of its two counts, in O(n).

OPTIMAL SOLUTION · Sum of min(count in the first k, count in the rest) over shared values

TIME	SPACE
O(n)	O(n)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Only elements of the first part may be deleted.

Then every shared value must be removed from the first part entirely, so the cost is the number of first-part elements whose value occurs in the rest (a set lookup per element).

FOLLOW-UP 2 Elements have weights (deleting some is more costly).

Replace counts by the sum of weights per side for each value; the choice per value is still independent, so take the cheaper side. **Cost:** unchanged O(n); the per-value choice sums weights instead of counts.

FOLLOW-UP 3 The statement may differ from this reading.

Ask whether deletions can come from both parts, whether duplicates count separately, and whether order matters. Everything above relies on those three answers.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Min deletions to avoid overlap in first k
#
# This question evaluates array and sequence manipulation
# skills, set-based reasoning about value overlap, and the
# ability to compute minimal edits while preserving
# relative order constraints.
#
# FOLLOW-UPS (answered below, in this order):
# 1. Only elements of the first part may be deleted.
# 2. Elements have weights (deleting some is more costly).
# 3. The statement may differ from this reading.
#
# INTERVIEW NOTES
# Values do not interact with each other, so each value
# can be decided on its own. A value that appears on both
# sides must disappear from one side entirely, and the
# cheapest side is the one with fewer copies. The answer
# is the sum, over values that appear on both sides, of
# the smaller of the two counts.
#
# BRUTE FORCE (say it first): Try every set of deletions
# Enumerate subsets of elements to delete in increasing
# size, and return the first size that leaves the two
# parts disjoint. Exact and a good oracle, but
# exponential. Time O(2^n * n). Space O(n).


import itertools
from collections import Counter


# BRUTE FORCE (say it first): Try every set of deletions |
# time O(2^n * n)
def min_deletions_brute(a, k):
    n = len(a)
    for size in range(n + 1):
        for gone in itertools.combinations(
            range(n), size
        ):
            left = {
                a[i]
                for i in range(min(k, n))
                if i not in gone
            }
            right = {
```

```
                a[i]
                for i in range(min(k, n), n)
                if i not in gone
            }
            if not left & right:
                return size
    return n


# BASE QUESTION (optimal): Sum of min(count in the first
# k, count in the rest) over shared values
# Idea: Count each value in the first k elements and in
# the rest. For every value with a positive count on both
# sides, one side has to lose all of its copies, so add
# the smaller count.
def min_deletions(a, k):
    # the values in a[:k] and a[k:] must not overlap; for
    # each
    # shared value delete the copies on the cheaper side
    left, right = Counter(a[:k]), Counter(a[k:])
    return sum(min(left[v], right[v])
               for v in left.keys() & right.keys())


# PLAN: which indices to delete
def deletions_plan(a, k):
    # which indices to delete (both sides allowed)
    left, right = Counter(a[:k]), Counter(a[k:])
    out = []
    for v in left.keys() & right.keys():
        # delete the cheaper side's copies of the value
        side = range(k, len(a))
        if left[v] <= right[v]:
            side = range(k)
        out += [i for i in side if a[i] == v]
    return sorted(out)


# FOLLOW-UP 1: Only elements of the first part may be
# deleted.
# Answer: Then every shared value must be removed from the
# first part entirely, so the cost is the number of first-
# part elements whose value occurs in the rest (a set
# lookup per element).
def min_deletions_first_only(a, k):
```

```
    # only the first part may lose elements: every first-
    # part
    # element whose value appears later must go
    rest = set(a[k:])
    return sum(1 for x in a[:k] if x in rest)


# FOLLOW-UP 2: Elements have weights (deleting some is
# more costly).
# Answer: Replace counts by the sum of weights per side
# for each value; the choice per value is still
# independent, so take the cheaper side. Cost: unchanged
# O(n); the per-value choice sums weights instead of
# counts.
def min_deletions_weighted(a, weights, k):
    # deleting element i costs weights[i]: per value take
    # the
    # cheaper side as a whole
    left, right = Counter(), Counter()
    for i, x in enumerate(a):
        (left if i < k else right)[x] += weights[i]
    return sum(min(left[v], right[v])
               for v in left.keys() & right.keys())


if __name__ == "__main__":
    a = [1, 2, 2, 3, 2, 1]
    # first part [1, 2, 2], rest [3, 2, 1]: shared 1 and 2
    assert min_deletions(a, 3) == 1 + 1
    assert min_deletions_first_only(a, 3) == 3
    assert min_deletions([1, 2, 3], 1) == 0
    assert min_deletions([], 0) == 0
    # a heavy copy on one side makes the other side
    # cheaper
    w = [1, 5, 5, 1, 1, 1]
    assert min_deletions_weighted(a, w, 3) == 1 + 1
    plan = deletions_plan(a, 3)
    kept = [x for i, x in enumerate(a) if i not in plan]
    kept_left = set(kept[:3 - sum(i < 3 for i in plan)])
```

```
    kept_right = set(kept[3 - sum(i < 3 for i in plan):])
    assert not kept_left & kept_right
    assert len(plan) == min_deletions(a, 3)
```

FUNCTION AND TEST CASES

Signature

```
def solution(list1: list[int], list2: list[int], k: int) -
> int:
```

Sample call

```
solution([1, 2, 3], [2, 1, 4, 5], 2)
```

Test cases

Input	Expected
([1, 2, 3], [2, 1, 4, 5], 2)	2
([1, 2, 3, 4], [2, 5, 3, 6], 2)	1

STUDY HERE: topic 2, Arrays, Hashing and Two Pointers

Site tags: array, two-pointer, medium

MEDIUM | 1Point3Acres | onsite-coding | 23 of 38

Sum of Good Subarrays (Adjacent Diff ±1)

ASKED 2026-02-08 | 254 DAYS AGO | OLDER

STUDY: READ | NO LEETCODE FREQUENCY | PRIORITY 208/228

ABOUT THIS QUESTION

Topic: Arrays, Hashing and Two Pointers

Optimal answer from: written and tested here for this exact question

Frequency: Single report | Last asked: 2026-02-08 | Roles: MLE, SWE

Original: https://www.1point3acres.com/interview/problems/company/google/good-subarrays-diff-one

QUESTION

MLE L5 coding round: sum over all contiguous subarrays whose elements form a strictly monotonic ±1 step sequence (every adjacent pair differs by exactly +1, OR every adjacent pair differs by exactly -1). Required time complexity is O(N).

Input: integer array arr of length N.
A subarray is good if its consecutive differences are either all +1 (monotone increasing by 1) or all -1 (monotone decreasing by 1).
Singleton subarrays are always good.
Output: the sum of element-sums across all good subarrays (each good subarray contributes the sum of its elements once).
Required: O(N) time.

EXAMPLES

```
arr = [3, 5, 6, 7, 6]
```

Good subarrays

```
  singletons:           [3], [5], [6], [7], [6]
  monotone +1 runs:     [5, 6], [6, 7], [5, 6, 7]
  monotone -1 runs:     [7, 6]
Not good: [3, 5] (diff 2), [6, 7, 6] (mixes +1 then
    -1).
```

APPROACH

Practice the "contribution-per-index" pattern: the sum-of-subarray-minimums and sum-of-subsequence-widths problems both use the same technique.
Practice splitting an array into maximal monotone-step runs, then computing per-run contributions in O(1) amortized using prefix sums of element values.
Sanity-check with arr = [1, 2, 3] and arr = [3, 2, 1] by hand — both should sum to 20.

HOW TO THINK ABOUT IT

A good subarray is a stretch where every step is +1 or every step is -1, so every good subarray ending at index i lies inside the current run. Instead of listing subarrays, keep for each direction the total of all suffix sums inside the run: extending a run of length L by a new element x adds (L + 1) * x to that total. Both directions can be updated in O(1), and the singleton is counted once.

Basis: Written for this question (a run-length DP in the style of LeetCode 2262, Total Appeal of A String).

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute(a):
    total = 0
    for i in range(len(a)):
        for j in range(i, len(a)):
            part = a[i:j + 1]
            if is_good(part):
                total += sum(part)
    return total
def is_good(part):
    pairs = zip(part, part[1:])
    steps = {y - x for x, y in pairs}
    return steps <= {1} or steps <= {-1}
```

TIME	SPACE
O(n^2) or O(n^3)	O(1)

WHY IT IS SLOW

O(n^2) subarrays (O(n^3) with the sums). Run totals per direction update in O(1) per element, so one pass is O(n).

OPTIMAL SOLUTION · Two running run totals

TIME	SPACE
O(n)	O(1)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Why is the singleton subtracted?

A one-element subarray [x] is both an increasing and a decreasing run of length 1, so it appears in both running totals. Subtracting x once at each index keeps it counted exactly once. A subarray of length 2 or more can only be in one total because its step is +1 or -1, not both.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```python
# MAIN QUESTION: Sum of Good Subarrays (Adjacent Diff ±1)
#
# MLE L5 coding round: sum over all contiguous subarrays
# whose elements form a strictly monotonic ±1 step
# sequence (every adjacent pair differs by exactly +1, OR
# every adjacent pair differs by exactly -1). Required
# time complexity is O(N). Input: integer array arr of
# length N. A subarray is good if its consecutive
# differences are either all +1 (monotone increasing by 1)
# or all -1 (monotone decreasing by 1). Singleton
# subarrays are always good. Output: the sum of element-
# sums across all good subarrays (each good subarray
# contributes the sum of its elements once). Required:
# O(N) time. Examples: arr = [3, 5, 6, 7, 6] Good
# subarrays: singletons: [3], [5], [6], [7], [6] monotone
# +1 runs: [5, 6], [6, 7], [5, 6, 7] monotone -1 runs: [7,
# 6] Not good: [3, 5] (diff 2), [6, 7, 6] (mixes +1 then
# -1).
#
# FOLLOW-UPS (answered below, in this order):
# 1. Why is the singleton subtracted?
#
# INTERVIEW NOTES
# A good subarray is a stretch where every step is +1 or
# every step is -1, so every good subarray ending at index
# i lies inside the current run. Instead of listing
# subarrays, keep for each direction the total of all
# suffix sums inside the run: extending a run of length L
# by a new element x adds (L + 1) * x to that total. Both
# directions can be updated in O(1), and the singleton is
# counted once.
#
# BRUTE FORCE (say it first): Enumerate every subarray
# Check every subarray's differences and add up its
# elements. Correct and the oracle; quadratic or worse,
# but the problem asks for O(n). Time O(n^2) or O(n^3).
# Space O(1).


# BRUTE FORCE (checker)
def good_subarray_sum_brute(a):
    # check every subarray: the oracle for the demo
    total = 0
    for i in range(len(a)):
        for j in range(i, len(a)):
```

```python
            steps = {a[k + 1] - a[k] for k in range(i, j)}
            if steps <= {1} or steps <= {-1}:
                total += sum(a[i:j + 1])
    return total


# BASE QUESTION (optimal): Two running run totals
# Idea: For an increasing run of length L ending at i let
# S be the sum of all suffix sums of the run. When the
# next element x extends it, S becomes S + (L + 1) * x and
# L becomes L + 1; a break restarts at S = x, L = 1.
def good_subarray_sum(a):
    # a good subarray has all steps +1 or all steps -1.
    # For each index keep, per direction, the sum of the
    # sums of all good subarrays that END here.
    total = 0
    up_sum = up_len = down_sum = down_len = 0
    for i, x in enumerate(a):
        step = x - a[i - 1] if i else None
        if step == 1:
            # extend the +1 run: every old subarray grows
            # by
            # x, and [x] alone is new: (len + 1) * x in
            # all
            up_sum += (up_len + 1) * x
            up_len += 1
        else:
            up_sum, up_len = x, 1
```

```python
        if step == -1:
            down_sum += (down_len + 1) * x
            down_len += 1
        else:
            down_sum, down_len = x, 1
        # [x] is in both totals: count it once
        total += up_sum + down_sum - x
    return total


if __name__ == "__main__":
    arr = [3, 5, 6, 7, 6]
    # singletons 27 + [5,6] 11 + [6,7] 13 + [5,6,7] 18 +
    # [7,6] 13
    assert good_subarray_sum(arr) == 82
    assert good_subarray_sum([]) == 0
    assert good_subarray_sum([4]) == 4
```

```python
    # the O(n) answer matches the brute force everywhere
    import itertools
    for n in range(7):
        every = itertools.product([1, 2, 3, 4], repeat=n)
        for a in every:
            assert good_subarray_sum(list(a)) == (
                good_subarray_sum_brute(list(a)))
```

NOTES FROM CANDIDATES

Walk the array maintaining two run lengths: upRun (current trailing run of +1 steps) and downRun (current trailing run of -1 steps). At index i, only one of them is nonzero (the other resets to 1).
Each index i ends upRun + downRun - 1 good subarrays (subtracting the double-counted singleton). For each ending position, contribute the running prefix-sum tail of the active run.
Multiple candidates report getting stuck — verbalize the "contribution per index" framing out loud before coding.
Easy to mis-handle the reset boundary; double-check with a 2-element example before writing the loop.

PREPARATION

Practice the "contribution-per-index" pattern: the sum-of-subarray-minimums and sum-of-subsequence-widths problems both use the same technique.
Practice splitting an array into maximal monotone-step runs, then computing per-run contributions in O(1) amortized using prefix sums of element values.
Sanity-check with arr = [1, 2, 3] and arr = [3, 2, 1] by hand — both should sum to 20.

STUDY HERE: topic 2, Arrays, Hashing and Two Pointers

Site tags: data-structure, two-pointer, medium

MEDIUM | 1Point3Acres | phone-screen, onsite-coding | 24 of 38

Run-Length Vector Storage + Dot Product

ASKED 2026-01-21 | 272 DAYS AGO | OLDER

STUDY: TALK | ALL TIME 5 | 30 DAYS 0 | PRIORITY 173/228

ABOUT THIS QUESTION

Topic: Arrays, Hashing and Two Pointers

Optimal answer from: written and tested here for this exact question

LeetCode match: LC 1570 Dot Product of Two Sparse Vectors

Frequency: Single report | Last asked: 2026-01-21 | Roles: SWE, INFRA-ENG

Original: https://www.1point3acres.com/interview/problems/company/google/compressed-vector-dot-product

QUESTION

EXAMPLES

```
Logical A = [3, 3, 3, 0, 0, 0, 4, 4]
Logical B = [1, 1, 1, 1, 2, 2, 2, 2]
Encoded A (RLE): [(3,3),(0,3),(4,2)]
Encoded B (RLE): [(1,4),(2,4)]
Dot product = 3·1·3 + 3·1·0 + 0·1·0 + 0·2·0 + 4·2·2 =
    9 + 0 + 16 = 25
```

APPROACH

Memorize the two-pointer RLE walk pattern (it generalizes to interval intersection, range addition, etc.).
Practice articulating when RLE wins vs sparse (O(distinct_runs) vs O(nonzeros)).
Have a one-liner for sparse case: store (idx -> value) map; iterate over the smaller map and look up the other.

INTERVIEW-DAY PLAN

CLARIFY FIRST

Ask for the encoding format (value, count pairs), whether runs of zeros are stored, the logical length of both vectors (must be equal), and whether updates are needed.

BRUTE FORCE TO SAY OUT LOUD

Expand both vectors and multiply element by element. O(n) memory.

OPTIMAL IDEA

Two pointers over the run lists. Take the overlap, which is min of the two remaining counts, add value_a * value_b * overlap, subtract the overlap from both, and advance whichever run is exhausted. O(r1 + r2), O(1) extra space.

DRY-RUN THESE CASES

The example giving 25. One vector of a single run. Runs that end at the same time. All zeros.

LIKELY FOLLOW-UPS

INTERVIEW-DAY PLAN

A sparse vector against an RLE vector, setting an element, or dot product of k vectors.

PITFALLS

Advancing only one pointer when both runs finish together. Overflowing the product for large counts.

HOW TO THINK ABOUT IT

A run is a stretch where both numbers are constant, so the dot product of two runs is value times value times the length of their overlap. Walk the two run lists like the merge step of merge sort: consume the smaller remaining length from both, and advance whichever run you finished.

Basis: LeetCode 1570, Dot Product of Two Sparse Vectors, with run-length encoding as the compression.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def decode(runs):
    dense = []
    for value, count in runs:
        dense += [value] * count
    return dense
def brute(runs_a, runs_b):
    a, b = decode(runs_a), decode(runs_b)
    return sum(x * y for x, y in zip(a, b))
```

TIME	SPACE
O(logical length)	O(logical length)

WHY IT IS SLOW

It needs memory and time proportional to the logical length, which can be 10^9. Walking the two run lists together with two pointers costs only the number of runs.

OPTIMAL SOLUTION · Two-pointer over runs

TIME	SPACE
O(r1 + r2)	O(1)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 What if one vector is mostly zeros and stored sparsely, and the other as runs?

Iterate the sparse side's nonzero indices and binary-search the run list for each index to read the other value: O(s log r). Choose the smaller representation to drive the loop.

FOLLOW-UP 2 How should the data be updated if you set element i?

Locate the run containing i (binary search over prefix run lengths), then split it into at most three runs and merge equal neighbours. That is O(log r) to find plus O(r) in the worst case to splice a list; use a balanced tree of runs for frequent updates.

FOLLOW-UP 3 One vector is a sparse map (index to value) and the other is run-length encoded.

Iterate the smaller structure (the sparse non-zeros) and binary search the run list for each index using prefix run lengths: O(s log r) instead of expanding anything.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Run-Length Vector Storage + Dot Product
#
# Examples: Logical A = [3, 3, 3, 0, 0, 0, 4, 4] Logical B
# = [1, 1, 1, 1, 2, 2, 2, 2] Encoded A (RLE):
# [(3,3),(0,3),(4,2)] Encoded B (RLE): [(1,4),(2,4)] Dot
# product = 3·1·3 + 3·1·0 + 0·1·0 + 0·2·0 + 4·2·2 = 9 + 0
# + 16 = 25
#
# FOLLOW-UPS (answered below, in this order):
# 1. What if one vector is mostly zeros and stored
# sparsely, and the other as runs?
# 2. How should the data be updated if you set element i?
# 3. One vector is a sparse map (index to value) and the
# other is run-length encoded.
#
# INTERVIEW NOTES
# A run is a stretch where both numbers are constant, so
# the dot product of two runs is value times value times
# the length of their overlap. Walk the two run lists like
# the merge step of merge sort: consume the smaller
# remaining length from both, and advance whichever run
# you finished.
#
# BRUTE FORCE (say it first): Expand both vectors and
# multiply
# Decompress each run into repeated values and add up the
# element-wise products. Correct and the oracle; with
# logical lengths like 10^9 it cannot even be stored. Time
# O(logical length). Space O(logical length).


from bisect import bisect_right
from itertools import accumulate


# BRUTE FORCE (say it first): Expand both vectors and
# multiply | time O(logical length)
def rle_dot_expand(a, b):
    x = [v for v, c in a for _ in range(c)]
    y = [v for v, c in b for _ in range(c)]
    return sum(p * q for p, q in zip(x, y))


# BASE QUESTION (optimal): Two-pointer over runs
# Idea: Each run is (value, count). Compare the remaining
# lengths of the two current runs, multiply the values by
# the overlap, then advance whichever run was exhausted
# (or both).
def rle_dot(a, b):
    # two pointers over the runs: never expand anything
    i = j = 0
    left_a = a[0][1] if a else 0
    left_b = b[0][1] if b else 0
    total = 0
    while i < len(a) and j < len(b):
        # both runs overlap for min(left_a, left_b)
        # positions
        take = min(left_a, left_b)
        total += a[i][0] * b[j][0] * take
        left_a -= take
        left_b -= take
        if left_a == 0:
            i += 1
            left_a = a[i][1] if i < len(a) else 0
        if left_b == 0:
            j += 1
            left_b = b[j][1] if j < len(b) else 0
    return total


# FOLLOW-UP 1: What if one vector is mostly zeros and
# stored sparsely, and the other as runs?
# Answer: Iterate the sparse side's nonzero indices and
# binary-search the run list for each index to read the
# other value: O(s log r). Choose the smaller
# representation to drive the loop.
# FOLLOW-UP 3: One vector is a sparse map (index to value)
# and the other is run-length encoded.
# Answer: Iterate the smaller structure (the sparse non-
# zeros) and binary search the run list for each index
# using prefix run lengths: O(s log r) instead of
# expanding anything.
def sparse_dot_rle(sparse, runs):
    # sparse: {index: value}; runs: RLE of the other
    # vector
    ends = list(accumulate(count for _, count in runs))
    total = 0
    # iterate the small side; binary search the runs
    for index, value in sparse.items():
        k = bisect_right(ends, index)
        if k < len(runs):
            total += value * runs[k][0]
    return total


# FOLLOW-UP 2: How should the data be updated if you set
# element i?
# Answer: Locate the run containing i (binary search over
# prefix run lengths), then split it into at most three
# runs and merge equal neighbours.
def set_at(runs, index, value):
    # update one position: split a run, then merge
    # neighbours
    flat, pos = [], 0
    for v, count in runs:
        if pos <= index < pos + count:
            flat += [(v, index - pos), (value, 1),
                     (v, pos + count - index - 1)]
        else:
            flat.append((v, count))
        pos += count
    merged = []
    for v, count in flat:
        if count == 0:
            continue
        if merged and merged[-1][0] == v:
            merged[-1] = (v, merged[-1][1] + count)
        else:
```

```
            merged.append((v, count))
    return merged


# HELPERS for the checks below
def rle_encode(values):
    runs = []
    for v in values:
        if runs and runs[-1][0] == v:
            runs[-1][1] += 1
        else:
            runs.append([v, 1])
    return [tuple(r) for r in runs]


def rle_decode(runs):
    return [v for v, count in runs for _ in range(count)]
```

```
if __name__ == "__main__":
    A = [3, 3, 3, 0, 0, 0, 4, 4]
    B = [1, 1, 1, 1, 2, 2, 2, 2]
    ra, rb = rle_encode(A), rle_encode(B)
    assert ra == [(3, 3), (0, 3), (4, 2)]
    assert rle_decode(ra) == A
    # 3*1*3 + 0 + 4*2*2 = 25
    assert rle_dot(ra, rb) == 25
    assert rle_dot([], rb) == 0
    dense = sum(x * y for x, y in zip(A, B))
    assert dense == 25
    # sparse vector with non-zeros at 0 and 7
    sparse = {0: 2, 7: 5}
    assert sparse_dot_rle(sparse, rb) == 2 * 1 + 5 * 2
    # setting a value splits and merges runs
    assert set_at(ra, 4, 3) == [(3, 3), (0, 1), (3, 1),
                                (0, 1), (4, 2)]
    assert set_at([(1, 3)], 1, 1) == [(1, 3)]
```

NOTES FROM CANDIDATES

Two-pointer walk over the two RLE encodings: at each step compute the overlap length min(remainA, remainB), multiply values, advance the pointer whose run is exhausted.
Mention compression-friendly variations: sparse format wins when most entries are 0 (just enumerate non-zero indices and skip zeros entirely in the dot product).
Interviewer cared more about clean data-structure choice and the walk over both encodings than about absolute optimality.

PREPARATION

Memorize the two-pointer RLE walk pattern (it generalizes to interval intersection, range addition, etc.).
Practice articulating when RLE wins vs sparse (O(distinct_runs) vs O(nonzeros)).
Have a one-liner for sparse case: store (idx -> value) map; iterate over the smaller map and look up the other.

STUDY HERE: topic 2, Arrays, Hashing and Two Pointers

MEDIUM | PracHub | Technical Screen | 25 of 38

Reconstruct original array from doubled shuffle

POSTED 2026-01-04 | 289 DAYS AGO | OLDER

STUDY: TALK | ALL TIME 29.8 | 30 DAYS 0 | PRIORITY 127/228

ABOUT THIS QUESTION

Topic: Arrays, Hashing and Two Pointers

Optimal answer from: the interview site's own solution

LeetCode match: LC 2007 Find Original Array From Doubled Array (inferred from overview)

Posted on PracHub: 2026-01-04 (updated 2026-05-22)

Original: https://prachub.com/coding-questions/reconstruct-original-array-from-doubled-shuffle

QUESTION

Overview: This question evaluates array manipulation, multiset/frequency reasoning, pairing logic, and careful handling of edge cases such as zeros and negative numbers.

APPROACH

Sort (by absolute value) and use a counter: for each smallest x with count > 0, pair with 2x; handle zeros (need even count) and negatives. O(n log n).

INTERVIEW-DAY PLAN

CLARIFY FIRST

Ask whether the output may be in any order, whether negatives and zeros are possible, and what to return if it is impossible (empty array).

BRUTE FORCE TO SAY OUT LOUD

Try every pairing recursively. Exponential.

OPTIMAL IDEA

If the length is odd return empty. Count values. Process numbers in order of absolute value; for each value x with remaining count, match it with 2x, decrement both counts, and fail if 2x has no count left. Zeros need an even count. Sorting by abs makes the smallest value always the original, never a double. O(n log n).

DRY-RUN THESE CASES

[1,3,4,2,6,8] gives [1,3,4]. An odd length. [0,0]. [0,0,0] fails. Negatives such as [-2,-4,1,2].

LIKELY FOLLOW-UPS

Linear time when the value range is small (counting array), or unsorted output.

PITFALLS

Sorting by value instead of absolute value for negatives. Mishandling zeros.

HOW TO THINK ABOUT IT

The element with the smallest absolute value cannot be the double of anything, so it is an original; its double must be present too. Pair them off, take the next smallest, and so on. Process values in order of absolute value so negatives and zeros follow the same rule, and handle zero separately (it is its own double, so it needs an even count).

Basis: LeetCode 2007, Find Original Array From Doubled Array.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute(changed):
    items = sorted(changed, key=abs)
    originals = []
    while items:
        x = items.pop(0)
        if 2 * x not in items:
            return []
        items.remove(2 * x)
        originals.append(x)
    return originals
```

TIME	SPACE
O(n^2)	O(n)

WHY IT IS SLOW

list.remove is a linear search, so each step is O(n) and the loop is O(n^2). A Counter processed in order of absolute value is O(n log n).

OPTIMAL SOLUTION · Counter processed in order of absolute value

TIME	SPACE
O(n log n)	O(n)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Return the originals in the order they appear in the input.

Compute the multiset of originals, then scan the input once and keep each element while its remaining original count is positive.

FOLLOW-UP 2 Values are small non-negative integers.

Use a counting array instead of sorting and a Counter: process values from small to large, O(n + range).

FOLLOW-UP 3 Why does processing by absolute value work with negatives?

For negatives the double has a larger absolute value, so the element with the smallest absolute value is always the original of its pair, exactly as with positives.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Reconstruct original array from doubled
# shuffle
#
# This question evaluates array manipulation,
# multiset/frequency reasoning, pairing logic, and careful
# handling of edge cases such as zeros and negative
# numbers.
#
# FOLLOW-UPS (answered below, in this order):
# 1. Return the originals in the order they appear in the
# input.
# 2. Values are small non-negative integers.
# 3. Why does processing by absolute value work with
# negatives?
#
# INTERVIEW NOTES
# The element with the smallest absolute value cannot be
# the double of anything, so it is an original; its double
# must be present too. Pair them off, take the next
# smallest, and so on. Process values in order of absolute
# value so negatives and zeros follow the same rule, and
# handle zero separately (it is its own double, so it
# needs an even count).
#
# BRUTE FORCE (say it first): Try to remove each smallest
# element together with its double from a list
# Sort by absolute value and, for each element still in
# the list, remove it and one copy of its double
# (list.remove is a linear search). Correct and short; the
# repeated linear searches make it quadratic. Time O(n^2).
# Space O(n).


from collections import Counter


# BRUTE FORCE (say it first): Try to remove each smallest
# element together with its double from a list | time
# O(n^2)
def original_brute(changed):
    if len(changed) % 2:
        return []
    rest = sorted(changed, key=lambda x: (abs(x), x))
    out = []
    while rest:
        x = rest.pop(0)
        if 2 * x not in rest:
            return []
        rest.remove(2 * x)
        out.append(x)
    return out


# BASE QUESTION (optimal): Counter processed in order of
# absolute value
# Idea: Count the values. For each value x in order of
# absolute value with a remaining count, it needs count[x]
# copies of 2x: if count[2x] is smaller, fail; otherwise
# consume them and append x that many times.
def find_original(changed):
    # `changed` holds every original x and its double 2x,
    # in
    # any order. Smallest absolute value first: it can
    # only be
    # an original, never someone's double.
    if len(changed) % 2:
        return []
    count, out = Counter(changed), []
    for x in sorted(count, key=lambda v: (abs(v), v)):
        if count[x] == 0:
            continue
        if x == 0:
            # zeros pair with each other
            if count[0] % 2:
                return []
            out += [0] * (count[0] // 2)
            count[0] = 0
            continue
        # every x needs its own 2x
        if count[2 * x] < count[x]:
            return []
        out += [x] * count[x]
        count[2 * x] -= count[x]
        count[x] = 0
    return out


# FOLLOW-UP 1: Return the originals in the order they
# appear in the input.
# Answer: Compute the multiset of originals, then scan the
# input once and keep each element while its remaining
# original count is positive.
def find_original_in_order(changed):
    # originals in the order they appear in the input
    originals = find_original(changed)
    if not originals and changed:
        return []
    left = Counter(originals)
    out = []
    for x in changed:
        if left[x] > 0:
            left[x] -= 1
            out.append(x)
    return out


# FOLLOW-UP 2: Values are small non-negative integers.
# Answer: Use a counting array instead of sorting and a
# Counter: process values from small to large, O(n +
# range).
def find_original_small(changed, limit):
    # values 0..limit: a counting array, O(n + limit)
    if len(changed) % 2:
        return []
    count = [0] * (2 * limit + 2)
    for x in changed:
        count[x] += 1
    out = []
    if count[0] % 2:
        return []
    out += [0] * (count[0] // 2)
    for x in range(1, limit + 1):
        if count[x]:
            if count[2 * x] < count[x]:
                return []
            out += [x] * count[x]
            count[2 * x] -= count[x]
    return out
```

```
if __name__ == "__main__":
    got = sorted(find_original([1, 3, 4, 2, 6, 8]))
    assert got == [1, 3, 4]
    assert find_original([6, 3, 0, 1]) == []
    assert find_original([1]) == []
```

```
    assert find_original([0, 0, 0, 0]) == [0, 0]
    assert find_original([0, 0, 0]) == []
    # negative numbers: -2 is the double of -1
    neg = sorted(find_original([-1, -2, 4, 2]))
    assert neg == [-1, 2]
    ordered = find_original_in_order([4, 1, 2, 8, 2, 4])
    assert ordered == [4, 1, 2]
    small = sorted(find_original_small([1, 2, 4, 8], 8))
    assert small == [1, 4]
    assert find_original_small([1, 3], 3) == []
```

FUNCTION AND TEST CASES

Signature

```
def solution(changed: list[int]) -> list[int]:
```

Sample call

```
solution([2, 1, 4, 8])
```

Test cases

Input	Expected
`([2, 1, 4, 8],)`	`[1, 4]`
`([-4, -2, -2, -1, 1, 2],)`	`[-2, -1, 1]`

STUDY HERE: topic 2, Arrays, Hashing and Two Pointers

MEDIUM | PracHub | Technical Screen | 26 of 38

Maximize 1D deliveries within distance and minimize total distance

POSTED 2025-12-27 | 297 DAYS AGO | OLDER

STUDY: TALK | ALL TIME 36.9 | 30 DAYS 0 | PRIORITY 101/228

ABOUT THIS QUESTION

Topic: Arrays, Hashing and Two Pointers

Optimal answer from: the interview site's own solution

Related in your app (a neighbouring problem, not this exact question): LeetCode #455 Assign Cookies, LeetCode #475 Heaters

LeetCode match: No direct match; related LC 455 Assign Cookies / LC 475 Heaters (inferred from overview)

Posted on PracHub: 2025-12-27 (updated 2026-04-23)

Original: https://prachub.com/coding-questions/maximize-1d-deliveries-within-distance-and-minimize-total-distance

QUESTION

Overview: This question evaluates combinatorial optimization and constrained matching skills, focusing on reasoning about distance-based pairings, lexicographic objective prioritization, and edge-case handling such as duplicates and empty inputs.

APPROACH

Sort both lists; two-pointer greedy matching each order to the nearest available driver within the limit, maximizing count first, then minimizing total distance as the tie-break.

INTERVIEW-DAY PLAN

CLARIFY FIRST

Ask what an order and a driver are (points on a line), the distance limit, whether each driver takes one order, and the objective order: first the maximum number of deliveries, then the smallest total distance among those.

BRUTE FORCE TO SAY OUT LOUD

Try every matching of orders to drivers. Exponential.

OPTIMAL IDEA

Sort both lists. On a line there is an optimal matching that does not cross, so a DP over (i, j), the first i orders and first j drivers, works: dp[i][j] is the best pair (maximum matches, minimum distance), taking the maximum of skipping an order, skipping a driver, or matching order i with driver j if their distance is within the limit. O(n * m), reducible to O(n * window) because only drivers within the limit matter. A two-pointer greedy that matches the nearest available driver gives the maximum count in O(n log n) but not necessarily the minimum total distance, so say that.

DRY-RUN THESE CASES

INTERVIEW-DAY PLAN

No feasible pairs, duplicates, empty inputs, an order that could match two drivers equally, more drivers than orders.

LIKELY FOLLOW-UPS

Minimizing the maximum distance instead, or drivers with capacities.

PITFALLS

Assuming greedy nearest matching minimizes the total. Comparing the two objectives as a sum instead of lexicographically.

HOW TO THINK ABOUT IT

On a line there is always an optimal matching that does not cross itself, so sort both lists and match in order: a DP over (orders considered, drivers considered) with the pair (matches, total distance) compared lexicographically solves both objectives together. A greedy that matches each order to the nearest free driver gets the count right but not always the smallest total distance.

Basis: Written; matching on a line. Assumed rules (confirm them): orders and drivers are positions on a line; an order can be served by a driver within distance D; each driver takes at most one order; first maximise the number of deliveries, then minimise the total distance.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
best = (0, 0)  # (matches, -distance)
def go(i, used, matches, dist):
    global best
    if i == len(orders):
        best = max(best, (matches, -dist))
        return
    go(i + 1, used, matches, dist)  # skip
    for d, x in enumerate(drivers):
        gap = abs(orders[i] - x)
        if d not in used and gap <= D:
            go(i + 1, used | {d},
               matches + 1, dist + gap)
```

TIME	SPACE
O(drivers^orders)	O(n)

WHY IT IS SLOW

Each order branches over all drivers: exponential. After sorting, a DP over (orders considered, drivers considered) that compares (matches, minus distance) is O(n * m).

OPTIMAL SOLUTION · DP over sorted orders and drivers, maximising (matches, -distance)

TIME	SPACE
O(n * m)	O(n * m)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Only the number of deliveries matters.

Two pointers on the sorted lists: match when within D, otherwise advance whichever is further left. O(n log n). It maximises the count (checked against exhaustive search) but not the total distance.

FOLLOW-UP 2 A driver can take several orders (capacity c).

Replicate each driver c times (as separate sorted points) and use the same DP, or treat it as min-cost flow with capacities.

FOLLOW-UP 3 Minimise the largest single distance instead of the total.

Binary search the largest allowed distance D and test whether all orders can be served with the two-pointer matching; the check is monotone in D. **Cost:** O(n log n + (n + m) log(range)): sorting plus a binary search whose check is a two-pointer pass.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Maximize 1D deliveries within distance
# and minimize total distance
#
# This question evaluates combinatorial optimization and
# constrained matching skills, focusing on reasoning about
# distance-based pairings, lexicographic objective
# prioritization, and edge-case handling such as
# duplicates and empty inputs.
#
# FOLLOW-UPS (answered below, in this order):
# 1. Only the number of deliveries matters.
# 2. A driver can take several orders (capacity c).
# 3. Minimise the largest single distance instead of the
# total.
#
# INTERVIEW NOTES
# On a line there is always an optimal matching that does
# not cross itself, so sort both lists and match in order:
# a DP over (orders considered, drivers considered) with
# the pair (matches, total distance) compared
# lexicographically solves both objectives together. A
# greedy that matches each order to the nearest free
# driver gets the count right but not always the smallest
# total distance.
#
# BRUTE FORCE (say it first): Try every set of matches
# recursively
# For each order either skip it or match it to any unused
# driver within D; keep the best (most matches, then
# smallest total distance). Exact and the oracle;
# exponential. Time O(drivers^orders). Space O(n).


# BRUTE FORCE (say it first): Try every set of matches
# recursively | time O(drivers^orders)
def deliveries_brute(orders, drivers, D):
    best = [(0, 0)]

    def go(i, used, count, dist):
        if i == len(orders):
            if (count, -dist) > (
                best[0][0],
                -best[0][1],
            ):
                best[0] = (count, dist)
            return
        go(i + 1, used, count, dist)
        for j, d in enumerate(drivers):
            if (
                j not in used
                and abs(orders[i] - d) <= D
            ):
                go(
                    i + 1,
                    used | {j},
                    count + 1,
                    dist + abs(orders[i] - d),
                )

    go(0, frozenset(), 0, 0)
    return best[0]


# BASE QUESTION (optimal): DP over sorted orders and
# drivers, maximising (matches, -distance)
# Idea: Sort both lists. dp[i][j] is the best (matches,
# -total distance) using the first i orders and the first
# j drivers: skip order i, skip driver j, or match them if
# their distance is at most D.
def deliveries(orders, drivers, D):
    # first maximise the number of deliveries, then
    # minimise
    # the total distance: a DP over the sorted positions
    o, d = sorted(orders), sorted(drivers)
    # dp[i][j] = (deliveries, -distance) using the first i
    # orders and j drivers; tuples compare
    # lexicographically
    dp = [[(0, 0)] * (len(d) + 1)
          for _ in range(len(o) + 1)]
    for i in range(1, len(o) + 1):
        for j in range(1, len(d) + 1):
            # skip this order or skip this driver ...
            best = max(dp[i - 1][j], dp[i][j - 1])
            gap = abs(o[i - 1] - d[j - 1])
            if gap <= D:
                # ... or pair them
                count, neg = dp[i - 1][j - 1]
                best = max(best, (count + 1, neg - gap))
            dp[i][j] = best
    count, neg = dp[-1][-1]
    return count, -neg


# FOLLOW-UP 1: Only the number of deliveries matters.
# Answer: Two pointers on the sorted lists: match when
# within D, otherwise advance whichever is further left.
# O(n log n). It maximises the count (checked against
# exhaustive search) but not the total distance.
def max_count(orders, drivers, D):
    # only the number matters: two pointers on sorted
    # lists
    o, d = sorted(orders), sorted(drivers)
    i = j = count = 0
    while i < len(o) and j < len(d):
        if abs(o[i] - d[j]) <= D:
            count, i, j = count + 1, i + 1, j + 1
        elif o[i] < d[j]:
            # this order is too far left for every driver
            i += 1
        else:
```

```
            j += 1
    return count


# FOLLOW-UP 2: A driver can take several orders (capacity
# c).
# Answer: Replicate each driver c times (as separate
# sorted points) and use the same DP, or treat it as min-
# cost flow with capacities.
def with_capacity(orders, drivers, D, cap):
    # each driver can take cap orders: repeat the driver
    return deliveries(orders, [x for x in drivers
                               for _ in range(cap)], D)


# FOLLOW-UP 3: Minimise the largest single distance
# instead of the total.
# Answer: Binary search the largest allowed distance D and
# test whether all orders can be served with the two-
# pointer matching; the check is monotone in D.
def smallest_limit(orders, drivers):
    # minimise the LARGEST single distance while serving
    # as
    # many orders as possible: binary search on the limit
    target = min(len(orders), len(drivers))
```

```
    lo, hi = 0, 10 ** 9
    while lo < hi:
        mid = (lo + hi) // 2
        if max_count(orders, drivers, mid) >= target:
            hi = mid
        else:
            lo = mid + 1
    return lo


if __name__ == "__main__":
    assert deliveries([1, 5], [2, 6], 1) == (2, 2)
    # pairing 1-2 and 3-4 beats 1-4 and 3-2? both count 2
    assert deliveries([1, 3], [2, 4], 1) == (2, 2)
    assert deliveries([], [1], 5) == (0, 0)
    # a distance limit of 0 needs identical positions
    assert deliveries([1, 2], [2, 3], 0) == (1, 0)
    assert max_count([1, 3], [2, 4], 1) == 2
    assert max_count([1, 10], [5], 2) == 0
    assert smallest_limit([1, 5], [2, 9]) == 4
    assert with_capacity([1, 2, 3], [2], 1, 3) == (3, 2)
```

FUNCTION AND TEST CASES

Signature

```
def solution(people: list[int], shops: list[int], D: int)
-> tuple[int, int]:
```

Sample call

```
solution([1, 7, 10], [2, 6, 12], 2)
```

Test cases

Input	Expected
([1, 7, 10], [2, 6, 12], 2)	(3, 4)
([0, 4], [2, 5], 3)	(2, 3)

STUDY HERE: topic 2, Arrays, Hashing and Two Pointers

Site tags: hashmap, two-pointer, streaming, medium

MEDIUM | 1Point3Acres | onsite-coding | 27 of 38

Two-Day Log Intersection under Memory Limit

ASKED 2025-11-27 | 327 DAYS AGO | OLDER

STUDY: TYPE | ALL TIME 50.4 | 30 DAYS 46.9 | PRIORITY 24/228

ABOUT THIS QUESTION

Topic: Arrays, Hashing and Two Pointers

Optimal answer from: written and tested here for this exact question

LeetCode match: LC 349 Intersection of Two Arrays (related)

Frequency: Single report | Last asked: 2025-11-27 | Roles: SWE, INFRA-ENG

Original: https://www.1point3acres.com/interview/problems/company/google/two-logs-intersection-memory-limit

QUESTION

PhD intern coding round 1: two log files with **`(timestamp, obj_id, client_id)`** entries. Find **`obj_id`**s that (a) appear in both files and (b) have at least two distinct **`client_id`**s. Follow-up imposes a memory limit, expecting external sort + two-pointer.

Input: two log files; each line is (timestamp, obj_id, client_id). Find every obj_id such that:

(a) appears in both files, and
(b) is associated with at least 2 distinct client_ids (aggregated across both files).

Follow-up 1: complexity of set-intersection on the file contents.
Follow-up 2: memory cannot hold either file fully — design a streaming solution.

EXAMPLES

```
File A:  (t1, X, c1), (t2, X, c2), (t3, Y, c1)
File B:  (t4, X, c1), (t5, Z, c2)
-> Only X is in both files; X has 2 distinct clients
    {c1, c2}.
Output: [X]
```

APPROACH

Practice external-sort / merge-step problems (classic in-memory merge / k-way-merge / partition problems — and discuss disk-based variants verbally).
Drill the streaming hashmap vs external sort trade-off conversation.
Have ready: HyperLogLog (or Count-Min Sketch) one-liner for approximate distinct-count under tight memory.

HOW TO THINK ABOUT IT

In memory it is a hash map from object to the set of clients, filtered to objects that appear in both files and have at least two clients. When the files do not fit in memory, sort both by object id (external sort) and walk them together: all rows of one object are adjacent, so you only ever hold the clients of one object at a time.

Basis: LeetCode 349, Intersection of Two Arrays, with a distinct-client condition and a limit on memory (external sort and a two-pointer merge).

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute(first, second):
    result = []
    for obj in {r.obj for r in first}:
        if obj not in {r.obj for r in second}:
            continue
        rows = [r for r in first + second
                if r.obj == obj]
        if len({r.client for r in rows}) >= 2:
            result.append(obj)
    return result
```

TIME	SPACE
O(n * m)	O(n + m)

WHY IT IS SLOW

O(n * m) time and both files must fit in memory. Sorting both files by object and merging them with two pointers is O(n log n) and holds one object's clients at a time.

OPTIMAL SOLUTION · Hash map of client sets (everything fits in memory)

TIME	SPACE
O(n + m)	O(n + m)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Why prefer sort and two pointers to a streaming hash map?

A hash map needs memory proportional to the number of distinct objects (and clients), which is exactly what the limit forbids. An external sort needs only a chunk of memory and sequential disk passes, and the merge needs memory for one object's clients. If one object has an enormous client set, count distinct clients approximately (HyperLogLog) or stop as soon as two are seen.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Two-Day Log Intersection under Memory
# Limit
#
# PhD intern coding round 1: two log files with
# (timestamp, obj_id, client_id) entries. Find obj_ids
# that (a) appear in both files and (b) have at least two
# distinct client_ids. Follow-up imposes a memory limit,
# expecting external sort + two-pointer. Input: two log
# files; each line is (timestamp, obj_id, client_id). Find
# every obj_id such that: (a) appears in both files, and
# (b) is associated with at least 2 distinct client_ids
# (aggregated across both files). Follow-up 1: complexity
# of set-intersection on the file contents. Follow-up 2:
# memory cannot hold either file fully — design a
# streaming solution. Examples: File A: (t1, X, c1), (t2,
# X, c2), (t3, Y, c1) File B: (t4, X, c1), (t5, Z, c2) ->
# Only X is in both files; X has 2 distinct clients {c1,
# c2}. Output: [X]
#
# FOLLOW-UPS (answered below, in this order):
# 1. Why prefer sort and two pointers to a streaming hash
# map?
#
# INTERVIEW NOTES
# In memory it is a hash map from object to the set of
# clients, filtered to objects that appear in both files
# and have at least two clients. When the files do not fit
# in memory, sort both by object id (external sort) and
# walk them together: all rows of one object are adjacent,
# so you only ever hold the clients of one object at a
# time.
#
# BRUTE FORCE (say it first): For every object scan the
# other file
# For each object in the first file scan the second file
# and the first file again to count clients. Correct,
# quadratic, and it needs both files in memory. Time O(n *
# m). Space O(n + m).


from collections import defaultdict


# BRUTE FORCE (say it first): For every object scan the
# other file | time O(n * m)
```

```
def log_objects_brute(a, b):
    out = set()
    for obj in {o for _, o, _ in a}:
        if (
            any(o == obj for _, o, _ in b)
            and len(
                {c for _, o, c in a + b if o == obj}
            )
            >= 2
        ):
            out.add(obj)
    return sorted(out)


# HELPERS for follow-up 1
def groups(rows):
    # rows sorted by object: yield (object, its clients)
    current, clients = None, set()
    for _, obj, client in rows:
        if obj != current:
            if current is not None:
                yield current, clients
            current, clients = obj, set()
        clients.add(client)
    if current is not None:
        yield current, clients


# BASE QUESTION (optimal): Hash map of client sets
# (everything fits in memory)
# Idea: Build object -> set of clients for each file, keep
# the objects that appear in both, and count the union of
# their client sets.
def log_objects(day1, day2):
    # everything fits in memory: object -> set of clients
    c1, c2 = defaultdict(set), defaultdict(set)
    for _, obj, client in day1:
        c1[obj].add(client)
    for _, obj, client in day2:
        c2[obj].add(client)
    # in both files, with 2+ clients over both days
    both = c1.keys() & c2.keys()
    return sorted(o for o in both
                  if len(c1[o] | c2[o]) >= 2)
```

```
# FOLLOW-UP 1: Why prefer sort and two pointers to a
# streaming hash map?
# Answer: A hash map needs memory proportional to the
# number of distinct objects (and clients), which is
# exactly what the limit forbids.
def log_objects_sorted(rows1, rows2):
    # memory limit: both files arrive sorted by object (an
    # external sort), so only one object's clients are
    # held
    out = []
    g1, g2 = groups(rows1), groups(rows2)
    a, b = next(g1, None), next(g2, None)
    while a and b:
        if a[0] == b[0]:
            # same object in both files
            if len(a[1] | b[1]) >= 2:
                out.append(a[0])
            a, b = next(g1, None), next(g2, None)
        elif a[0] < b[0]:
            a = next(g1, None)
        else:
            b = next(g2, None)
    return out


if __name__ == "__main__":
    day1 = [(1, "X", "c1"), (2, "X", "c2"),
            (3, "Y", "c1")]
    day2 = [(4, "X", "c1"), (5, "Z", "c2")]
    assert log_objects(day1, day2) == ["X"]
    def by_object(rows):
        return sorted(rows, key=lambda r: r[1])

    assert log_objects_sorted(by_object(day1),
                              by_object(day2)) == ["X"]
    # one client in total is not enough
    one = [(1, "A", "c1")]
    assert log_objects(one, one) == []
    assert log_objects_sorted(one, one) == []
    # clients may be split across the two days
    d1, d2 = [(1, "A", "c1")], [(2, "A", "c2")]
    assert log_objects(d1, d2) == ["A"]
    assert log_objects_sorted(d1, d2) == ["A"]
    assert log_objects([], day2) == []
```

NOTES FROM CANDIDATES

In-memory solution: build a hashmap obj_id -> set(client_id) while scanning each file; intersect keys across the two hashmaps at the end.
Memory-limited solution: external sort each file by obj_id, then run a merge / two-pointer scan over both sorted files. For each matching obj_id group, count distinct client_id on the fly (HyperLogLog if duplicates are large).
The interviewer explicitly preferred sort + two-pointer over a streaming hashmap — be prepared to defend the choice (sort is O(n log n) on disk vs O(n) RAM-bound for the hashmap).
For distributed scale, hash-partition by obj_id and run the same algorithm per partition.

PREPARATION

Practice external-sort / merge-step problems (classic in-memory merge / k-way-merge / partition problems — and discuss disk-based variants verbally).
Drill the streaming hashmap vs external sort trade-off conversation.
Have ready: HyperLogLog (or Count-Min Sketch) one-liner for approximate distinct-count under tight memory.

STUDY HERE: topic 2, Arrays, Hashing and Two Pointers

MEDIUM | IN APP LC 2 | PracHub | Technical Screen | 28 of 38

★ TOP 75: rank 1 in the Medium and Hard priority list

★ Add two big integers from digit lists

POSTED 2025-11-24 | 330 DAYS AGO | OLDER

STUDY: TYPE ALL TIME 78.7 30 DAYS 69.3 PRIORITY 1/228

ABOUT THIS QUESTION

Topic: Arrays, Hashing and Two Pointers

Optimal answer from: the interview site's own solution

Also in your app: LeetCode #2 Add Two Numbers

LeetCode match: LC 2 Add Two Numbers

Posted on PracHub: 2025-11-24 (updated 2026-06-26)

Original:
https://prachub.com/coding-questions/add-two-big-integers-from-digit-lists

QUESTION

You are given two non-empty singly linked lists (or arrays) representing two non-negative integers. The digits are stored in **reverse order**, and each node contains a single digit from 0 to 9. Your task is to add the two numbers and return the sum as a new linked list, also in reverse order. You may assume:

- Each list has at least one node.
- The integers do not contain leading zeros, except the number 0 itself.

Example

Input:

- List 1: 2 -> 4 -> 3 (represents the integer 342)
- List 2: 5 -> 6 -> 4 (represents the integer 465)

Output:

- 7 -> 0 -> 8 (represents the integer 807, since 342 + 465 = 807)

Another example

Input:

- List 1: 9 -> 9 -> 9 (represents 999)
- List 2: 1 (represents 1)

Output:

- 0 -> 0 -> 0 -> 1 (represents 1000)

Constraints

- Let n be the length of the first list and m be the length of the second list.
- 1 <= n, m <= 10^5.
- Each node value is an integer in the range [0, 9].

Task: Describe an algorithm to perform this addition and construct the resulting linked list in O(n + m) time and O(1) additional space (excluding the space for the output list).

QUESTION

You are given two non-empty arrays of digits representing two non-negative integers. The digits are stored in **reverse order** (least significant digit first), and each element is a single digit from 0 to 9. Add the two numbers and return the sum as a new array of digits, also in reverse order.

Assumptions

- Each array has at least one element.
- The integers do not contain leading zeros, except the number 0 itself.

Example 1

```
Input: l1 = [2, 4, 3], l2 = [5, 6, 4] (342 + 465)
Output: [7, 0, 8] (807)
```

Example 2

```
Input: l1 = [9, 9, 9], l2 = [1] (999 + 1)
Output: [0, 0, 0, 1] (1000)
```

Implement **`add_two_numbers(l1, l2)`** (camelCase **`addTwoNumbers`** in Java/C++/JS) returning the digit array of the sum in reverse order. Aim for O(n + m) time and O(1) extra space beyond the output.

Constraints

1 <= n, m <= 10^5 where n = len(l1), m = len(l2)Each element is an integer in the range [0, 9]Digits are stored in reverse order (least significant digit first)No leading zeros except for the number 0 itself

EXAMPLES

```
Input: ([2, 4, 3], [5, 6, 4])
Expected Output: [7, 0, 8]
Explanation: 342 + 465 = 807, stored reversed as [7,
    0, 8].
Input: ([9, 9, 9], [1])
Expected Output: [0, 0, 0, 1]
```

Explanation: 999 + 1 = 1000; carry propagates through all three digits and creates a new leading digit.

Hints

Walk both arrays from index 0 (the least significant digit) toward the end, just like grade-school column addition.Maintain a single carry variable. At each position, sum the available digits plus the carry; append (sum % 10) and set carry = sum // 10.When one array runs out, keep going with just the carry and the remaining longer array. Don't forget a final carry can add one extra digit (e.g. 999 + 1 -> [0,0,0,1]).

APPROACH

Walk both lists with a carry, build the result list. O(max(m, n)).

HOW TO THINK ABOUT IT

Addition by hand: start at the least significant end, add the two digits and the carry, write digit % 10, carry digit // 10. The only edge cases are different lengths and a final carry. Never convert to a machine integer in an interview about big numbers, since the point is that the numbers do not fit.

Basis: LeetCode 989 / 415 (Add to Array-Form of Integer, Add Strings). Assumed input: lists of decimal digits, most significant digit first.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def add_lists(a, b):
    x = digits_to_int(a)  # reverse-order digits
    y = digits_to_int(b)
    return int_to_digits(x + y)
```

TIME	SPACE
O(n^2) for the string-to-integer conversions	O(n)

WHY IT IS SLOW

It relies on arbitrary-size integers and the conversions are quadratic for very long inputs. Adding digit by digit from the right with a carry is O(max length).

OPTIMAL SOLUTION · Digit by digit with a carry

TIME	SPACE
O(max(m, n))	O(max(m, n))

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 The digits are stored least significant first in a linked list.

Then the head is already the low end, so walk both lists together with the same carry loop and build the result list as you go. No reversal needed (LeetCode 2).

FOLLOW-UP 2 Multiply instead of add.

Schoolbook multiplication: the product of digits a[i] and b[j] lands in position i + j + 1, with the carry spilling to i + j. O(m * n); Karatsuba or FFT for very long numbers.

FOLLOW-UP 3 Subtract, or numbers in another base.

Subtraction borrows instead of carrying and needs the larger number first (compare lengths, then digits). Another base replaces 10 with the base in % and //.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Add two big integers from digit lists
#
# You are given two non-empty singly linked lists (or
# arrays) representing two non-negative integers. The
# digits are stored in **reverse order**, and each node
# contains a single digit from 0 to 9. Your task is to add
# the two numbers and return the sum as a new linked list,
# also in reverse order. You may assume: - Each list has
# at least one node. - The integers do not contain leading
# zeros, except the number 0 itself. ### Example Input: -
# List 1: 2 -> 4 -> 3 (represents the integer 342) - List
2:
# 5 -> 6 -> 4 (represents the integer 465) Output: - 7 ->
0 ->
# 8 (represents the integer 807, since 342 + 465 = 807)
# ### Another example Input: - List 1: 9 -> 9 -> 9
# (represents 999) - List 2: 1 (represents 1) Output: - 0
# -> 0 -> 0 -> 1 (represents 1000) ### Constraints - Let n
 be
# the length of the first list and m be the length of the
# second list. - 1 <= n, m <= 10^5.
#
# FOLLOW-UPS (answered below, in this order):
# 1. The digits are stored least significant first in a
# linked list.
# 2. Multiply instead of add.
# 3. Subtract, or numbers in another base.
#
# INTERVIEW NOTES
# Addition by hand: start at the least significant end,
# add the two digits and the carry, write digit % 10,
# carry digit // 10. The only edge cases are different
# lengths and a final carry. Never convert to a machine
# integer in an interview about big numbers, since the
# point is that the numbers do not fit.
#
# BRUTE FORCE (say it first): Convert both lists to
# integers and add
# Join the digits, int() them, add, and split the result
# into digits. It is a valid shortcut in Python (big
# integers), but it hides the algorithm, depends on the
# language having big integers, and the conversions are
# quadratic for very long inputs. Time O(n^2) for the
# string-to-integer conversions. Space O(n).


# BRUTE FORCE (say it first): Convert both lists to
# integers and add | time O(n^2) for the string-to-integer
```

```
# conversions
def add_digits_int(a, b):
    return [
        int(ch)
        for ch in str(
            int("".join(map(str, a)))
            + int("".join(map(str, b)))
        )
    ]


# HELPERS for follow-up 1
class ListNode:
    def __init__(self, val=0, next=None):
        self.val, self.next = val, next
```

```
# BASE QUESTION (optimal): Digit by digit with a carry
# Idea: Walk both lists from the right with indices i and
# j; at each step add a[i] (or 0 once exhausted), b[j] (or
# 0) and the carry, append sum % 10, and carry = sum //
# 10.
def add_digits(a, b):
    # arrays with the most significant digit FIRST
    i, j, carry, out = len(a) - 1, len(b) - 1, 0, []
    while i >= 0 or j >= 0 or carry:
        total = carry
        total += a[i] if i >= 0 else 0
        total += b[j] if j >= 0 else 0
        out.append(total % 10)
        carry = total // 10
        i, j = i - 1, j - 1
    return out[::-1] or [0]


# FOLLOW-UP 1: The digits are stored least significant
# first in a linked list.
# Answer: Then the head is already the low end, so walk
# both lists together with the same carry loop and build
# the result list as you go. No reversal needed (LeetCode
# 2).
def add_lists(a, b):
    # digits least significant first: add with a carry,
    # one pass
    dummy = tail = ListNode()
```

```
    carry = 0
    while a or b or carry:
        total = carry
        total += a.val if a else 0
        total += b.val if b else 0
        tail.next = ListNode(total % 10)
        tail, carry = tail.next, total // 10
        a = a.next if a else None
        b = b.next if b else None
    return dummy.next


# FOLLOW-UP 2: Multiply instead of add.
# Answer: Schoolbook multiplication: the product of digits
# a[i] and b[j] lands in position i + j + 1, with the
# carry spilling to i + j. O(m * n); Karatsuba or FFT for
# very long numbers.
def multiply_digits(a, b):
    # schoolbook: digit a[i] * b[j] lands at position i +
    # j + 1
    res = [0] * (len(a) + len(b))
    for i in range(len(a) - 1, -1, -1):
        for j in range(len(b) - 1, -1, -1):
            total = a[i] * b[j] + res[i + j + 1]
            res[i + j + 1] = total % 10
            res[i + j] += total // 10
    while len(res) > 1 and res[0] == 0:
        res.pop(0)
    return res


# FOLLOW-UP 3: Subtract, or numbers in another base.
# Answer: Subtraction borrows instead of carrying and
# needs the larger number first (compare lengths, then
# digits). Another base replaces 10 with the base in % and
# //.
def subtract_digits(a, b):
    # a - b where a >= b (most significant first): borrow
    i, j, borrow, out = len(a) - 1, len(b) - 1, 0, []
    while i >= 0:
        d = a[i] - borrow - (b[j] if j >= 0 else 0)
        borrow = 1 if d < 0 else 0
        out.append(d + 10 * borrow)
        i, j = i - 1, j - 1
```

```
    while len(out) > 1 and out[-1] == 0:
```

```
        out.pop()
    return out[::-1]


# HELPERS for the checks below
def make(digits):
    # digits least significant first -> linked list
    head = tail = ListNode(digits[0])
    for d in digits[1:]:
        tail.next = ListNode(d)
        tail = tail.next
    return head


def to_digits(node):
    out = []
    while node:
        out.append(node.val)
        node = node.next
    return out


if __name__ == "__main__":
    # 342 + 465 = 807, digits reversed
    got = add_lists(make([2, 4, 3]), make([5, 6, 4]))
    assert to_digits(got) == [7, 0, 8]
    # a final carry adds a digit: 99 + 1 = 100
    carry = add_lists(make([9, 9]), make([1]))
    assert to_digits(carry) == [0, 0, 1]
    assert add_digits([9, 9, 9], [1]) == [1, 0, 0, 0]
    assert add_digits([0], [0]) == [0]
    product = multiply_digits([1, 2, 3], [4, 5])
    assert product == [5, 5, 3, 5]
    assert multiply_digits([0], [9, 9]) == [0]
    assert subtract_digits([1, 0, 0], [1]) == [9, 9]
    assert subtract_digits([5, 0], [5, 0]) == [0]
```

FUNCTION AND TEST CASES

Sample call

```
add_two_numbers([2, 4, 3], [5, 6, 4])
```

Test cases

Input	Expected
([2, 4, 3], [5, 6, 4])	[7, 0, 8]
([9, 9, 9], [1])	[0, 0, 0, 1]

STUDY HERE: topic 2, Arrays, Hashing and Two Pointers

MEDIUM | PracHub | Technical Screen | 29 of 38

Minimize calls to find all bad test pairs

POSTED 2025-11-05 | 349 DAYS AGO | OLDER

STUDY: TALK NO LEETCODE FREQUENCY PRIORITY 194/228

ABOUT THIS QUESTION

Topic: Arrays, Hashing and Two Pointers

Optimal answer from: the interview site's own solution

LeetCode match: No direct match (adaptive group testing)

Posted on PracHub: 2025-11-05 (updated 2026-06-18)

Original: https://prachub.com/coding-questions/minimize-calls-to-find-all-bad-test-pairs

QUESTION

You have (n) atomic tests (t_1, t_2, \dots, t_n). You are given access to a black-box function:

```
bool runTest(set<Test> S)
```

which behaves as follows:

- **`runTest(S) == True`** if the subset (S) contains **no** incompatible (bad) pair of tests.
- **`runTest(S) == False`** if the subset (S) contains **at least one** incompatible (bad) pair of tests. There is some unknown set of unordered **bad pairs** ((t_i, t_j)), where (i\ne j). You do **not** know in advance which pairs are bad or how many bad pairs there are. Your goal is to discover **all** bad pairs while making as few calls to **`runTest`** as possible. You may adaptively choose each subset (S) based on the results of previous calls.

Tasks

1. Design an algorithm that, using only calls to **`runTest`**, finds **all** bad pairs ((t_i, t_j)).
2. Analyze the number of calls to **`runTest`** your algorithm requires, as a function of:

- (n): the total number of tests, and - (k): the total number of bad pairs.

3. Argue why your algorithm is correct (i.e., why it will eventually find every bad pair and not report any good pair as bad). You do not need to provide full production code, but describe the algorithm clearly enough that it could be implemented.

You have n atomic tests numbered 0 to n - 1. Some unknown unordered pairs of tests are incompatible, called bad pairs. A black-box oracle runTest(S) returns True if the subset S contains no bad pair, and False if S contains at least one bad pair. In this non-interactive version, you are given bad_pairs only so your code can simulate the oracle. Implement the following deterministic oracle-discovery strategy and return both the bad pairs it discovers and the number of oracle calls it makes. Strategy:

QUESTION

1. Process vertices from right to left: v = n - 1, n - 2, ..., 0.
2. Maintain all bad pairs already discovered among vertices greater than the current v.
3. For the suffix vertices v + 1 through n - 1, greedily partition them into color classes such that no discovered bad pair appears inside one color class. Since all suffix-suffix bad pairs have already been discovered, every color class is truly compatible internally.
4. For each color class C, find all bad partners of v inside C using recursive group testing:

- Call runTest({v} union C).
- If the oracle returns True, v has no bad pair with any test in C.
- If C has size 1 and the oracle returns False, that single pair is bad.
- Otherwise split C into two halves and recurse on both halves. Return all discovered bad pairs sorted lexicographically, where each pair is represented as [i, j] with i < j, along with the exact number of runTest calls made by this strategy.

Constraints
0 <= n <= 5000 <= len(bad_pairs) <= n * (n - 1) / 2Each bad pair contains two distinct test indices in the range [0, n - 1]bad_pairs may be given in any endpoint order

EXAMPLES

```
Input: (0, [])
Expected Output: ([], 0)
```

Explanation: There are no tests and therefore no oracle calls are needed.

```
Input: (4, [])
Expected Output: ([], 3)
```

Explanation: For v = 2, 1, and 0, the algorithm queries the compatible suffix once and finds no bad pairs. v = 3 has an empty suffix.

Hints

If a candidate set C is internally compatible, then runTest({v} union C) tells you whether v conflicts with at least one element of C.Process tests from right to left so that when handling v, all bad pairs inside the suffix v + 1 ... n - 1 are already known and can be used to build compatible groups.

APPROACH

Binary search to find a minimal failing prefix (its last element is in a bad pair), then binary search among earlier tests for its partner; remove and repeat. About O(p log n) calls for p pairs.

INTERVIEW-DAY PLAN

CLARIFY FIRST

Ask what runTest returns (true if the set contains no incompatible pair), whether a test can be in more than one bad pair, and whether you may call it on any subset.

BRUTE FORCE TO SAY OUT LOUD

Call runTest on every pair of tests. O(n^2) calls.

OPTIMAL IDEA

Find a minimal failing prefix by binary search: add tests one at a time conceptually and binary search for the smallest prefix that fails; the last element of that prefix is part of a bad pair. Then binary search over the earlier tests to find its partner. Record the pair, remove one of the two as needed, and repeat. About O(p log n) calls for p bad pairs.

DRY-RUN THESE CASES

No bad pairs (one call). One bad pair. A test that conflicts with many others. Two disjoint bad pairs.

LIKELY FOLLOW-UPS

Batch calls, noisy tests, or a limit on subset size.

PITFALLS

Removing both members of a pair and missing other pairs that involve them. Binary searching without the monotone property of prefixes.

HOW TO THINK ABOUT IT

Each call is a yes/no question about a whole set, so binary search is possible: a failing set can be narrowed by halving. Keep the tests you have processed in groups that each pass runTest (independent sets). A new test only needs to be checked against each group, and if a group plus the new test fails, binary search finds its partners in that group in about log(size) calls each.

Basis: Written; group testing / finding all edges of a hidden graph with independent-set queries. runTest(S) is True when S contains no incompatible (bad) pair.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
bad = []
for a, b in combinations(tests, 2):
    if not runTest({a, b}):
        bad.append((a, b))
```

TIME	SPACE
n(n - 1) / 2 calls	O(1)

WHY IT IS SLOW

n(n - 1) / 2 calls even when nothing is bad. Keeping groups of mutually compatible tests and binary searching only when a group fails needs about n + p log n calls for p bad pairs.

OPTIMAL SOLUTION · Independent groups plus binary search for partners

TIME	SPACE
O(n * g + p log n) calls (g = number of groups, p = number of bad pairs)	O(n)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 How many calls does it need, and can you prove a lower bound?

With p bad pairs among n tests about p log n calls are needed in the worst case (each pair carries log n bits of information about where it is), and the group method uses O(n * g + p log n). With no bad pairs it uses n - 1 calls, one per test after the first.

FOLLOW-UP 2 Only one bad pair exists.

Binary search for the smallest failing prefix (its last element is in the pair), then binary search among the earlier elements for the partner: about 2 log n calls.

FOLLOW-UP 3 runTest is expensive but can run in parallel.

Test all groups for the new element in the same round, or process several elements at once with their own groups, trading a few extra calls for fewer sequential rounds.

FOLLOW-UP 4 Only one bad pair exists.

Binary search the shortest failing prefix (its last element is in the pair), then binary search the earlier tests for the partner: about 2 log2(n) calls instead of n(n - 1) / 2 (verified by counting calls).

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Minimize calls to find all bad test pairs
#
# You have \(n\) atomic tests \(t_1, t_2, \dots, t_n\).
# You are given access to a black-box function: python
# bool runTest(set<Test> S) which behaves as follows: -
# runTest(S) == True if the subset \(S\) contains **no**
# incompatible (bad) pair of tests. - runTest(S) == False
# if the subset \(S\) contains **at least one**
# incompatible (bad) pair of tests. There is some unknown
# set of unordered **bad pairs** \((t_i, t_j)\), where
# \(i\ne j\). You do **not** know in advance which pairs
# are bad or how many bad pairs there are. Your goal is to
# discover **all** bad pairs while making as few calls to
# runTest as possible. You may adaptively choose each
# subset \(S\) based on the results of previous calls. ###
```

```python
# Tasks 1. Design an algorithm that, using only calls to
# runTest, finds **all** bad pairs \((t_i, t_j)\). 2.
#
# FOLLOW-UPS (answered below, in this order):
# 1. How many calls does it need, and can you prove a
# lower bound?
# 2. Only one bad pair exists.
# 3. runTest is expensive but can run in parallel.
# 4. Only one bad pair exists.
#
# INTERVIEW NOTES
# Each call is a yes/no question about a whole set, so
# binary search is possible: a failing set can be narrowed
# by halving. Keep the tests you have processed in groups
# that each pass runTest (independent sets). A new test
# only needs to be checked against each group, and if a
# group plus the new test fails, binary search finds its
# partners in that group in about log(size) calls each.
#
# BRUTE FORCE (say it first): Call runTest on every pair
# Try each pair {a, b} on its own. Always correct, but the
# number of calls grows quadratically even if there are no
# bad pairs at all. Time n(n - 1) / 2 calls. Space O(1).


import itertools


# BRUTE FORCE: ask about every pair
def bad_pairs_brute(tests, run_test):
    # every pair costs a call: n * (n - 1) / 2 calls
    pairs = itertools.combinations(tests, 2)
    return {(a, b) for a, b in pairs
            if not run_test({a, b})}


# HELPERS for the base question
def shortest_failing_prefix(items, extra, run_test):
    # binary search: the smallest prefix of `items` that
    # fails
    # together with `extra`; its last item is a partner
    lo, hi = 1, len(items)
    while lo < hi:
        mid = (lo + hi) // 2
        if run_test(set(items[:mid]) | extra):
            lo = mid + 1
        else:
            hi = mid
    return items[lo - 1]


def partners(group, t, run_test):
    # every member of the group that is bad with t
    rest, found = list(group), []
    while rest and not run_test(set(rest) | {t}):
        x = shortest_failing_prefix(rest, {t}, run_test)
        found.append(x)
        rest.remove(x)
    return found


# BASE QUESTION (optimal): Independent groups plus binary
# search for partners
# Idea: Process tests one by one. Keep groups of tests
# that pass runTest.
def find_bad_pairs(tests, run_test):
    # keep groups of tests that pass together; a new test
    # is
    # checked against each group and joins one with no
    # partner
    groups, pairs = [], set()
    for t in tests:
        placed = False
        for group in groups:
            found = partners(group, t, run_test)
            for x in found:
                pairs.add((min(x, t), max(x, t)))
            if not found and not placed:
                group.append(t)
                placed = True
        if not placed:
            groups.append([t])
    return pairs


# FOLLOW-UP 2: Only one bad pair exists.
# Answer: Binary search for the smallest failing prefix
# (its last element is in the pair), then binary search
# among the earlier elements for the partner: about 2 log
# n calls.
# FOLLOW-UP 4: Only one bad pair exists.
# Answer: Binary search the shortest failing prefix (its
# last element is in the pair), then binary search the
# earlier tests for the partner: about 2 log2(n) calls
# instead of n(n - 1) / 2 (verified by counting calls).
def one_bad_pair(tests, run_test):
    # exactly one bad pair: about 2 * log2(n) calls
    if run_test(set(tests)):
        return None
    # the shortest failing prefix ends with one of the
    # pair
    b = shortest_failing_prefix(tests, set(), run_test)
    earlier = tests[:tests.index(b)]
    # then search the earlier tests for the partner
    a = shortest_failing_prefix(earlier, {b}, run_test)
    return (a, b)


if __name__ == "__main__":
    def make_oracle(bad):
        calls = [0]

        def run_test(subset):
            calls[0] += 1
            ordered = sorted(subset)
            pairs = itertools.combinations(ordered, 2)
            return not any(p in bad for p in pairs)
        return run_test, calls

    bad = {(3, 17), (5, 29)}

    run, calls = make_oracle(bad)
    assert find_bad_pairs(list(range(30)), run) == bad
    assert calls[0] < 435 // 2
    run, calls = make_oracle(set())
    assert find_bad_pairs(list(range(30)), run) == set()
    assert calls[0] <= 30
    run, calls = make_oracle({(10, 40)})
    assert one_bad_pair(list(range(64)), run) == (10, 40)
    # about 2 * log2(64) = 12 calls, far below 2016 pairs
    assert calls[0] <= 16
    run, _ = make_oracle({(1, 2)})
    assert bad_pairs_brute([1, 2, 3, 4], run) == {(1, 2)}
```

FUNCTION AND TEST CASES

Signature

```
def solution(n: int, bad_pairs: list[list[int]]) -> tuple[
list[list[int]], int]:
```

Sample call

```
solution(0, [])
```

Test cases

Input	Expected
(0, [])	([], 0)
(4, [])	([], 3)

ALSO REPORTED · 1 more

Find All Bad Pairs Using runTest Oracle | 1Point3Acres | onsite-coding | asked 2025-11-27 | Single report

https://www.1point3acres.com/interview/problems/company/google/find-bad-pairs-with-oracle

Standard "group testing" / Dorfman testing problem. Walk through the analogy with COVID pooled tests for intuition.
Algorithm sketch:
Test the full set. If true, no bad pairs.
Otherwise, binary-search to find one bad pair by partitioning the set and testing halves.
Remove the items that participate in the located bad pair; repeat.
Tighter bound: with up to K bad pairs, total queries are O(K log N) using adaptive group testing.
A naive $O(N^2)$ pairwise scheme works but the interviewer expects you to beat it with adaptive splitting.
Verbalize correctness: each binary-search step rules out half the candidates; you can always reduce the search space until exactly one bad pair is isolated.

Preparation: Read the Wikipedia article on group testing and Dorfman's two-stage scheme.
Drill recursive binary partitioning on a candidate set with positive/negative oracle feedback.
Have a back-of-envelope analysis ready: "Total calls O(K log N) for K bad pairs and N items."

STUDY HERE: topic 2, Arrays, Hashing and Two Pointers

Site tags: array, grid, medium

MEDIUM | 1Point3Acres | onsite-coding | 30 of 38

Rotate Any m×n 2D Matrix

STUDY: TYPE | **ALL TIME 56.3** | **30 DAYS 0** | **PRIORITY 57/228**

ABOUT THIS QUESTION

Topic: Arrays, Hashing and Two Pointers

Optimal answer from: written and tested here for this exact question

Related in your app (a neighbouring problem, not this exact question): LeetCode #48 Rotate Image

LeetCode match: LC 48 Rotate Image (related)

Frequency: Single report | Roles: SWE

Original: https://www.1point3acres.com/interview/problems/company/google/rotate-matrix-rectangle

QUESTION

EXAMPLES

```
[[1,2,3],[4,5,6]] (2×3) rotated 90° CW ->
    [[4,1],[5,2],[6,3]] (3×2).
```

APPROACH

Write square in-place rotation from memory in under 8 min. Have a 60-second explanation for why rectangle in-place is hard and what trade-offs you'd take.

INTERVIEW-DAY PLAN

CLARIFY FIRST

Ask which direction and angle, whether in place is required, and the dimensions. A non-square matrix cannot be rotated in place in the same buffer, since its shape changes.

BRUTE FORCE TO SAY OUT LOUD

Create a new matrix and copy each cell to its rotated position with an index formula. O(mn) space.

OPTIMAL IDEA

Clockwise 90 degrees: reverse the rows and transpose, [list(r) for r in zip(*matrix[::-1])]. Counter-clockwise: transpose then reverse the rows. 180 degrees: reverse the rows and each row. For a square matrix rotate in place by cycling four cells per layer.

DRY-RUN THESE CASES

The 2 x 3 example giving [[4,1],[5,2],[6,3]], 1 x 1, 1 x n, an empty matrix, four rotations returning the original.

LIKELY FOLLOW-UPS

In place square rotation, rotation by k times 90 degrees, or non-square in place with a flat array and cycles.

PITFALLS

Mixing up the index formula direction. Trying to rotate a rectangle in place in a 2D list.

HOW TO THINK ABOUT IT

A 90 degree clockwise rotation sends the cell at row i, column j to row j, column (m - 1 - i) of the new n x m matrix. That formula is the whole problem; 'reverse the rows then transpose' is just a short way to write it. A rectangle cannot be rotated in place because its shape changes.

Basis: Generalizes LeetCode 48, Rotate Image, from an n x n matrix to any m x n matrix.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def rotate(old):
    m, n = len(old), len(old[0])
    new = [[0] * m for _ in range(n)]
    for i in range(m):
        for j in range(n):
            new[j][m - 1 - i] = old[i][j]
    return new
```

TIME	SPACE
O(m * n)	O(m * n)

WHY IT IS SLOW

This is already optimal in time, O(m * n); its cost is the second matrix, O(m * n) extra space. A rectangle cannot be rotated in place (its shape changes), so it is the expected answer; the zip(reversed rows) one-liner is the same idea.

OPTIMAL SOLUTION · Rotate 90 degrees clockwise by reverse + transpose

TIME	SPACE
O(mn)	O(mn)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Can a non-square matrix be rotated in place?

Not in a rectangular buffer, because the dimensions swap. You can do it in place only if the matrix is stored as a flat array and you permute the elements along cycles of the index map (i, j) to (j, m - 1 - i), which is O(mn) time but needs cycle bookkeeping. In an interview, allocate the new matrix and say why.

FOLLOW-UP 2 How would you rotate by k * 90 degrees?

Reduce k modulo 4 first. Then apply the single-turn function that many times, or write the four direct index maps so you only build the output once.

FOLLOW-UP 3 Rotate by k quarter turns.

Reduce k modulo 4 first; four turns are the identity. Apply the single-turn function that many times (at most three), or write the three index formulas directly to build the answer once.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Rotate Any m×n 2D Matrix
#
# Examples: [[1,2,3],[4,5,6]] (2×3) rotated 90° CW ->
# [[4,1],[5,2],[6,3]] (3×2).
#
# FOLLOW-UPS (answered below, in this order):
# 1. Can a non-square matrix be rotated in place?
# 2. How would you rotate by k * 90 degrees?
# 3. Rotate by k quarter turns.
#
# INTERVIEW NOTES
# A 90 degree clockwise rotation sends the cell at row i,
# column j to row j, column (m - 1 - i) of the new n x m
# matrix. That formula is the whole problem; 'reverse the
# rows then transpose' is just a short way to write it. A
# rectangle cannot be rotated in place because its shape
# changes.
#
# BRUTE FORCE (say it first): Allocate the new matrix and
# apply the index formula
# Create an n x m matrix and copy each cell old[i][j] to
# new[j][m - 1 - i]. Correct and the clearest statement of
# the rotation; the zip trick below is the same thing in
# one line. Time O(m * n). Space O(m * n).


# BRUTE FORCE (say it first): Allocate the new matrix and
# apply the index formula | time O(m * n)
def rotate_by_formula(matrix):
    m = len(matrix)
    if m == 0:
        return []
    n = len(matrix[0])
    new = [[None] * m for _ in range(n)]
    for i in range(m):
        for j in range(n):
            new[j][m - 1 - i] = matrix[i][j]
    return new


# BASE QUESTION (optimal): Rotate 90 degrees clockwise by
# reverse + transpose
# Idea: Rotating clockwise puts the last row first, so
# reverse the rows and transpose (zip with *). A non-
# square matrix changes shape from m x n to n x m, which
```

```
# is why it cannot be done in place the way LeetCode 48
# is.
def rotate_cw(matrix):
    # reverse the rows, then swap rows and columns
    return [list(row) for row in zip(*matrix[::-1])]


# COUNTER-CLOCKWISE
def rotate_ccw(matrix):
    # swap rows and columns, then reverse the rows
    return [list(row) for row in zip(*matrix)][::-1]


# INDEX RULE version
def rotate_by_index(matrix):
    # the same result from the index rule, built once:
    # new[j][m - 1 - i] = old[i][j]
    m, n = len(matrix), len(matrix[0]) if matrix else 0
    new = [[None] * m for _ in range(n)]
    for i in range(m):
        for j in range(n):
            new[j][m - 1 - i] = matrix[i][j]
    return new


# FOLLOW-UP 2: How would you rotate by k * 90 degrees?
# Answer: Reduce k modulo 4 first. Then apply the single-
# turn function that many times, or write the four direct
# index maps so you only build the output once.
# FOLLOW-UP 3: Rotate by k quarter turns.
# Answer: Reduce k modulo 4 first; four turns are the
# identity. Apply the single-turn function that many times
# (at most three), or write the three index formulas
# directly to build the answer once.
def rotate_k(matrix, k):
    # four quarter turns are the identity: use k mod 4
    for _ in range(k % 4):
        matrix = rotate_cw(matrix)
    return matrix


if __name__ == "__main__":
    a = [[1, 2, 3], [4, 5, 6]]
    # the 2 x 3 example becomes 3 x 2
    assert rotate_cw(a) == [[4, 1], [5, 2], [6, 3]]
```

```
    assert rotate_by_index(a) == rotate_cw(a)
    assert rotate_ccw(a) == [[3, 6], [2, 5], [1, 4]]
    # a clockwise then an anticlockwise turn cancel out
    assert rotate_ccw(rotate_cw(a)) == a
    assert rotate_k(a, 4) == a
    assert rotate_k(a, 2) == [[6, 5, 4], [3, 2, 1]]
    assert rotate_k(a, 5) == rotate_cw(a)
    assert rotate_cw([[7]]) == [[7]]
    assert rotate_cw([]) == []
```

NOTES FROM CANDIDATES

The in-place rectangle rotation is essentially cycle-decomposition on the index permutation — explain it but don't burn time implementing.
Reported follow-up: rotate inside a window of a larger matrix while keeping the rest untouched.

PREPARATION

Write square in-place rotation from memory in under 8 min. Have a 60-second explanation for why rectangle in-place is hard and what trade-offs you'd take.

STUDY HERE: topic 2, Arrays, Hashing and Two Pointers

MEDIUM | IN APP LC 76 | Exponent | 31 of 38

★ TOP 75: rank 55 in the Medium and Hard priority list

★ Find the minimum window substring.

ASKED (ANY CO.) ~7 MONTHS AGO | OLDER

STUDY: TYPE | ALL TIME 42.5 | 30 DAYS 33 | PRIORITY 37/228

ABOUT THIS QUESTION

Topic: Arrays, Hashing and Two Pointers

Optimal answer from: your app's interview approach (LeetCode #76)

LeetCode match: LC 76 Minimum Window Substring (inferred from short statement)

Last asked (any company): 7 months ago, about 2026-03 (Exponent shows only a rough age) | +36 interviews

Original: https://www.aced.io/questions/1687/minimum-window-substring

QUESTION

Find the minimum window substring.

APPROACH

Sliding window with need/have counts: expand right until valid, then shrink left while valid, tracking the best window. O(n).

HOW TO THINK ABOUT IT

Expand the right end until the window covers everything in t, then shrink from the left as far as it stays valid, record the length, and continue. A count of 'characters still missing' makes the validity check O(1): it goes down when a needed character is added and up when one is removed. Each index enters and leaves the window once.

Basis: LeetCode 76, Minimum Window Substring: the shortest substring of s containing every character of t with multiplicity.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute(s, t):
    found = []
    for i in range(len(s)):
        for j in range(i + len(t), len(s) + 1):
            part = s[i:j]
            if contains_all(part, t):
                found.append(part)
    return min(found, key=len, default="")
```

TIME	SPACE
O(n^2 * m)	O(m)

WHY IT IS SLOW

BRUTE FORCE

O(n^2) substrings, each checked in O(m) or more. A sliding window that grows until valid and then shrinks from the left is O(n + m).

OPTIMAL SOLUTION · Sliding window with a 'still needed' counter

TIME	SPACE
O(n + m)	O(m)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 The smallest window containing every distinct character of s itself.

Use t = the set of distinct characters of s and run the same window. A related problem: the longest substring with at most k distinct characters uses a shrink-when-too-many window.

FOLLOW-UP 2 Return all minimum windows, or the number of valid substrings.

For all minimum windows record every window equal to the best length. For counting, when the window [left, right] is valid, every extension to the right is valid too, so add len(s) - right for each left.

FOLLOW-UP 3 Order matters: t must appear as a subsequence.

That is a different problem (LeetCode 727): DP over positions, or a two-pass scan (forward to match, backward to shrink) for each possible end.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Find the minimum window substring.
#
# Find the minimum window substring.
#
# FOLLOW-UPS (answered below, in this order):
# 1. The smallest window containing every distinct
# character of s itself.
# 2. Return all minimum windows, or the number of valid
# substrings.
# 3. Order matters: t must appear as a subsequence.
#
# INTERVIEW NOTES
# Expand the right end until the window covers everything
# in t, then shrink from the left as far as it stays
# valid, record the length, and continue. A count of
# 'characters still missing' makes the validity check
# O(1): it goes down when a needed character is added and
# up when one is removed. Each index enters and leaves the
# window once.
```

```
#
# BRUTE FORCE (say it first): Check every substring
# For every substring test whether its letter counts cover
# t, and keep the shortest. Correct and the oracle; cubic-
# ish in practice. Time O(n^2 * m). Space O(m).


from collections import Counter


def brute_window(s, t):
    # oracle: try every substring, shortest first
    need = Counter(t)
    for size in range(len(t), len(s) + 1):
        for i in range(len(s) - size + 1):
            if not need - Counter(s[i:i + size]):
                return s[i:i + size]
    return ""


# BRUTE FORCE (say it first): Check every substring | time
# O(n^2 * m)
def min_window_brute(s, t):
    need = Counter(t)
    best = ""
    for i in range(len(s)):
```

```
        for j in range(i + len(t), len(s) + 1):
            if not need - Counter(s[i:j]) and (
                best == "" or j - i < len(best)
            ):
                best = s[i:j]
    return best


# HELPERS for the base question
def valid_windows(s, t):
    # sliding window: grow `right` until the window holds
    # all
    # of t, then shrink `left` while it still does. Yield
    # every (left, right) that is valid when it is seen.
    need = Counter(t)
    missing = len(t)
    left = 0
    for right, ch in enumerate(s):
        if need[ch] > 0:
            missing -= 1
        need[ch] -= 1
        while missing == 0:
            yield left, right
            # drop the leftmost character and see if it
            # hurts
            need[s[left]] += 1
            if need[s[left]] > 0:
                missing += 1
            left += 1


# BASE QUESTION (optimal): Sliding window with a 'still
# needed' counter
# Idea: need[c] is how many more of c the window requires,
# and missing counts the distinct requirements still
# unmet.
def min_window(s, t):
    # the shortest window; the first one if there is a tie
    best, width = None, 0
    for left, right in valid_windows(s, t):
        if best is None or right - left < width:
            best, width = (left, right), right - left
    return s[best[0]:best[1] + 1] if best else ""
```

```
# FOLLOW-UP 1: The smallest window containing every
# distinct character of s itself.
# Answer: Use t = the set of distinct characters of s and
# run the same window. A related problem: the longest
# substring with at most k distinct characters uses a
# shrink-when-too-many window.
def window_all_distinct(s):
    # follow-up: contain every distinct character of s
    # itself
    return min_window(s, "".join(set(s)))


# FOLLOW-UP 2: Return all minimum windows, or the number
# of valid substrings.
# Answer: For all minimum windows record every window
# equal to the best length. For counting, when the window
# [left, right] is valid, every extension to the right is
# valid too, so add len(s) - right for each left.
def all_min_windows(s, t):
    # follow-up: every window of the minimum length
    found = list(valid_windows(s, t))
    if not found:
        return []
    size = min(r - l for l, r in found)
    return [s[l:r + 1] for l, r in found if r - l == size]


# FOLLOW-UP 2: Return all minimum windows, or the number
# of valid substrings.
# Answer: For all minimum windows record every window
# equal to the best length. For counting, when the window
# [left, right] is valid, every extension to the right is
# valid too, so add len(s) - right for each left.
def count_valid_substrings(s, t):
    # follow-up: how many substrings contain all of t. For
    # each left the first valid right is seen once; every
    # longer end is valid too.
    found = valid_windows(s, t)
    return sum(len(s) - right for _, right in found)


# FOLLOW-UP 3: Order matters: t must appear as a
# subsequence.
# Answer: That is a different problem (LeetCode 727): DP
# over positions, or a two-pass scan (forward to match,
```

```
# backward to shrink) for each possible end.
def min_window_subsequence(s, t):
    # follow-up (LeetCode 727): t must appear IN ORDER.
    # Match
    # forward from a start to find the end, then walk back
    # from that end to find the tightest start.
    best, width, i = None, 0, 0
    while i < len(s):
        j, k = 0, i
        while k < len(s) and j < len(t):
            if s[k] == t[j]:
                j += 1
            k += 1
        if j < len(t):
            break
        end, j, k = k - 1, len(t) - 1, k - 1
        while j >= 0:
            if s[k] == t[j]:
                j -= 1
            k -= 1
        start = k + 1
        if best is None or end - start < width:
            best, width = (start, end), end - start
        i = start + 1
    return s[best[0]:best[1] + 1] if best else ""


# HELPERS for the checks below
```

```
def longest_k_distinct(s, k):
    # related: the longest substring with at most k
    # distinct
    # letters shrinks when it has too many
    seen, left, best = Counter(), 0, 0
    for right, ch in enumerate(s):
        seen[ch] += 1
        while len(seen) > k:
            seen[s[left]] -= 1
            if seen[s[left]] == 0:
                del seen[s[left]]
            left += 1
        best = max(best, right - left + 1)
    return best


if __name__ == "__main__":
```

```
    assert min_window("ADOBECODEBANC", "ABC") == "BANC"
    assert min_window("a", "a") == "a"
    assert min_window("a", "aa") == ""
    assert min_window("ab", "b") == "b"
    assert min_window("aab", "aab") == "aab"
    assert all_min_windows("abcabc", "abc") == [
        "abc", "bca", "cab", "abc"]
    assert all_min_windows("xyz", "q") == []
    assert count_valid_substrings("abcabc", "abc") == 10
    assert window_all_distinct("aabcbcdbca") == "dbca"
    assert longest_k_distinct("eceba", 2) == 3
    assert longest_k_distinct("aa", 1) == 2
    text = "abcdebdde"
    assert min_window_subsequence(text, "bde") == "bcde"
    assert min_window_subsequence("abc", "ca") == ""
    import random
    for _ in range(400):
        s = "".join(random.choice("abc")
                    for _ in range(random.randint(0, 9)))
        t = "".join(random.choice("abc")
                    for _ in range(random.randint(1, 3)))
        assert min_window(s, t) == brute_window(s, t)
```

OFFICIAL PAGE TEXT

Upgrade to see our video answerBecome a member today100+ expert answer videosAccess to all coursesPeer-to-peer practicePrivate Discord communityFull question databaseUpgrade nowLewin (Dropbox SWE) answers the minimum window substring interview question.

STUDY HERE: topic 2, Arrays, Hashing and Two Pointers

MEDIUM | Exponent | 32 of 38

Given positions of N kids and M ice cream sellers on a 1D line, find the smallest distance E such that every kid lies within the range [P - E, P + E] of at least one seller.

ASKED (ANY CO.) ~A YEAR AGO | OLDER

STUDY: TALK | ALL TIME 26.5 | 30 DAYS 0 | PRIORITY 133/228

ABOUT THIS QUESTION

Topic: Arrays, Hashing and Two Pointers

Optimal answer from: written and tested here for this exact question

Related in your app (a neighbouring problem, not this exact question): LeetCode #475 Heaters

LeetCode match: No direct match; related LC 475 Heaters (inferred from short statement)

Last asked (any company): a year ago, about 2025-10 (Exponent shows only a rough age)

Original: https://www.aced.io/questions/5138/minimum-distance-kids-ice-cream-sellers

QUESTION

Given positions of N kids and M ice cream sellers on a 1D line, find the smallest distance E such that every kid lies within the range [P - E, P + E] of at least one seller.

APPROACH

Sort sellers; for each kid binary search the nearest seller; E = max over kids of the nearest distance. O((N + M) log M). Alternative: binary search E + two pointers.

INTERVIEW-DAY PLAN

CLARIFY FIRST

Ask whether positions are integers, whether several sellers can share a position, and whether E may be fractional (the answer is a distance, so it is one of the kid-seller distances).

BRUTE FORCE TO SAY OUT LOUD

Try every E over the range and check all kids. O(N * M * range).

OPTIMAL IDEA

Sort the sellers. For each kid binary search the nearest seller (check the neighbors on both sides of the insertion point). E is the maximum of those nearest distances. O((N + M) log M). Alternatively, binary search on E and check coverage with two pointers (LeetCode 475, Heaters).

DRY-RUN THESE CASES

A kid at a seller's position (0), all kids left of every seller, one seller, duplicates, kids between two sellers.

LIKELY FOLLOW-UPS

INTERVIEW-DAY PLAN

Each seller can serve at most C kids, which turns it into a matching problem solved by binary search plus greedy.

PITFALLS

Checking only the seller to the right of the kid. Forgetting to sort.

HOW TO THINK ABOUT IT

Each kid only cares about its nearest seller, and the radius E must cover the worst kid. So E is the maximum over kids of the distance to the nearest seller. With sellers sorted, the nearest seller of a kid is one of the two neighbours around its insertion point, which is a binary search.

Basis: LeetCode 475, Heaters (houses = kids, heaters = sellers); the answer is the largest distance from a kid to the nearest seller.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute(kids, sellers):
    worst = 0
    for kid in kids:
        gaps = [abs(kid - s) for s in sellers]
        nearest = min(gaps)
        worst = max(worst, nearest)
    return worst
```

TIME	SPACE
O(N * M)	O(1)

WHY IT IS SLOW

N * M distance checks. With the sellers sorted, binary search finds each kid's nearest seller among two neighbours, O((N + M) log M).

OPTIMAL SOLUTION · Sort the sellers, binary search each kid

TIME	SPACE
O((N + M) log M)	O(1) extra

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Both lists are already sorted, or you want to avoid the binary search.

Walk the kids in order with one pointer on the sellers that only moves forward while the next seller is at least as close. O(N + M) after sorting.

FOLLOW-UP 2 Each seller can serve at most C kids.

Now it is an assignment problem. Binary search the radius E and test feasibility with a greedy left-to-right matching that gives each seller up to C of the leftmost uncovered kids in its range.

FOLLOW-UP 3 Sellers can be placed anywhere (minimise the radius with S sellers).

That is the 1D k-center problem: binary search the radius and greedily place a seller at the farthest point covering the leftmost uncovered kid. O(N log range).

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Given positions of N kids and M ice cream
# sellers on a 1D line, find the smallest distance E such
# that every kid lies within the range [P - E, P + E] of
# at least one seller.
#
# Given positions of N kids and M ice cream sellers on a
# 1D line, find the smallest distance E such that every
# kid lies within the range [P - E, P + E] of at least one
# seller.
#
# FOLLOW-UPS (answered below, in this order):
# 1. Both lists are already sorted, or you want to avoid
# the binary search.
# 2. Each seller can serve at most C kids.
# 3. Sellers can be placed anywhere (minimise the radius
# with S sellers).
#
# INTERVIEW NOTES
# Each kid only cares about its nearest seller, and the
# radius E must cover the worst kid. So E is the maximum
# over kids of the distance to the nearest seller. With
# sellers sorted, the nearest seller of a kid is one of
# the two neighbours around its insertion point, which is
# a binary search.
#
# BRUTE FORCE (say it first): For each kid check every
# seller
# For each kid compute the distance to every seller and
# take the minimum, then take the maximum over kids.
# Correct; every kid scans all sellers. Time O(N * M).
# Space O(1).


from bisect import bisect_left


def brute(kids, sellers):
    # oracle: try E = 0, 1, 2, ... until every kid is
    # covered
    e = 0
    while not all(any(abs(k - s) <= e for s in sellers)
                  for k in kids):
        e += 1
    return e
```

```
# BRUTE FORCE (say it first): For each kid check every
# seller | time O(N * M)
```

```
def smallest_radius_brute(kids, sellers):
    return max(
        (
            min(abs(k - s) for s in sellers)
            for k in kids
        ),
        default=0,
    )


# HELPERS for follow-up 2
def feasible_capacity(kids, sellers, e, cap):
    # can every kid be served when a seller takes at most
    # `cap` kids? Sellers left to right each take the
    # leftmost kids that are still unserved and in range.
    i = 0
    for s in sellers:
        if i < len(kids) and kids[i] < s - e:
            return False
        taken = 0
        while i < len(kids) and taken < cap:
            if kids[i] > s + e:
                break
            i += 1
            taken += 1
    return i == len(kids)


# HELPERS for follow-up 3
def sellers_needed(kids, e):
    # greedy: the leftmost uncovered kid at x gets a
    # seller at
    # x + e, who covers every kid up to x + 2e
    count, reach = 0, None
    for k in sorted(kids):
        if reach is None or k > reach:
            count += 1
            reach = k + 2 * e
    return count


# BASE QUESTION (optimal): Sort the sellers, binary search
```

```
# each kid
# Idea: With the sellers sorted, bisect_left gives the
# first seller at or after the kid; the nearest seller is
# that one or the one before it. E is the largest of these
# nearest distances.
def smallest_e(kids, sellers):
    # each kid goes to the nearest seller; the answer is
    # the
    # worst such distance. The nearest seller is next to
    # kid's place in the sorted list of sellers.
    sellers = sorted(sellers)
    worst = 0
    for k in kids:
        i = bisect_left(sellers, k)
        near = INF = float("inf")
        if i < len(sellers):
            near = sellers[i] - k
        if i > 0:
            near = min(near, k - sellers[i - 1])
        worst = max(worst, near)
    return worst


# FOLLOW-UP 1: Both lists are already sorted, or you want
# to avoid the binary search.
# Answer: Walk the kids in order with one pointer on the
# sellers that only moves forward while the next seller is
# at least as close. O(N + M) after sorting.
def smallest_e_sorted(kids, sellers):
    # follow-up: sorted input, no binary search. One
```

```
    # pointer
    # on the sellers only moves forward.
    kids, sellers = sorted(kids), sorted(sellers)
    worst, j = 0, 0
    for k in kids:
        # move on while the next seller is at least as
        # close
        while j + 1 < len(sellers):
            here = abs(sellers[j] - k)
            if abs(sellers[j + 1] - k) > here:
                break
            j += 1
        worst = max(worst, abs(sellers[j] - k))
    return worst
```

```
# FOLLOW-UP 2: Each seller can serve at most C kids.
# Answer: Now it is an assignment problem. Binary search
# the radius E and test feasibility with a greedy left-to-
# right matching that gives each seller up to C of the
# leftmost uncovered kids in its range.
def smallest_e_capacity(kids, sellers, cap):
    # follow-up: binary search the distance
    kids, sellers = sorted(kids), sorted(sellers)
    if len(kids) > cap * len(sellers):
        return None
    lo, hi = 0, max(kids[-1], sellers[-1]) - min(
        kids[0], sellers[0])
    while lo < hi:
        mid = (lo + hi) // 2
        if feasible_capacity(kids, sellers, mid, cap):
            hi = mid
        else:
            lo = mid + 1
    return lo


# FOLLOW-UP 3: Sellers can be placed anywhere (minimise
# the radius with S sellers).
# Answer: That is the 1D k-center problem: binary search
# the radius and greedily place a seller at the farthest
# point covering the leftmost uncovered kid. O(N log
# range).
def best_radius(kids, how_many):
    # follow-up: sellers can stand ANYWHERE; the smallest
    # (whole number) radius that is enough for how_many
    kids = sorted(kids)
    lo, hi = 0, kids[-1] - kids[0]
    while lo < hi:
        mid = (lo + hi) // 2
        if sellers_needed(kids, mid) <= how_many:
            hi = mid
        else:
            lo = mid + 1
    return lo


if __name__ == "__main__":
    assert smallest_e([1, 5, 9], [2, 8]) == 3
    assert smallest_e([4], [4]) == 0
```

```
    assert smallest_e_sorted([1, 5, 9], [2, 8]) == 3
    # capacity 1: three kids need three sellers' worth
    assert smallest_e_capacity([1, 2, 3], [2], 1) is None
    spread = smallest_e_capacity([1, 2, 3], [1, 3, 9], 1)
    assert spread == 6
    assert smallest_e_capacity([1, 2], [5, 6], 1) == 4
    assert sellers_needed([1, 2, 3, 20], 1) == 2
    assert best_radius([1, 2, 3, 20], 2) == 1
    assert best_radius([5], 1) == 0
    import random
    for _ in range(400):
        kids = [random.randint(0, 20)
                for _ in range(random.randint(1, 6))]
        sellers = [random.randint(0, 20)
                   for _ in range(random.randint(1, 4))]
        want = brute(kids, sellers)
        assert smallest_e(kids, sellers) == want
        assert smallest_e_sorted(kids, sellers) == want
        assert smallest_e_capacity(
            kids, sellers, len(kids)) == want
```

OFFICIAL PAGE TEXT

Log in or sign up to answer this question.

STUDY HERE: topic 2, Arrays, Hashing and Two Pointers

MEDIUM | HackTheRounds | Onsite | 33 of 38

★ TOP 75: rank 66 in the Medium and Hard priority list

★ Shortest Subarray with K Distinct Integers

STUDY: TYPE | ALL TIME 42.5 | 30 DAYS 33 | PRIORITY 38/228

ABOUT THIS QUESTION

Topic: Arrays, Hashing and Two Pointers

Optimal answer from: the interview site's own solution

Related in your app (a neighbouring problem, not this exact question): LeetCode #992 Subarrays with K Different Integers, LeetCode #76 Minimum Window Substring

LeetCode match: LC 992 / LC 76 flavor

QUESTION

Given an array nums and an integer k, find the length of the shortest contiguous subarray that contains at least k distinct integers. If no such subarray exists, return -1.

EXAMPLES

Example 1	
Input	`nums = [1, 2, 1, 2, 3], k = 3`
Output	`3`
Why	`The shortest contiguous subarray with 3 distinct integers is [1, 2, 3].`

Example 2	
Input	`nums = [1, 1, 1, 1], k = 2`
Output	`-1`
Why	`Only 1 distinct integer exists.`

Example 3	
Input	`nums = [1, 2, 3, 4, 5], k = 3`
Output	`3`
Why	`Any 3 consecutive elements give 3 distinct integers.`

CONSTRAINTS

- 1 <= nums.length <= 10^5
- 1 <= nums[i] <= 10^9
- 1 <= k <= nums.length

APPROACH

Sliding window with a frequency hash map, O(n); shrink while distinct >= k

INTERVIEW-DAY PLAN

CLARIFY FIRST

Ask for the output if none exists (-1), whether k can exceed the number of distinct values, and whether the subarray must be contiguous (yes).

BRUTE FORCE TO SAY OUT LOUD

Check every subarray and count distinct values. O(n^3), or O(n^2) with a running set.

OPTIMAL IDEA

Sliding window with a count dict. Move right to add elements; whenever the window has at least k distinct values, record its length and then shrink from the left while it still has at least k distinct, updating the answer each time. Each index enters and leaves once, so O(n).

DRY-RUN THESE CASES

k = 1 (answer 1), k larger than distinct values (-1), all equal values, the shortest window in the middle, k = 0.

LIKELY FOLLOW-UPS

Longest subarray with at most k distinct, count of subarrays with exactly k distinct (LeetCode 992 using at-most minus at-most).

PITFALLS

Shrinking the window before recording the answer. Not deleting keys when counts reach zero.

HOW TO THINK ABOUT IT

Once a window has at least k distinct values, shrinking it from the left can only keep or lose distinct values, so the shortest valid window ending at each right end is found by shrinking as far as possible. A frequency map tells you when a value leaves the window, so each index enters and leaves once.

Basis: Sliding window (compare LeetCode 992 and 340). Shortest contiguous subarray with at least k distinct values; -1 if none.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute(nums, k):
    best = -1
    for i in range(len(nums)):
        seen = set()
        for j in range(i, len(nums)):
            seen.add(nums[j])
            if len(seen) >= k:
                size = j - i + 1
                if best == -1 or size < best:
                    best = size
                break
    return best
```

TIME	SPACE
O(n^2)	O(n)

WHY IT IS SLOW

BRUTE FORCE

O(n^2) in the worst case. A sliding window with a count map expands on the right and shrinks on the left, so each element is handled twice: O(n).

OPTIMAL SOLUTION · Sliding window with a frequency map

TIME	SPACE
O(n)	O(n)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Count the subarrays with exactly k distinct values (LeetCode 992).

exactly(k) = atMost(k) - atMost(k - 1), where atMost is a sliding window that shrinks while the distinct count exceeds the limit. Both are O(n).

FOLLOW-UP 2 Longest subarray with at most k distinct values.

The same window, but keep the longest window whose distinct count never exceeds k (shrink while it does). O(n).

FOLLOW-UP 3 The values are strings or large objects.

The frequency map works on any hashable value; for huge objects hash them first or map them to integer ids so the window comparisons stay cheap. **Cost:** O(n) plus hashing each element; mapping values to integer ids costs O(n) once.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Shortest Subarray with K Distinct
# Integers
#
# Given an array nums and an integer k, find the length of
# the shortest contiguous subarray that contains at least
# k distinct integers. If no such subarray exists, return
# -1.
#
# FOLLOW-UPS (answered below, in this order):
# 1. Count the subarrays with exactly k distinct values
# (LeetCode 992).
# 2. Longest subarray with at most k distinct values.
# 3. The values are strings or large objects.
#
# INTERVIEW NOTES
# Once a window has at least k distinct values, shrinking
# it from the left can only keep or lose distinct values,
# so the shortest valid window ending at each right end is
# found by shrinking as far as possible. A frequency map
# tells you when a value leaves the window, so each index
# enters and leaves once.
#
```

```
# BRUTE FORCE (say it first): Check every subarray
# For every start, extend the end while counting distinct
# values with a set, and record the shortest that reaches
# k. Correct, and quadratic. Time O(n^2). Space O(n).


from collections import Counter


def brute_shortest(nums, k):
    best = None
    for i in range(len(nums)):
        for j in range(i, len(nums)):
            if len(set(nums[i:j + 1])) >= k:
                size = j - i + 1
                if best is None or size < best:
                    best = size
    return -1 if best is None else best


# BRUTE FORCE (say it first): Check every subarray | time
# O(n^2)
def shortest_k_distinct_brute(nums, k):
```

```
    best = None
    for i in range(len(nums)):
        seen = set()
        for j in range(i, len(nums)):
            seen.add(nums[j])
            if len(seen) >= k:
                best = (
                    j - i + 1
                    if best is None
                    else min(best, j - i + 1)
                )
                break
    return -1 if best is None else best


# HELPERS for follow-up 1
def at_most(nums, k):
    # the number of subarrays with at most k distinct
    # values:
    # for every right end, all windows starting in
    # [left, right] qualify
    seen, left, total = Counter(), 0, 0
    for right, x in enumerate(nums):
        seen[x] += 1
        while len(seen) > k:
            seen[nums[left]] -= 1
            if seen[nums[left]] == 0:
                del seen[nums[left]]
            left += 1
        total += right - left + 1
    return total


# BASE QUESTION (optimal): Sliding window with a frequency
# map
# Idea: Expand the right end, counting each value. While
# the window has at least k distinct values, record its
# length and shrink from the left (deleting a value from
# the map when its count reaches 0).
def shortest_k_distinct(nums, k):
    # a window grows on the right; whenever it holds at
    # least k distinct values, record it and shrink from
    # the
    # left as far as it stays valid
    seen, left, best = Counter(), 0, None
```

```
    for right, x in enumerate(nums):
        seen[x] += 1
        while len(seen) >= k:
```

```
            size = right - left + 1
            if best is None or size < best:
                best = size
            seen[nums[left]] -= 1
            if seen[nums[left]] == 0:
                del seen[nums[left]]
            left += 1
    return -1 if best is None else best


# FOLLOW-UP 1: Count the subarrays with exactly k distinct
# values (LeetCode 992).
# Answer: exactly(k) = atMost(k) - atMost(k - 1), where
# atMost is a sliding window that shrinks while the
# distinct count exceeds the limit. Both are O(n).
def exactly(nums, k):
    # follow-up (LeetCode 992): exactly k distinct values
    return at_most(nums, k) - at_most(nums, k - 1)


# FOLLOW-UP 2: Longest subarray with at most k distinct
# values.
# Answer: The same window, but keep the longest window
# whose distinct count never exceeds k (shrink while it
# does). O(n).
def longest_at_most(nums, k):
    # follow-up: the longest window with at most k
    # distinct
    seen, left, best = Counter(), 0, 0
    for right, x in enumerate(nums):
        seen[x] += 1
        while len(seen) > k:
            seen[nums[left]] -= 1
            if seen[nums[left]] == 0:
                del seen[nums[left]]
            left += 1
        best = max(best, right - left + 1)
    return best


# FOLLOW-UP 3: The values are strings or large objects.
# Answer: The frequency map works on any hashable value;
```

```
# for huge objects hash them first or map them to integer
# ids so the window comparisons stay cheap. Cost: O(n)
# plus hashing each element; mapping values to integer ids
# costs O(n) once.
def to_ids(items):
    # follow-up: strings or big objects. Map each distinct
    # value to a small integer once, then reuse the same
    # code.
    ids = {}
    return [ids.setdefault(x, len(ids)) for x in items]


if __name__ == "__main__":
    assert shortest_k_distinct([1, 2, 1, 3, 4], 3) == 3
    assert shortest_k_distinct([1, 1, 1], 2) == -1
    assert shortest_k_distinct([5], 1) == 1
    assert shortest_k_distinct([], 1) == -1
    assert exactly([1, 2, 1, 2, 3], 2) == 7
    assert exactly([1, 2, 1, 3, 4], 3) == 3
    assert longest_at_most([1, 2, 1, 2, 3], 2) == 4
    words = ["a", "b", "a", "c"]
    assert shortest_k_distinct(to_ids(words), 3) == 3
    assert to_ids(["x", "y", "x"]) == [0, 1, 0]
    import random
    for _ in range(400):
        nums = [random.randint(1, 4)
                for _ in range(random.randint(0, 10))]
        k = random.randint(1, 5)
        want = brute_shortest(nums, k)
        assert shortest_k_distinct(nums, k) == want
```

```
        count = sum(len(set(nums[i:j + 1])) == k
                    for i in range(len(nums))
                    for j in range(i, len(nums)))
        assert exactly(nums, k) == count
```

ALSO REPORTED · 2 more

Find shortest subarray with at least k distinct | PracHub | Technical Screen

https://prachub.com/coding-questions/find-shortest-subarray-with-at-least-k-distinct

Find shortest subarray with >=k distinct integers | PracHub | Technical Screen

https://prachub.com/coding-questions/find-shortest-subarray-with-k-distinct-integers

STUDY HERE: topic 2, Arrays, Hashing and Two Pointers

MEDIUM | HackTheRounds | Onsite | 34 of 38

Sum of Two Squares in Two Ways

STUDY: READ | NO LEETCODE FREQUENCY | PRIORITY 206/228

ABOUT THIS QUESTION

Topic: Arrays, Hashing and Two Pointers

Optimal answer from: the interview site's own solution

QUESTION

Given an integer n, find all integers s < n such that s can be expressed as the sum of two squares in exactly two distinct unordered ways. For example: 50 = 1^2 + 7^2 = 5^2 + 5^2.

EXAMPLES

Example 1

Input	`n = 100`
Output	`[50, 65, 85]`
Why	`50 = 1^2 + 7^2 = 5^2 + 5^2, 65 = 1^2 + 8^2 = 4^2 + 7^2, 85 = 2^2 + 9^2 = 6^2 + 7^2`

Example 2

Input	`n = 50`
Output	`[]`
Why	`No number less than 50 has exactly two representations.`

CONSTRAINTS

- 1 <= n <= 2^31 - 1

APPROACH

Taxicab-number style: count a^2 + b^2 over a <= b with a hash map, keep sums with exactly two representations

INTERVIEW-DAY PLAN

CLARIFY FIRST

Ask whether pairs are unordered with a <= b, whether zero is allowed, and whether the bound is strict (s < n).

BRUTE FORCE TO SAY OUT LOUD

For each s below n try every a and b. O(n * sqrt(n)).

OPTIMAL IDEA

Loop over a <= b with a^2 + b^2 < n and count each sum in a hash map. Return the sums whose count is exactly two. There are about pi * n / 8 such pairs, so O(n) time and space. For the example, 50 = 1^2 + 7^2 = 5^2 + 5^2, so 50 qualifies when n > 50.

DRY-RUN THESE CASES

INTERVIEW-DAY PLAN

n = 50 (50 itself is excluded because the bound is strict), a small n with no answers, a sum with three representations (excluded), a perfect square such as 25 = 0 + 25 = 9 + 16 if zero is allowed.

LIKELY FOLLOW-UPS

Exactly k ways, or counting without a dictionary using a sieve.

PITFALLS

Counting (a, b) and (b, a) as different. Including s equal to n.

HOW TO THINK ABOUT IT

Do not test each s separately. Generate every pair (a, b) once, add its sum to a counter, and read the answer from the counter. The loop is bounded by a^2 + b^2 < n, so a and b are at most sqrt(n); a <= b removes order duplicates.

Basis: Taxicab-number style counting. The examples fix the rule: a and b are positive integers with a <= b, so 25 = 3^2 + 4^2 has only one representation (0^2 + 5^2 is not allowed).

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute(n):
    result = []
    for s in range(1, n):
        ways = 0
        for a in range(1, isqrt(s) + 1):
            rest = s - a * a
            b = isqrt(rest)
            if b >= a and b * b == rest:
                ways += 1
        if ways == 2:
            result.append(s)
    return result
```

TIME	SPACE
O(n * sqrt(n))	O(1)

WHY IT IS SLOW

Every s searches all pairs again, O(n * sqrt(n)). Generating each pair once and counting sums in a hash map is O(n).

OPTIMAL SOLUTION · Count every pair once in a hash map

TIME	SPACE
O(n)	O(n)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Exactly k ways instead of two.

Change the comparison from == 2 to == k; the counting loop is the same. For k = 3 the smallest is 325.

FOLLOW-UP 2 Allow zero (0^2 + b^2).

Start a at 0; then perfect squares like 25 gain a representation, so some answers appear or disappear. Check against the examples to see which convention the question uses (here, positive only).

FOLLOW-UP 3 n is huge (10^12) and you only need to test one number.

Factorise the number: the count of representations as a sum of two squares depends only on the primes congruent to 1 mod 4 (Jacobi's formula), so a factorisation answers it in sublinear time.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Sum of Two Squares in Two Ways
#
# Given an integer n, find all integers s < n such that s
# can be expressed as the sum of two squares in exactly
# two distinct unordered ways. For example: 50 = 1^2 + 7^2
# = 5^2 + 5^2.
#
# FOLLOW-UPS (answered below, in this order):
# 1. Exactly k ways instead of two.
# 2. Allow zero (0^2 + b^2).
# 3. n is huge (10^12) and you only need to test one
# number.
#
# INTERVIEW NOTES
# Do not test each s separately. Generate every pair (a,
# b) once, add its sum to a counter, and read the answer
# from the counter. The loop is bounded by a^2 + b^2 < n,
# so a and b are at most sqrt(n); a <= b removes order
# duplicates.
#
# BRUTE FORCE (say it first): For each s below n, try
# every a and b
# For every s count the pairs (a, b) with a <= b and a^2 +
# b^2 = s. Correct, but it re-searches pairs for every s.
# Time O(n * sqrt(n)). Space O(1).


from collections import Counter
from math import isqrt


def brute_count(m, zero=False):
    a = 0 if zero else 1
    count = 0
    while 2 * a * a <= m:
        rest = m - a * a
        b = isqrt(rest)
        if b * b == rest and rest > 0 and b >= a:
            count += 1
        a += 1
    return count


# BRUTE FORCE (say it first): For each s below n, try
# every a and b | time O(n * sqrt(n))
def two_squares_brute(n):
    out = []
    for s in range(1, n):
        ways = sum(
            1
            for a in range(1, int(s**0.5) + 1)
            for b in range(a, int(s**0.5) + 1)
            if a * a + b * b == s
        )
        if ways == 2:
            out.append(s)
    return out


# HELPERS for follow-up 3
def factorise(m):
    # trial division: fine up to about 10 ** 12
    out, p = {}, 2
    while p * p <= m:
        while m % p == 0:
            out[p] = out.get(p, 0) + 1
            m //= p
        p += 1
    if m > 1:
        out[m] = out.get(m, 0) + 1
    return out


# BASE QUESTION (optimal): Count every pair once in a hash
# map
# Idea: Loop a from 1 while 2a^2 < n and b from a while
# a^2 + b^2 < n, adding each sum to a Counter. There are
# about pi * n / 8 pairs. The answer is the sorted list of
# sums whose count is exactly 2.
# FOLLOW-UP 1: Exactly k ways instead of two.
# Answer: Change the comparison from == 2 to == k; the
# counting loop is the same. For k = 3 the smallest is
# 325.
# FOLLOW-UP 2: Allow zero (0^2 + b^2).
# Answer: Start a at 0; then perfect squares like 25 gain
# a representation, so some answers appear or disappear.
# Check against the examples to see which convention the
# question uses (here, positive only).
def sums_in_k_ways(n, k=2, zero=False):
    # every s < n is counted once per pair a <= b with
    # a*a + b*b = s. Keep the sums seen exactly k times.
    ways = Counter()
    # FOLLOW-UP 2: zero=True allows 0^2 + b^2
    a = 0 if zero else 1
    while 2 * a * a < n:
        b = a
        while a * a + b * b < n:
            ways[a * a + b * b] += 1
            b += 1
        a += 1
    # FOLLOW-UP 1: k ways instead of exactly two
    return sorted(s for s, c in ways.items()
                  if c == k and s > 0)


# FOLLOW-UP 3: n is huge (10^12) and you only need to test
# one number.
# Answer: Factorise the number: the count of
# representations as a sum of two squares depends only on
# the primes congruent to 1 mod 4 (Jacobi's formula), so a
# factorisation answers it in sublinear time.
def representations(m, zero=False):
    # follow-up: ONE huge number. The number of ways
    # depends
    # only on the primes that are 1 mod 4 (Jacobi): with
    # B = product of (exponent + 1) there are 4B ordered
    # signed pairs. Remove the special shapes a == 0 and
    # a == b, then every other unordered pair is counted 8
```

```
    # times.
    if m <= 0:
        return 0
    boost = 1
    for p, e in factorise(m).items():
        if p % 4 == 3 and e % 2:
            return 0
        if p % 4 == 1:
            boost *= e + 1
    is_square = isqrt(m) ** 2 == m
    half = m % 2 == 0 and isqrt(m // 2) ** 2 == m // 2
    plain = (4 * boost - 4 * is_square - 4 * half) // 8
    return plain + half + (is_square if zero else 0)


if __name__ == "__main__":
    assert sums_in_k_ways(100) == [50, 65, 85]
```

```
    assert sums_in_k_ways(10) == []
    assert sums_in_k_ways(1000, 3)[0] == 325
    # with zero allowed, perfect squares gain a way
    assert 25 not in sums_in_k_ways(100)
    assert 25 in sums_in_k_ways(100, 2, zero=True)
    assert representations(50) == 2
    assert representations(325) == 3
    assert representations(3) == 0
    assert representations(25) == 1
    assert representations(25, zero=True) == 2
    # a 12 digit number in a moment (13 * 5 * 5 * ...)
    assert representations(5 ** 5 * 13 * 17 * 29) > 10
    for m in range(1, 600):
        assert representations(m) == brute_count(m), m
        both = representations(m, True)
        assert both == brute_count(m, True)
        if m < 100:
            two = representations(m) == 2
            assert (m in sums_in_k_ways(100)) == two
```

ALSO REPORTED · 1 more

Find numbers with exactly two sum-of-squares forms | PracHub | Technical Screen

https://prachub.com/coding-questions/find-numbers-with-exactly-two-sum-of-squares-forms

STUDY HERE: topic 2, Arrays, Hashing and Two Pointers

Site tags: math, hashmap, hard

HARD | 1Point3Acres | onsite-coding | 35 of 38

Maximum-Area Rectangle from Planar Points

ASKED 2026-09-29 | 21 DAYS AGO | LAST 30 DAYS

STUDY: TALK **NO LEETCODE FREQUENCY** **PRIORITY 177/228**

ABOUT THIS QUESTION

Topic: Arrays, Hashing and Two Pointers

Optimal answer from: written and tested here for this exact question

Related in your app (a neighbouring problem, not this exact question): LeetCode #963 Minimum Area Rectangle II

LeetCode match: LC 963 Minimum Area Free Rectangle (related)

Frequency: Low | Last asked: 2026-09-29 | Roles: SWE

Original: https://www.1point3acres.com/interview/problems/company/google/max-area-rectangle-from-points

QUESTION

Given 2D points, find the maximum area of a rectangle whose four corners are input points; sides need not be parallel to the axes. Candidates compare it to LC 963 (Minimum Area Rectangle II) but maximizing. Interviewers ask to improve beyond an O(N^3) approach.

Input: a list of 2D coordinate points.
Return the maximum area of any rectangle whose four vertices are all input points.
Rectangle sides do not need to be parallel to the coordinate axes.
Follow-up pressure: start from an O(N^3) idea, then optimize toward a better complexity.

APPROACH

Practice the diagonal-grouping idea: two point pairs form a rectangle when they share the same midpoint and the same length, so bucket all pairs by (midpoint, squared length) and compare pairs within each bucket.
Write the area computation from two adjacent side vectors and verify it on both an axis-aligned and a 45-degree rotated rectangle.
Prepare the complexity argument: O(N^2) pairs to bucket, with the worst case driven by bucket sizes.

HOW TO THINK ABOUT IT

Four points form a rectangle exactly when two of its diagonals share the same midpoint and have the same length. So you never need to pick four points: bucket every pair of points by (sum of the two points, squared distance). Two different pairs in one bucket are the two diagonals of a rectangle, and the area is the cross product of two adjacent sides.

HOW TO THINK ABOUT IT

Basis: LeetCode 963, Minimum Area Free Rectangle, with the objective flipped to maximum area.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute(points):
    best = 0
    for p, q, r in permutations(points, 3):
        if dot(q - p, r - p) != 0:
            continue  # no right angle at p
        s = q + r - p  # the fourth corner
        if s in points:
            area = abs(cross(q - p, r - p))
            best = max(best, area)
    return best
```

TIME	SPACE
O(n^3)	O(n)

WHY IT IS SLOW

Three nested loops over the points, O(n^3). Bucketing pairs by their diagonal midpoint and length finds rectangles from O(n^2) pairs.

OPTIMAL SOLUTION · Bucket pairs by diagonal midpoint and length

TIME	SPACE
O(n^2) pairs (worst case O(n^3) if every pair shares a bucket)	O(n^2)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Can you do better than O(n^2) pairs?

Not in general: the answer can depend on all pairs, and even deciding whether any rectangle exists is believed to need about n^2 work. The practical gain is in the buckets: most buckets hold one pair, so the work after bucketing is small. State the worst case honestly (points on a circle put many pairs in few buckets).

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Maximum-Area Rectangle from Planar Points
#
# Given 2D points, find the maximum area of a rectangle
# whose four corners are input points; sides need not be
# parallel to the axes. Candidates compare it to LC 963
# (Minimum Area Rectangle II) but maximizing. Interviewers
# ask to improve beyond an O(N^3) approach. Input: a list
```

```
# of 2D coordinate points. Return the maximum area of any
# rectangle whose four vertices are all input points.
# Rectangle sides do not need to be parallel to the
# coordinate axes. Follow-up pressure: start from an
# O(N^3) idea, then optimize toward a better complexity.
#
# FOLLOW-UPS (answered below, in this order):
# 1. Can you do better than O(n^2) pairs?
#
# INTERVIEW NOTES
# Four points form a rectangle exactly when two of its
# diagonals share the same midpoint and have the same
# length. So you never need to pick four points: bucket
# every pair of points by (sum of the two points, squared
# distance). Two different pairs in one bucket are the two
# diagonals of a rectangle, and the area is the cross
# product of two adjacent sides.
#
# BRUTE FORCE (say it first): Try every triple with a
# right angle
# Pick a corner p and two neighbours q and r with (q - p)
# . (r - p) = 0, then check that the fourth point q + r -
# p exists. Correct and the first thing to say; cubic is
# too slow for thousands of points. Time O(n^3). Space
# O(n).


from collections import defaultdict
from itertools import combinations


# HELPERS for the brute force
def cross(o, p, q):
    # twice the signed area of triangle o, p, q
    ax, ay = p[0] - o[0], p[1] - o[1]
    bx, by = q[0] - o[0], q[1] - o[1]
    return ax * by - ay * bx
```

```
# BRUTE FORCE (say it first)
def max_rect_area_cubic(points):
    # first idea to say aloud: try every corner triple
    pts = set(map(tuple, points))
    best = 0
    for p in pts:
        for q in pts:
            for r in pts:
                if len({p, q, r}) < 3:
                    continue
                # right angle at p between p->q and p->r
                dot = ((q[0] - p[0]) * (r[0] - p[0])
                       + (q[1] - p[1]) * (r[1] - p[1]))
                # the fourth corner must also exist
                s = (q[0] + r[0] - p[0],
                     q[1] + r[1] - p[1])
                if dot == 0 and s in pts:
                    area = abs(cross(p, q, r))
                    best = max(best, area)
    return best


# HELPERS for the base question
# FOLLOW-UP 1: Can you do better than O(n^2) pairs?
# Answer: Not in general: the answer can depend on all
# pairs, and even deciding whether any rectangle exists is
# believed to need about n^2 work.
# NEEDED BY THE BASE QUESTION: it is also the answer to
# the follow-up below
def all_rectangle_areas(points):
    # duplicates cannot help, so drop them
    pts = list(set(map(tuple, points)))
    buckets = defaultdict(list)
    # every pair is a possible diagonal
    for p, q in combinations(pts, 2):
        # same centre (sum of corners) and same length
        key = (p[0] + q[0], p[1] + q[1],
               (p[0] - q[0]) ** 2 + (p[1] - q[1]) ** 2)
        buckets[key].append((p, q))
    # two diagonals in one bucket form a rectangle
    for pairs in buckets.values():
        for (p, q), (r, s) in combinations(pairs, 2):
            # area = |cross| of two sides leaving p
            yield abs(cross(p, r, s))
```

```


# BASE QUESTION (optimal): Bucket pairs by diagonal
# midpoint and length
# Idea: For every pair (p, q) use the key (p.x + q.x, p.y
# + q.y, squared length). All pairs with the same key are
# diagonals of one circle around the same centre, so any
# two of them form a rectangle.
def max_rect_area(points):
    # best area, or 0 when no rectangle exists
    return max(all_rectangle_areas(points), default=0)


# RELATED VARIANT: smallest area (LeetCode 963)
def min_rect_area(points):
    # LeetCode 963 asks for the smallest instead
    return min(all_rectangle_areas(points), default=0)


if __name__ == "__main__":
    # a tilted square of area 2
    sq = [(0, 0), (1, 1), (0, 2), (-1, 1)]
    assert max_rect_area(sq) == 2
    assert max_rect_area_cubic(sq) == 2
    # axis-aligned 2 x 1 rectangle plus a stray point
    rect = [(0, 0), (2, 0), (0, 1), (2, 1), (5, 5)]
    assert max_rect_area(rect) == 2
    # collinear points make no rectangle
    assert max_rect_area([(0, 0), (1, 1), (2, 2)]) == 0
    # the minimum version picks the small one
    two = rect + [(0, 3), (2, 3)]
    assert min_rect_area(two) == 2
    assert max_rect_area(two) == 6
```

NOTES FROM CANDIDATES

Candidates describe it as LC 963 (Minimum Area Rectangle II) with the objective flipped to maximum area.
Asked as a standalone L4 onsite coding round; in another L4 onsite it was the second problem after a CPU scheduling question, and the interviewer explicitly expected a better-than-cubic solution.
Clarify up front: whether duplicate points can appear, what to return when no rectangle exists (commonly 0), and whether integer or floating-point coordinates are used (area precision matters for rotated rectangles).

PREPARATION

Practice the diagonal-grouping idea: two point pairs form a rectangle when they share the same midpoint and the same length, so bucket all pairs by (midpoint, squared length) and compare pairs within each bucket.
Write the area computation from two adjacent side vectors and verify it on both an axis-aligned and a 45-degree rotated rectangle.
Prepare the complexity argument: O(N^2) pairs to bucket, with the worst case driven by bucket sizes.

STUDY HERE: topic 2, Arrays, Hashing and Two Pointers

HARD | IN APP LC 41 | PracHub | Technical Screen | 36 of 38

Smallest Missing Positive Integer in Linear Time and Constant Extra Space

POSTED 2026-09-09 | 41 DAYS AGO | LAST 90 DAYS

STUDY: TYPE ALL TIME 48 30 DAYS 0 PRIORITY 73/228

ABOUT THIS QUESTION

Topic: Arrays, Hashing and Two Pointers

Optimal answer from: the interview site's own solution

Also in your app: LeetCode #41 First Missing Positive

LeetCode match: LC 41 First Missing Positive

Posted on PracHub: 2026-09-09 (updated 2026-09-28)

Original: https://prachub.com/coding-questions/smallest-missing-positive-integer-in-linear-time-and-constant-extra-space

QUESTION

You are given an unsorted list of integers **nums**, which may contain negative numbers, zeros, duplicates, and values far larger than the length of the list. Return the smallest positive integer, that is the smallest integer that is at least **1**, that does not appear in **nums**. In the interview, a first solution that ran in linear time but used extra memory proportional to the input was accepted as a starting point, and the interviewer then asked for the same result using only constant extra space. Aim for that tighter target.

Function Signature

```
def first_missing_positive(nums: list[int]) -> int:
```

Rules

- Target complexity: **O(n)** time and **O(1)** auxiliary space, where **n = len(nums)**. The list may be rearranged or overwritten in place; its contents after the call do not matter.
- Return only the missing value.

Constraints

- **1 <= len(nums) <= 10^5**
- **-2147483648 <= nums[i] <= 2147483647**
- The answer is a positive integer that fits in a signed 32-bit integer.

Examples

Example 1

- Input: **nums = [4, -2, 1, 2]**
- Output: **3**
- Explanation: **1** and **2** are present and **3** is not.

Example 2

- Input: **nums = [5, 6, 100, -3]**
- Output: **1**
- Explanation: **1** does not appear, so it is the answer, however large the other values are.

QUESTION

Example 3

- Input: **nums = [1, 2, 2, 3]**
- Output: **4**
- Explanation: **1**, **2** and **3** all appear (**2** twice), so the smallest missing positive integer is **4**.

APPROACH

Cyclic placement: swap each value v in 1..n into index v-1 until settled, then the first index i where nums[i] != i+1 gives the answer (else n+1). O(n) time, O(1) space.

HOW TO THINK ABOUT IT

The answer is between 1 and n + 1, because n numbers can cover at most n of the positive integers. So the array itself can be used as the 'seen' table: put each value v (1 <= v <= n) at index v - 1. After that, the first index i where nums[i] != i + 1 gives the answer i + 1.

Basis: LeetCode 41, First Missing Positive.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute(nums):
    seen = set(nums)
    x = 1
    while x in seen:
        x += 1
    return x
```

TIME	SPACE
O(n)	O(n)

WHY IT IS SLOW

O(n) time but O(n) extra memory, which is what the follow-up asks to remove. Placing each value v at index v - 1 inside the array itself, then finding the first mismatch, uses O(1) space.

OPTIMAL SOLUTION · Cyclic placement: put each value v at index v - 1

TIME	SPACE
O(n)	O(1)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Do it without swapping (sign marking).

Replace out-of-range values by n + 1, then for every value v in 1..n make the entry at index v - 1 negative as a 'seen' flag; the first positive entry marks the missing number. Same O(n) time and O(1) space.

FOLLOW-UP 2 The array must not be modified.

Then O(1) space with O(n) time is not possible in general; copy the array (O(n) space) or accept O(n log n) time with a sort of a copy. State the trade-off.

FOLLOW-UP 3 Smallest missing non-negative integer, or the k-th missing positive.

Shift the target range by one for non-negative. For the k-th missing positive in a sorted array use binary search on 'how many numbers are missing before index i' = a[i] - (i + 1).

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Smallest Missing Positive Integer in
# Linear Time and Constant Extra Space
#
# You are given an unsorted list of integers nums, which
# may contain negative numbers, zeros, duplicates, and
# values far larger than the length of the list. Return
# the smallest positive integer, that is the smallest
# integer that is at least 1, that does not appear in
# nums. In the interview, a first solution that ran in
# linear time but used extra memory proportional to the
# input was accepted as a starting point, and the
# interviewer then asked for the same result using only
# constant extra space. Aim for that tighter target. ###
# Function Signature python def
# first_missing_positive(nums: list[int]) -> int: ###
# Rules - Target complexity: O(n) time and O(1) auxiliary
# space, where n = len(nums). The list may be rearranged
# or overwritten in place; its contents after the call do
# not matter. - Return only the missing value.
#
# FOLLOW-UPS (answered below, in this order):
# 1. Do it without swapping (sign marking).
# 2. The array must not be modified.
# 3. Smallest missing non-negative integer, or the k-th
# missing positive.
#
# INTERVIEW NOTES
# The answer is between 1 and n + 1, because n numbers can
# cover at most n of the positive integers. So the array
# itself can be used as the 'seen' table: put each value v
# (1 <= v <= n) at index v - 1. After that, the first
# index i where nums[i] != i + 1 gives the answer i + 1.
#
# BRUTE FORCE (say it first): Hash set of the values, then
# count up
# Put all numbers in a set and test 1, 2, 3, ... until one
# is missing. Linear time but linear extra memory, which
# is the version the interviewer wants to improve. Time
# O(n). Space O(n).


from bisect import bisect_left


# BASE QUESTION (optimal): Cyclic placement: put each
# value v at index v - 1
# Idea: For each position, while nums[i] is in 1..n and
# not already at its home index, swap it to index nums[i]
# - 1. Each swap puts one value in its final place, so the
# total number of swaps is at most n.
def first_missing_positive(nums):
    # put each value v in 1..n at index v - 1 by swapping
    a, n = list(nums), len(nums)
    for i in range(n):
        while 1 <= a[i] <= n and a[a[i] - 1] != a[i]:
            j = a[i] - 1
            a[i], a[j] = a[j], a[i]
    # the first index that does not hold index + 1
    for i in range(n):
        if a[i] != i + 1:
            return i + 1
    return n + 1


# EASY VERSION: O(n) memory (say it first)
def first_missing_set(nums):
    # the easy O(n) memory answer (fine as a first
    # version)
    seen, v = set(nums), 1
    while v in seen:
        v += 1
    return v


# VARIANT: k-th missing positive
def kth_missing_positive(sorted_nums, k):
    # follow-up: k-th missing positive in a sorted array.
    # missing before index i is a[i] - (i + 1): binary
    # search
    lo, hi = 0, len(sorted_nums)
    while lo < hi:
        mid = (lo + hi) // 2
        if sorted_nums[mid] - (mid + 1) < k:
            lo = mid + 1
        else:
            hi = mid
    return lo + k


# FOLLOW-UP 1: Do it without swapping (sign marking).
# Answer: Replace out-of-range values by n + 1, then for
# every value v in 1..n make the entry at index v - 1
# negative as a 'seen' flag; the first positive entry
# marks the missing number. Same O(n) time and O(1) space.
def first_missing_marking(nums):
    # no swaps: use the sign of a[v - 1] as a "seen" flag
    a, n = list(nums), len(nums)
    # values outside 1..n cannot matter: replace them
    for i in range(n):
        if a[i] <= 0 or a[i] > n:
            a[i] = n + 1
    for i in range(n):
        v = abs(a[i])
        if v <= n:
            a[v - 1] = -abs(a[v - 1])
    # the first positive entry marks the missing number
    for i in range(n):
        if a[i] > 0:
            return i + 1
    return n + 1


if __name__ == "__main__":
    cases = [([3, 4, -1, 1], 2), ([1, 2, 0], 3),
             ([7, 8, 9, 11, 12], 1), ([], 1),
             ([1, 1, 1], 2), ([2, 1], 3)]
    for nums, want in cases:
        assert first_missing_positive(nums) == want
        assert first_missing_marking(nums) == want
        assert first_missing_set(nums) == want
```

```
    # the input list is left untouched
    original = [3, 4, -1, 1]
    first_missing_positive(original)
    assert original == [3, 4, -1, 1]
    # 2, 3, 4, 7, 11: the 5th missing positive is 9
    assert kth_missing_positive([2, 3, 4, 7, 11], 5) == 9
    assert kth_missing_positive([1, 2, 3, 4], 2) == 6
```

FUNCTION AND TEST CASES

Signature

```
def first_missing_positive(nums: list[int]) -> int:
```

Sample call

```
first_missing_positive([4, -2, 1, 2])
```

Test cases

Input	Expected
([4, -2, 1, 2],)	3
([5, 6, 100, -3],)	1

STUDY HERE: topic 2, Arrays, Hashing and Two Pointers

Site tags: string, two-pointer, math-reasoning, hard

HARD | 1Point3Acres | onsite-coding | 37 of 38

Necklace Cut into Two Halves with Equal D/R Counts

ASKED 2026-05-03 | 170 DAYS AGO | LAST 6 MONTHS

STUDY: TALK **NO LEETCODE FREQUENCY** **PRIORITY 188/228**

ABOUT THIS QUESTION

Topic: Arrays, Hashing and Two Pointers

Optimal answer from: written and tested here for this exact question

Frequency: Single report | Last asked: 2026-05-03 | Roles: SWE, MLE

Original: https://www.1point3acres.com/interview/problems/company/google/necklace-cut-equal-counts

QUESTION

EXAMPLES

```
s = "DRDR"    -> cut between index 1 and 2: "DR" +
    "DR".
s = "DDRR"    -> 1-cut not balanced ("DD" vs "RR");
    rotate, two cuts at offsets 1 and 3:
              pieces "DR" and "DR".
```

APPROACH

Practice the running-balance technique on +1/-1 arrays (the contiguous-array and subarray-sums-divisible-by-k families). Drill sliding-window-on-circular-array unrolling (duplicate the string s+s and use a window of length n).

INTERVIEW-DAY PLAN

CLARIFY FIRST

Ask whether the string is circular, whether the two pieces must be contiguous arcs, whether equal counts of each letter or equal strings are needed, and whether one cut or two are allowed.

BRUTE FORCE TO SAY OUT LOUD

Try every pair of cut positions and count letters. O(n^3) or O(n^2) with prefix counts.

OPTIMAL IDEA

Each arc must hold half of the D's and half of the R's, so the total counts of both must be even and each arc has length n / 2. Slide a window of length n / 2 around s + s tracking its D count; the window starting at i and the one starting at i + n / 2 are complementary, and counts change by at most one per step, so some start hits exactly half. O(n).

DRY-RUN THESE CASES

DRDR, DDRR (two cuts needed), an odd D count (impossible), a string of one letter.

LIKELY FOLLOW-UPS

Equal strings instead of equal counts, or three pieces.

PITFALLS

INTERVIEW-DAY PLAN

Assuming a linear cut is always enough. Ignoring parity.

HOW TO THINK ABOUT IT

Two arcs of a circle with the same letter counts must each have exactly half the D's and half the R's, so the total counts must be even and each arc has length n / 2. Slide a window of length n / 2 around the circle: the window and its complement have D counts adding to the total, and the count changes by at most one per step, so it must hit half somewhere.

Basis: Circular sliding window / prefix balance (cf. LeetCode 525, Contiguous Array). Assumption: cut the circular necklace at two points into two arcs, each with the same number of D and R.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute(s):
    n = len(s)
    for i, j in combinations(range(n), 2):
        arc1 = s[i:j]
        arc2 = s[j:] + s[:i]
        d1, d2 = arc1.count("D"), arc2.count("D")
        r1, r2 = arc1.count("R"), arc2.count("R")
        if d1 == d2 and r1 == r2:
            return True
    return False
```

TIME	SPACE
O(n^3)	O(n)

WHY IT IS SLOW

O(n^2) pairs and each recount costs O(n), so O(n^3). Because the arcs must both have length n / 2, one circular window of that length slid around the string finds it in O(n).

OPTIMAL SOLUTION · Fixed-length circular window with the discrete intermediate value argument

TIME	SPACE
O(n)	O(n)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 When is a single cut enough?

A single linear cut works when some prefix has exactly half the D's and half the R's, i.e. a prefix of length n/2 whose D count is d_total / 2. Otherwise rotate: the circular window above finds the start, which is equivalent to two cuts on the circle.

FOLLOW-UP 2 What if the two pieces must be equal strings, not just equal counts?

That is a different question: the necklace must be a square rotation, i.e. some rotation of s is t + t. Check whether s appears inside (s + s)[1:-1] at index n/2, or compare s[i:i + n/2] with the complementary arc using a rolling hash or KMP over s + s. It is a string matching problem, not a counting one.

FOLLOW-UP 3 The two pieces must be equal strings, not just equal counts.

Then the necklace must be a square in some rotation: some rotation of s equals t + t. Test with s in (s + s)[1:-1] restricted to an even-length split, or compare s[i:i + n/2] with s[i + n/2:i + n] using a rolling hash or KMP.

FOLLOW-UP 4 The two pieces must be equal strings, not just have equal counts.

A rotation of the necklace must be of the form t + t. Try every rotation and compare its two halves (O(n^2), or O(n) by searching s in (s + s)[1:-1] and checking the match at an offset of n / 2).

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Necklace Cut into Two Halves with Equal
# D/R Counts
#
# Examples: s = "DRDR" -> cut between index 1 and 2: "DR"
+
# "DR". s = "DDRR" -> 1-cut not balanced ("DD" vs "RR");
# rotate, two cuts at offsets 1 and 3: pieces "DR" and
# "DR".
#
# FOLLOW-UPS (answered below, in this order):
# 1. When is a single cut enough?
# 2. What if the two pieces must be equal strings, not
# just equal counts?
# 3. The two pieces must be equal strings, not just equal
# counts.
# 4. The two pieces must be equal strings, not just have
# equal counts.
#
# INTERVIEW NOTES
# Two arcs of a circle with the same letter counts must
# each have exactly half the D's and half the R's, so the
# total counts must be even and each arc has length n / 2.
# Slide a window of length n / 2 around the circle: the
# window and its complement have D counts adding to the
# total, and the count changes by at most one per step, so
# it must hit half somewhere.
#
# BRUTE FORCE (say it first): Try every pair of cut points
# Choose two cut positions, form the two arcs, and compare
# their letter counts. Correct and the oracle; it checks
# every pair of cuts and recounts letters each time. Time
# O(n^3). Space O(n).
```

```
# BRUTE FORCE (say it first): Try every pair of cut points
# | time O(n^3)
def necklace_brute(s):
    n = len(s)
    for i in range(n):
        for j in range(i + 1, n):
            a, b = s[i:j], s[j:] + s[:i]
            if (
                a
                and b
                and a.count("D") == b.count("D")
                and a.count("R") == b.count("R")
            ):
                return True
    return False


# BASE QUESTION (optimal): Fixed-length circular window
# with the discrete intermediate value argument
# Idea: If each arc has half of the D's and half of the
# R's, each arc has length n/2, and the total D and R
# counts must both be even. Slide a window of length n/2
# around the circle (use s + s), tracking its D count.
def necklace_two_cuts(s):
    n = len(s)
    d_total, r_total = s.count("D"), s.count("R")
    # each piece needs half of both letters
    if n % 2 or d_total % 2 or r_total % 2:
        return None
    half, target = n // 2, d_total // 2
    doubled = s + s
    # slide a window of length n/2 around the circle
    window = doubled[:half].count("D")
    for i in range(n):
        if window == target:
            # cuts at i and i + n/2 give two good halves
            first = doubled[i:i + half]
            second = doubled[i + half:i + n]
            return i, (i + half) % n, first, second
        # move the window one step: add the new, drop the
        # old
        window += (doubled[i + half] == "D")
        window -= (doubled[i] == "D")
    return None


# SINGLE CUT case
def single_cut(s):
    # one cut on the line: a prefix with half of
    # everything
    half = len(s) // 2
    if len(s) % 2:
        return None
    d_prefix = s[:half].count("D")
    return half if d_prefix * 2 == s.count("D") else None
```

```
# FOLLOW-UP 2: What if the two pieces must be equal
# strings, not just equal counts?
# Answer: That is a different question: the necklace must
# be a square rotation, i.e. some rotation of s is t + t.
# FOLLOW-UP 3: The two pieces must be equal strings, not
# just equal counts.
# Answer: Then the necklace must be a square in some
# rotation: some rotation of s equals t + t. Test with s
# in (s + s)[1:-1] restricted to an even-length split, or
# compare s[i:i + n/2] with s[i + n/2:i + n] using a
# rolling hash or KMP.
# FOLLOW-UP 4: The two pieces must be equal strings, not
# just have equal counts.
# Answer: A rotation of the necklace must be of the form t
# + t. Try every rotation and compare its two halves
```

```python
# (O(n^2), or O(n) by searching s in (s + s)[1:-1] and
# checking the match at an offset of n / 2).
def equal_string_halves(s):
    # follow-up: the two pieces must be the same string
    n = len(s)
    if n % 2:
        return None
    half, doubled = n // 2, s + s
    for i in range(n):
        if doubled[i:i + half] == doubled[i + half:i + n]:
            return i, (i + half) % n
    return None


if __name__ == "__main__":
    # DRDR: one cut between the halves works
    assert single_cut("DRDR") == 2
    cuts = necklace_two_cuts("DRDR")
    assert cuts is not None and cuts[2].count("D") == 1
    # DDRR needs two cuts on the circle: DR + DR
    assert single_cut("DDRR") is None
    i, j, a, b = necklace_two_cuts("DDRR")
    assert sorted(a) == sorted(b) == ["D", "R"]
    # odd totals have no answer
    assert necklace_two_cuts("DDR") is None
    assert necklace_two_cuts("DRDRD" + "R") is None
    # equal strings: DRDR is DR + DR, DDRR is not
    assert equal_string_halves("DRDR") is not None
    assert equal_string_halves("DDRR") is None
```

NOTES FROM CANDIDATES

Why 2 cuts always suffice: think of a running balance cnt(D) - cnt(R); over the full string it returns to 0. Any prefix splits the necklace into two arcs whose imbalances sum to 0; if the imbalance at some prefix is already 0, one cut suffices, otherwise a sliding window of length n/2 proves a 2-cut works.
Sliding-window approach: fix one cut at index i, look for j such that the window [i, j) has half the Ds and half the Rs. As i advances, j moves monotonically.
Don't get distracted by the necklace framing; once unwrapped onto a line of length 2n and constrained to half-length windows, the problem is standard.
Verbalize the 2-cut bound out loud before coding — interviewers ask why.

PREPARATION

Practice the running-balance technique on +1/-1 arrays (the contiguous-array and subarray-sums-divisible-by-k families).
Drill sliding-window-on-circular-array unrolling (duplicate the string s+s and use a window of length n).

STUDY HERE: topic 2, Arrays, Hashing and Two Pointers

HARD | IN APP LC 1044 | PracHub | Technical Screen | 38 of 38

Find longest duplicated substring

POSTED 2025-12-08 | 316 DAYS AGO | OLDER

STUDY: TALK | ALL TIME 30.5 | 30 DAYS 0 | PRIORITY 122/228

ABOUT THIS QUESTION

Topic: Arrays, Hashing and Two Pointers

Optimal answer from: the interview site's own solution

Also in your app: LeetCode #1044 Longest Duplicate Substring

LeetCode match: LC 1044 Longest Duplicate Substring

Posted on PracHub: 2025-12-08 (updated 2026-07-09)

Original: https://prachub.com/coding-questions/find-longest-duplicated-substring

QUESTION

You are given a string **s** consisting of lowercase English letters. A **substring** of **s** is any contiguous sequence of characters, i.e., `s[i..j)` for `0 <= i < j <= len(s)`. Design an algorithm to find **any one** of the longest substrings that appears **at least twice** in **s**. The two occurrences may overlap. If no substring appears at least twice, return the empty string.

Requirements

- Input: a string **s** of length **n** (e.g., `1 <= n <= 2 × 10^5`).
- Output: a string representing one of the longest duplicated substrings.
- Aim for an algorithm substantially better than the naive `O(n^2)` substring comparison approach.

Examples

- `s = "banana"`
- Possible duplicated substrings: "a", "an", "ana" - Longest length is 3 ("ana"), so acceptable outputs include `"ana"`.
- `s = "abcd"`
- No duplicated substring, so output should be `""` (empty string). Describe your algorithm, its time and space complexity, and how it scales for large **n**.

APPROACH

Binary search on the substring length; check each length with a Rabin-Karp rolling hash plus a set (verify on collision). Alternative: suffix array + LCP. O(n log n).

HOW TO THINK ABOUT IT

If a substring of length L repeats, then a substring of every shorter length repeats too, so 'is there a duplicate of length L' is monotone and the answer length can be binary searched. Each check slides a window of length L over the string with a rolling hash and looks for a repeated hash, verifying the match to guard against collisions.

HOW TO THINK ABOUT IT

Basis: LeetCode 1044, Longest Duplicate Substring (binary search on the length with a rolling hash).

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute(s):
    n = len(s)
    for L in range(n - 1, 0, -1):
        seen = set()
        for i in range(n - L + 1):
            sub = s[i:i + L]
            if sub in seen:
                return sub
            seen.add(sub)
    return ""
```

TIME	SPACE
O(n^3)	O(n^2)

WHY IT IS SLOW

Each length builds up to n substrings of up to n characters: O(n^3) time and O(n^2) space. If a duplicate of length L exists then so does one of every shorter length, so binary search on L with a rolling hash is O(n log n).

OPTIMAL SOLUTION · Binary search on the length with a rolling hash (Rabin-Karp)

TIME	SPACE
O(n log n) expected	O(n)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Avoid hash collisions entirely.

Use a suffix array with the LCP array: the longest duplicate is the largest LCP between neighbours in suffix order. Naive construction (sorting suffixes) is O(n^2 log n); real suffix array algorithms are O(n log n) or O(n).

FOLLOW-UP 2 The substring must appear at least k times, or must not overlap.

At least k times: count equal hashes in the check (need k of them). Non-overlapping: when two windows match, require their start positions to differ by at least L.

FOLLOW-UP 3 The string is huge and does not fit in memory.

Hash windows on disk in passes, or use a suffix automaton built online; for approximate answers use minhash sketches. State that exact answers need memory proportional to the number of distinct windows. **Cost:** O(n log n) time and O(n) memory for the hashes; on disk the hashing passes multiply the I/O by about log n.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Find longest duplicated substring
#
# You are given a string s consisting of lowercase English
# letters. A **substring** of s is any contiguous sequence
# of characters, i.e., s[i..j) for 0 <= i < j <= len(s).
# Design an algorithm to find **any one** of the longest
# substrings that appears **at least twice** in s. The two
# occurrences may overlap. If no substring appears at
# least twice, return the empty string. ### Requirements -
# Input: a string s of length n (e.g., 1 <= n <= 2 × 10^5)
.
# - Output: a string representing one of the longest
# duplicated substrings. - Aim for an algorithm
# substantially better than the naive O(n^2) substring
# comparison approach. ### Examples - s = "banana" -
# Possible duplicated substrings: "a", "an", "ana" -
# Longest length is 3 ("ana"), so acceptable outputs
# include "ana". - s = "abcd" - No duplicated substring,
# so output should be "" (empty string).
#
# FOLLOW-UPS (answered below, in this order):
# 1. Avoid hash collisions entirely.
# 2. The substring must appear at least k times, or must
# not overlap.
# 3. The string is huge and does not fit in memory.
#
# INTERVIEW NOTES
# If a substring of length L repeats, then a substring of
# every shorter length repeats too, so 'is there a
# duplicate of length L' is monotone and the answer length
# can be binary searched. Each check slides a window of
# length L over the string with a rolling hash and looks
# for a repeated hash, verifying the match to guard
# against collisions.
#
# BRUTE FORCE (say it first): Check every length from
# longest down with a set of substrings
# For each length from n - 1 down, put all substrings of
# that length in a set until one repeats. Correct and the
# oracle; every check builds up to n substrings of length
# up to n. Time O(n^3). Space O(n^2).


# BRUTE FORCE (say it first): Check every length from
# longest down with a set of substrings | time O(n^3)
def longest_dup_brute(s):
```

```
    for L in range(len(s) - 1, 0, -1):
        seen = set()
        for i in range(len(s) - L + 1):
            sub = s[i : i + L]
            if sub in seen:
                return sub
            seen.add(sub)
    return ""


# HELPERS for the base question
MOD = (1 << 61) - 1


BASE = 131


def find_length(s, length):
    # a repeated window of this length? (rolling hash)
    codes = [ord(c) for c in s]
    power = pow(BASE, length, MOD)
    h = 0
    for i in range(length):
        h = (h * BASE + codes[i]) % MOD
    seen = {h: [0]}
    for i in range(1, len(s) - length + 1):
        # slide the window: drop the left, add the right
        h = (h * BASE - codes[i - 1] * power
             + codes[i + length - 1]) % MOD
        # a hash match may be a collision: compare the
        # text
        for j in seen.get(h, ()):
            if s[j:j + length] == s[i:i + length]:
                return s[i:i + length]
        seen.setdefault(h, []).append(i)
    return None


# BASE QUESTION (optimal): Binary search on the length
# with a rolling hash (Rabin-Karp)
# Idea: For a fixed length L compute the polynomial hash
# of every window in O(1) each by sliding it, store hashes
# with their start positions, and when a hash repeats
# compare the actual substrings (to rule out a collision).
def longest_dup(s):
```

```
    # binary search the length: if length L repeats, so do
    # all shorter lengths
    lo, hi, best = 1, len(s) - 1, ""
    while lo <= hi:
        mid = (lo + hi) // 2
        found = find_length(s, mid)
        if found:
            best, lo = found, mid + 1
        else:
            hi = mid - 1
    return best


# FOLLOW-UP 1: Avoid hash collisions entirely.
# Answer: Use a suffix array with the LCP array: the
# longest duplicate is the largest LCP between neighbours
# in suffix order. Naive construction (sorting suffixes)
# is O(n^2 log n); real suffix array algorithms are O(n
# log n) or O(n).
def longest_dup_suffix(s):
    # no hashing: sort the suffixes, compare neighbours
    # (the
    # simple O(n^2 log n) version of a suffix array)
    suffixes = sorted(s[i:] for i in range(len(s)))
    best = ""
    for a, b in zip(suffixes, suffixes[1:]):
        k = 0
        while k < min(len(a), len(b)) and a[k] == b[k]:
            k += 1
        if k > len(best):
            best = a[:k]
    return best
```

```
# FOLLOW-UP 2: The substring must appear at least k times,
# or must not overlap.
# Answer: At least k times: count equal hashes in the
# check (need k of them). Non-overlapping: when two
# windows match, require their start positions to differ
# by at least L.
def longest_repeated(s, times=2, non_overlapping=False):
    # at least `times` occurrences; optionally not
    # overlapping
    best = ""
    n = len(s)
```

```
    for length in range(1, n):
        spots = {}
        for i in range(n - length + 1):
            piece = s[i:i + length]
            spots.setdefault(piece, []).append(i)
        ok = None
        for text, starts in spots.items():
            if non_overlapping:
                # greedy: count starts at least `length`
                # apart
                taken, last = 0, -length
                for p in starts:
                    if p - last >= length:
                        taken, last = taken + 1, p
            else:
                taken = len(starts)
            if taken >= times:
                ok = text
                break
        if ok is None:
            break
        best = ok
    return best


if __name__ == "__main__":
    assert longest_dup("banana") in ("ana",)
    assert longest_dup("abcd") == ""
    assert len(longest_dup("aaaaa")) == 4
    assert len(longest_dup_suffix("banana")) == 3
    assert longest_dup_suffix("abcd") == ""
    for s in ("abracadabra", "mississippi", "abab"):
        fast = len(longest_dup(s))
        assert fast == len(longest_dup_suffix(s))
    # three occurrences needed
    assert longest_repeated("banana", 3) == "a"
    # overlapping copies are not allowed: "aa" cannot fit
    # in aaaa
    assert len(longest_repeated("aaaa", 2, True)) == 2
    assert len(longest_repeated("aaaa", 2)) == 3
```

FUNCTION AND TEST CASES

Signature

```
def solution(s) -> str:
```

Sample call

```
solution('banana')
```

Test cases

Input	Expected
`('banana',)`	`'ana'`
`('abcd',)`	`''`